Sicurezza
Sapienza Università di Roma
A.A 2024-2025

Parte del corso di Sicurezza del Prof. D'Elia (2/2)

Indice

1 Software security **8**
- 1.1 Perché è importante la sicurezza informatica? . . . 8
- 1.2 Sicurezza vs. Ingegneria del software . . . 8
- 1.3 Sicurezza del software . . . 9
  - 1.3.1 Crollo del ponte . . . 9

2 Basic properties and principles **9**
- 2.1 La Triade CIA . . . 9
- 2.2 Altre Proprietà della Sicurezza . . . 10
  - 2.2.1 Isolamento (Isolation) . . . 10
  - 2.2.2 Principio del Minimo Privilegio (Principle of Least Privilege) . . . 10
  - 2.2.3 Compartmentalizzazione (Compartmentalization) . . . 10
- 2.3 Modello di Minaccia (Threat Model) . . . 11
  - 2.3.1 Esempio: Utente di rete . . . 11
  - 2.3.2 Esempio: Utente Snooping . . . 11
  - 2.3.3 Esempio: Utente Co-locato . . . 12
- 2.4 Threat-driven design . . . 12
- 2.5 Ciclo di vita del software sicuro . . . 13
  - 2.5.1 Misure di sicurezza in Ubuntu . . . 13
  - 2.5.2 Ingegneria del Software (Software Engineering - SE) . . . 13
- 2.6 Passaggi del ciclo di vita del software sicuro . . . 14
  - 2.6.1 Definizione dei requisiti . . . 14
  - 2.6.2 Progettazione (Design) . . . 14
  - 2.6.3 Implementazione . . . 15
  - 2.6.4 Testing . . . 15
  - 2.6.5 Analisi della sicurezza . . . 15
  - 2.6.6 Aggiornamenti e patching . . . 15
  - 2.6.7 Conclusioni . . . 16

3 Programs fail **17**
  - 3.0.1 Dal difetto al guasto . . . 17
  - 3.0.2 Debugging di un guasto . . . 17
  - 3.0.3 Problem 1: Program state . . . 18
  - 3.0.4 Problem 2: Defects and failures . . . 18
  - 3.0.5 Problema 2: Difetti e guasti . . . 18
  - 3.0.6 Il Principio del Traffico . . . 18
- 3.1 Approaching software testing . . . 19
  - 3.1.1 Testing nei libri di Ingegneria del Software . . . 19
  - 3.1.2 Testing works outwards . . . 19

3.1.3 Chi si occupa del testing? . . . 19
3.1.4 La cultura della ricerca nell'Ingegneria del Software . . . 20
3.1.5 La cultura della ricerca sulla Sicurezza . . . 20
3.1.6 Ma io sono un ingegnere, non (ancora) un ricercatore?!? . . . 20
3.2 Methodologies and techniques . . . 20
3.2.1 Test statici e dinamici . . . 20
3.2.2 Tecniche di test statici . . . 21
3.2.3 Tecniche di test dinamico . . . 21
3.2.4 Distinzione tra test dinamici: Black-box e White-box . . . 21
3.2.5 Tecniche di test dinamico: esempio di black-box . . . 22
3.2.6 Tecniche di test dinamico: esempio di white-box . . . 22
3.2.7 Tecniche di Testing Dinamico . . . 23
3.2.8 Ingredienti per le Tecniche di Testing Moderne . . . 23
3.3 Bug and vulnerabilities . . . 23
3.3.1 Terminologia . . . 23
3.3.2 Da Bug a Exploit . . . 23
3.3.3 Gravità delle Vulnerabilità . . . 23
3.3.4 Comportamento quando si trova una vulnerabilità . . . 24
3.3.5 CVE (Common Vulnerabilities and Exposures) . . . 24
3.3.6 Takeaways . . . 24

4 Memory and type safety 25
4.1 Low-level bugs . . . 25
4.1.1 Dai bug software alle primitive di attacco . . . 25
4.1.2 Il caso più famoso: buffer overflow . . . 25
4.1.3 Richiamo: struttura base dello stack . . . 26
4.1.4 Conseguenze di sicurezza . . . 26
4.2 Memory and type safety . . . 27
4.2.1 Memory safety . . . 27
4.2.2 Spatial safety . . . 27
4.2.3 Temporal Safety . . . 28
4.2.4 Type Safety . . . 28
4.2.5 Violazione della sicurezza dei tipi e della memoria . . . 29
4.2.6 Come ottenere sicurezza . . . 29
4.2.7 Sicurezza della memoria in C e C++ . . . 30
4.3 Consequences of safety violations . . . 31
4.3.1 Stack Buffer Overflow . . . 31
4.3.2 Heap Buffer Overflow . . . 31
4.3.3 Read Overflow . . . 31
4.3.4 Heartbleed . . . 32
4.3.5 Type Confusion . . . 32
4.4 More subtle CdSes . . . 33
4.4.1 Uso di memoria non inizializzata (Use of uninitialized memory) . . . 33
4.4.2 Uso dopo il 'free' (Use after free) . . . 33
4.4.3 Overflow intero (Integer overflow) . . . 34
4.4.4 Stringhe di formato (Format strings) . . . 34
4.4.5 Altri esempi di uso errato delle stringhe di formato . . . 35

5 Sanitizers 36
5.1 Silent bugs . . . 36
5.1.1 Testing per errori . . . 36
5.1.2 Testing per problemi di sicurezza nel codice di basso livello . . . 36
5.1.3 Rilevamento degli errori (fault detection) . . . 36
5.1.4 Recall: Overflow in HeartBleed . . . 37
5.1.5 Sanitizers . . . 37
5.2 AddressSanitizer (ASan) . . . 38

5.2.1 Redzones . . . 39
5.2.2 Instrumentation for Checks . . . 40
5.2.3 Memory layout is affected . . . 40
5.2.4 ASan è qualcosa di nuovo? . . . 40
5.2.5 Stack variables . . . 41
5.2.6 Limitazioni di ASan . . . 41
5.2.7 Temporal safety . . . 42
5.2.8 Diagnosing a violation . . . 42
5.2.9 Availability . . . 43
5.3 Undefined Behavior Sanitizer (UBSan) . . . 43
5.3.1 Overflow di interi con segno . . . 43
5.3.2 Misaligned pointer . . . 44
5.3.3 UBSan operation . . . 44
5.4 Other sanitizers . . . 45
5.4.1 MemorySanitizer . . . 45
5.4.2 ThreadSanitizer . . . 45
5.4.3 LeakSanitizer . . . 46
5.4.4 Altri sanitizers . . . 46
5.4.5 Conclusioni . . . 46

6 Laboratorio **47**
6.1 Esempi di funzioni . . . 47
6.1.1 Heap Out of Bound . . . 47
6.1.2 Espansione del buffer . . . 48
6.1.3 Stack Buffer Overflow (Stack OOB) . . . 48
6.1.4 Analisi di accessi oltre la redzone . . . 50
6.1.5 Debug con GDB e sanitizer . . . 50
6.1.6 Stack Out of Bounds (Stack OOB) . . . 51
6.1.7 Global OOB . . . 52
6.1.8 Memory Leak . . . 52
6.2 Libxml2 . . . 53
6.2.1 Download e compilazione della versione 2.9.2 di Libxml2 . . . 53
6.2.2 Test harness e test suite sui file XML . . . 53
6.2.3 Differenze harness e seed . . . 54
6.2.4 Analisi e risoluzione dei bug . . . 55
6.2.5 Considerazioni finali . . . 55
6.3 Mitigations . . . 57
6.4 Bugs vs mitigations . . . 57
6.4.1 Il software rimarrà vulnerabile . . . 57
6.4.2 Distruggere lo stack per divertimento e profitto . . . 57
6.4.3 Esempio di vulnerabilità: buffer overflow . . . 57
6.5 DEP . . . 59
6.5.1 Processo in esecuzione senza DEP . . . 59
6.5.2 DEP è stato solo il primo passo . . . 59
6.5.3 Perché DEP non è sufficiente? . . . 60
6.6 Address randomization . . . 60
6.6.1 Attacco ret2libc . . . 60
6.6.2 Code Reuse Attacks: gadget e ROP . . . 60
6.6.3 Esempio di catena ROP . . . 61
6.6.4 Scoperta dei gadget . . . 61
6.6.5 Mitigazione: ASLR . . . 61
6.6.6 Effetti dell'ASLR . . . 61
6.6.7 Effetti della randomizzazione . . . 62
6.7 PIE/PIC . . . 62
6.7.1 PIE e PIC: Randomizzazione del codice . . . 62
6.7.2 Position-Independent Code (PIC) . . . 62

6.7.3 PLT . . . 63
6.7.4 PIC su x64 . . . 63
6.7.5 Implicazioni per la sicurezza . . . 64
6.7.6 Position-Independent Executable (PIE) . . . 64
6.7.7 Verso più randomizzazione? . . . 64
6.8 Altri meccanismi di protezione del compilatore . . . 65
6.8.1 Stack Canaries . . . 65
6.8.2 Fortified source . . . 65
6.8.3 Protezione dei trasferimenti di controllo . . . 66
6.8.4 Protezione dai trasferimenti di controllo . . . 66

7 Fuzzing **68**
7.1 Coverage da una test suite . . . 68
7.1.1 Branch vs path coverage . . . 68
7.1.2 Generazione automatica di test case . . . 68
7.1.3 Symbolic Execution . . . 69
7.2 Fuzz testing . . . 69
7.2.1 Fuzz Testing . . . 69
7.2.2 Fuzz Testing vs. Testing Standard . . . 70
7.2.3 Origini del Fuzzing . . . 70
7.2.4 Come Funziona il Fuzzing . . . 70
7.2.5 Considerazioni sulla Sicurezza . . . 70
7.2.6 Seed Corpus . . . 71
7.2.7 Mutazione degli Input . . . 71
7.2.8 Esecuzione del Target . . . 71
7.2.9 Feedback di Coverage . . . 71
7.2.10 Ottimizzazioni del runtime . . . 71
7.2.11 Casi d'uso e Considerazioni . . . 72
7.2.12 Evoluzione del Fuzzing . . . 72
7.3 Fuzzing: Tecniche e Sfide . . . 72
7.3.1 Runtime Operation e Ottimizzazione . . . 73
7.3.2 Harness per il Fuzzing . . . 73
7.3.3 Monitoraggio del Programma . . . 74
7.3.4 Coverage Feedback . . . 74
7.3.5 Coverage Map . . . 74
7.4 Fuzzer in azione . . . 76
7.4.1 American Fuzzy Lop (AFL) . . . 76
7.4.2 AFL++ . . . 76
7.4.3 Metriche di performance . . . 77
7.4.4 Output di una campagna di fuzzing con AFL++ . . . 77
7.4.5 Analisi dei crash (bug triaging) . . . 78
7.5 Suggerimenti pratici e metodi . . . 80
7.5.1 Scelta dei seed appropriati . . . 80
7.5.2 Esecuzione di più prove . . . 80
7.5.3 Applicazioni di rete . . . 81
7.5.4 Minimizzazione della coda e dei seed . . . 81
7.5.5 Fuzzing con sanitizer . . . 82
7.5.6 Vantaggi del fuzzing guidato dalla copertura . . . 82

8 Laboratorio Fuzzing **83**
8.1 Introduzione al Fuzzing . . . 83
8.2 Installazione di AFL++ . . . 83
8.2.1 Installazione della versione più recente . . . 83
8.2.2 Installazione tramite repository . . . 83
8.2.3 Configurazione del kernel per il fuzzing . . . 83
8.3 Fuzzing di xpdf . . . 84

8.3.1 Download e estrazione di xpdf . . . 84
8.3.2 Compilazione con strumentazione per fuzzing . . . 84
8.3.3 Preparazione dei seed per il fuzzing . . . 85
8.3.4 Avvio della campagna di fuzzing . . . 85
8.4 Interpretazione dell'interfaccia di AFL++ . . . 85
8.5 Struttura della coda di AFL++ . . . 86
8.6 Strategie di mutazione . . . 87
8.7 Analisi dei crash . . . 87
8.7.1 Tempo di esposizione (TTE) . . . 88
8.7.2 Preparazione per il debug . . . 88
8.7.3 Debug dei crash con GDB . . . 88
8.7.4 Clustering dei crash . . . 89
8.8 Utilizzo di sanitizer per il debug . . . 89
8.9 Considerazioni sulla performance . . . 90
8.10 Fuzzing di Network Services . . . 90
8.11 Segnalazione etica dei bug . . . 91

9 Altro laboratorio fuzzing 92
9.1 Fuzzing con AddressSanitizer . . . 92
9.1.1 Motivazioni . . . 92
9.1.2 Compilazione con ASan . . . 92
9.1.3 Procedura di compilazione . . . 93
9.1.4 Verifica dell'instrumentazione . . . 93
9.1.5 Impatto sulle Performance . . . 93
9.1.6 Lancio del Fuzzing . . . 93
9.2 Clustering dei Crashes . . . 94
9.2.1 Problema della Deduplificazione . . . 94
9.2.2 Binario per il Triage . . . 94
9.2.3 AFLTriage . . . 95
9.2.4 Utilizzo di AFLTriage . . . 95
9.2.5 Versione migliorata . . . 95
9.3 Analisi della Code Coverage . . . 95
9.3.1 Coverage con AFL++ . . . 95
9.3.2 Interpretazione dettagliata di afl-showmap . . . 96
9.3.3 Salvare Identity degli Edges . . . 96
9.4 Coverage Dettagliata con LLVM . . . 97
9.4.1 Compilazione per Coverage . . . 97
9.4.2 Script per Raccolta Coverage . . . 97
9.4.3 Generazione report . . . 97
9.5 Progressione del fuzzer nel tempo . . . 98
9.5.1 AFL-Plot . . . 98
9.5.2 Grafico di progressione degli edges . . . 98
9.5.3 Interpretazione dettagliata dei grafici AFL-Plot . . . 100
9.5.4 Analisi di questi grafici . . . 100
9.5.5 Significato della progressione . . . 101
9.6 Altri grafici di AFL-Plot . . . 101
9.6.1 Corpus count e pending items . . . 101
9.6.2 Crashes e Hangs nel Tempo . . . 102
9.6.3 Execution speed (exec/sec) . . . 102
9.6.4 Interpretazione dei grafici nel contesto ASan . . . 102
9.7 Considerazioni Pratiche . . . 102
9.7.1 Performance con ASan . . . 102
9.7.2 Confronto tra Approcci . . . 102
9.7.3 Best practices . . . 103
9.7.4 Troubleshooting comune . . . 103
9.8 Resume e Gestione delle Code . . . 103

9.8.1 Riavvio di Campagne di Fuzzing . . . 103
9.8.2 Warning sui cicli incompleti . . . 103
9.8.3 Dettagli sui profili LLVM . . . 103
9.8.4 Timeout negli script di coverage . . . 104
9.9 Confronto coverage tra campagne . . . 104
9.10 Troubleshooting AFLTriage . . . 104
9.10.1 Limitazioni attuali . . . 104
9.11 Conclusioni . . . 105

10 Domande d'esame **106**
10.1 Domanda 1: Clustering dei Crashes . . . 106
10.1.1 Risposta corretta . . . 106
10.1.2 Errori comuni da evitare . . . 106
10.2 Domanda 2: Funzionamento dei sanitizers: redzone di AddressSanitizer . . . 106
10.2.1 Risposta corretta . . . 106
10.2.2 Casi particolari e limitazioni (Bonus) . . . 107
10.2.3 Errori comuni da evitare . . . 108
10.3 Domanda 3: Use of Uninitialized Memory (UUM) . . . 108
10.3.1 Risposta corretta . . . 108

11 Tabelle riepilogative **110**
11.1 Sanitizers . . . 110
11.2 Fuzzer . . . 111

Disclaimer

Questi appunti potrebbero contenere imprecisioni o errori. Verificare sempre con il materiale ufficiale del corso. L'autore non si assume responsabilità per eventuali problematiche.

1 Software security

1.1 Perché è importante la sicurezza informatica?

La sicurezza informatica è fondamentale perché ha un impatto diretto sulla **vita quotidiana** di tutti. Ignorare la sicurezza può portare a violazioni dei dati, perdite economiche e danni alla reputazione. Ecco perché diversi attori devono preoccuparsene:

- **Utenti finali**: Devono prendere decisioni sicure per proteggere i propri dati personali e la privacy.
- **Sviluppatori**: Hanno la responsabilità di progettare e implementare sistemi sicuri fin dalle prime fasi dello sviluppo (*security by design*).
- **Ricercatori di sicurezza**: Identificano e mitigano le vulnerabilità prima che possano essere sfruttate. Ad esempio, un avversario può usare un bug che ha il codice per effettuare un attacco: gli utenti non posso fare nulla, gli sviluppatori sviluppano in base a delle guide (magari non dando alla sicurezza la giusta priorità), c'è bisogno di figure esperte che sappiano come gestire e come creare direttamente codice sicuro.
- **Attori malevoli (Threat actors)**: Cercano di sfruttare le vulnerabilità per ottenere guadagni illeciti o causare danni.
- **Aziende e organizzazioni**: Devono proteggere i loro asset, i dati sensibili e la continuità operativa da attacchi informatici. Ad esempio chiedere riscatti con i dati sensibili copiati per non divulgarli.

La sicurezza informatica non può più essere un ripensamento secondario, come avveniva in passato. Deve essere una priorità **integrata in ogni fase** dello sviluppo e della gestione dei sistemi informatici.

1.2 Sicurezza vs. Ingegneria del software

L'ingegneria del software si concentra principalmente su diversi aspetti fondamentali nello sviluppo di applicazioni e sistemi informatici, tra cui:

- **Affidabilità (Dependability)**: Il sistema deve funzionare in modo prevedibile e senza errori critici. Ad esempio, è considerato affidabile se fornisce risposte corrette almeno nel 99% dei casi. L'affidabilità include sia la correttezza (*correctness*) che la disponibilità (*reliability*).
- **Efficienza (Efficiency)**: Le risorse devono essere utilizzate in modo ottimale per garantire prestazioni elevate.
- **Manutenibilità (Maintainability)**: Il codice deve essere strutturato in modo da facilitare correzioni, aggiornamenti e miglioramenti.
- **Portabilità e riusabilità (Portability/Reusability)**: Il software dovrebbe poter essere adattato a diversi ambienti e riutilizzato in più contesti.
- **Usabilità (Usability)**: L'interfaccia e l'interazione devono essere intuitive e accessibili agli utenti.

La sicurezza informatica fa parte del processo di sviluppo del software, ma tradizionalmente è stata considerata secondaria e spesso limitata alla fase di test. Tuttavia, un approccio moderno alla sicurezza (*security by design*) prevede che essa sia integrata sin dalle prime fasi della progettazione e dello sviluppo **e non alla fine e in maniera secondaria**, riducendo così il rischio di vulnerabilità e attacchi informatici.

1.3 Sicurezza del software

> *"L'obiettivo della sicurezza del software è consentire qualsiasi uso previsto del software, ma prevenire qualsiasi uso non intenzionale."*
> (M. Payer - SS3P book)

Ciò implica l'applicazione e l'imposizione di politiche di sicurezza attraverso meccanismi che regolano l'accesso ai dati e alle risorse.

- **Politiche di sicurezza (Security Policies)**: Definiscono **cosa** è permesso e cosa è vietato.
- **Meccanismi di sicurezza (Security Mechanisms)**: Implementano **come** far rispettare queste politiche.

Per creare una base comune di comprensione, è fondamentale definire e discutere alcuni principi fondamentali della sicurezza. Ma perché la sicurezza è così complessa?
I principali fattori che la rendono difficile da garantire sono:

- **Fattore umano**: Gli errori umani e i comportamenti non sicuri rappresentano una delle principali vulnerabilità.
- **Concetto dell'anello più debole(*)**: Un sistema è sicuro solo quanto il suo componente più vulnerabile.
- **Prestazioni**: Le misure di sicurezza possono introdurre overhead che influiscono sulle performance.
- **Usabilità**: Un sistema troppo sicuro ma complesso da usare porta gli utenti a trovare soluzioni meno sicure per aggirare le restrizioni. Ad esempio in un ospedale, la priorità non è cambiare la password ogni due mesi, occorre quindi adottare altre tecniche.
- **Costi (tempo/denaro)**: Implementare e mantenere un sistema sicuro richiede investimenti significativi.

(*) Gli attaccanti mirano spesso all'anello più debole, mentre i difensori devono proteggere l'intero sistema, affrontando tutti i potenziali vettori d'attacco.

1.3.1 Crollo del ponte

Il crollo di un ponte è un problema di **safety**, non di sicurezza informatica. Tuttavia, nella maggior parte dei problemi di sicurezza (circa il 95%), ma come nella realtà di molte ingegnerie, è presente un elemento umano.
Il vero problema è che una piccola percentuale di persone contribuisce in modo sproporzionato agli incidenti di sicurezza. Spesso ciò accade perché alcuni sviluppatori o progettisti creano sistemi senza considerare adeguatamente la sicurezza, lasciando vulnerabilità sfruttabili dagli attaccanti.

2 Basic properties and principles

2.1 La Triade CIA

La sicurezza delle informazioni si basa su tre principi fondamentali, noti come la **triade CIA** (*riservatezza (Confidentiality), integrità (Integrity), disponibilità (Availability)*).
Sebbene questo **modello sia parzialmente datato e non esaustivo**, rimane un riferimento accettato per valutare la sicurezza di software e sistemi.

- **Riservatezza (Confidentiality)**: Un attaccante non deve essere in grado di accedere o recuperare dati protetti senza autorizzazione.
- **Integrità (Integrity)**: Un attaccante non deve poter modificare i dati protetti senza essere rilevato.

- **Disponibilità (Availability)**: Un attaccante non deve essere in grado di impedire il corretto funzionamento del sistema o l'accesso ai dati.

Le definizioni un più formali sono:

- **Riservatezza (Confidentiality)**: L'accesso alle informazioni è limitato solo a entità autorizzate.
  - *Esempio*: Una vulnerabilità software consente a utenti non autorizzati di accedere a dati protetti.
- **Integrità (Integrity)**: La modifica delle informazioni è consentita solo a entità autorizzate.
  - *Esempio*: Il sistema operativo impedisce ai processi di scrivere negli spazi di indirizzamento di altri processi.
  - Il **Confidential Computing** è una tecnica basata su hardware che protegge l'elaborazione dei dati in uso. Consiste nell'eseguire un processo all'interno di una memoria cifrata, accessibile solo dalla CPU che lo sta eseguendo, grazie a una chiave crittografica integrata. Questo approccio garantisce che i dati rimangano protetti anche in presenza di un sistema operativo compromesso o di attacchi da altri processi in esecuzione sulla stessa macchina.
- **Disponibilità (Availability)**: Il servizio deve rimanere accessibile agli utenti legittimi.
  - *Esempio*: Il sistema operativo assegna equamente il tempo di CPU ai processi, evitando starvation.

2.2 Altre Proprietà della Sicurezza

2.2.1 Isolamento (Isolation)

L'isolamento separa i componenti di un sistema, limitando le loro interazioni a interfacce e primitive ben definite, in modo da poterle analizzare e verificare.
Tutte le forme di isolamento richiedono un **security monitor**, un componente con privilegi più elevati rispetto ai componenti isolati. Questo monitor rileva e blocca eventuali violazioni.
Hai già incontrato questo concetto nello studio della memoria virtuale e delle istruzioni privilegiate della CPU, poiché il sistema operativo utilizza entrambi per implementare l'astrazione dei processi.

2.2.2 Principio del Minimo Privilegio (Principle of Least Privilege)

Il principio del minimo privilegio assicura che **ogni componente disponga solo dei privilegi strettamente necessari** per svolgere la propria funzione.
L'isolamento è un prerequisito fondamentale, poiché limita l'accesso a parti non autorizzate del sistema. Questa proprietà costringe un attaccante a operare con i privilegi del componente compromesso, rendendo necessaria un'ulteriore vulnerabilità per ottenere un'**escalation** di privilegi. Quindi l'attaccante prima deve trovare il "buco", ma poi deve anche salire di privilegi, il che rende l'attacco più complicato.

2.2.3 Compartmentalizzazione (Compartmentalization)

La compartmentalizzazione si basa sui principi di **isolamento** e **minimo privilegio**. Un sistema complesso viene suddiviso in componenti più piccoli, ognuno dei quali comunica con gli altri attraverso protocolli ben definiti, rimanendo isolato altrimenti.
I permessi e la conformità ai protocolli possono essere verificati ai confini tra compartimenti, rendendo più difficile per un attaccante comprometterli tutti.

Browser Un esempio concreto sono i moderni browser web, che eseguono componenti separati con privilegi minimi e in isolamento reciproco. Ad esempio, non ha senso mescolare nella stessa pagina la parte visiva e il codice JavaScript. Questa compartmentalizzazione è ben visibile quando si ha il browser aperto e si hanno tanti processi del broswer.

2.3 Modello di Minaccia (Threat Model)

Il **modello di minaccia (Threat Model)** è un processo fondamentale per identificare ed elencare le minacce potenziali (ad esempio, vulnerabilità strutturali, assenza di salvaguardie) e per dare **priorità alle contromisure**.

> *Contro quale tipo di avversario stai proteggendo il tuo sistema?*

Il modello risultante deve **riflettere la realtà**, altrimenti l'analisi del rischio sarà errata. Magari delle mitigazioni costano troppo rispetto al beneficio che ne portano.

Confini della minaccia Il **modello di minaccia** definisce i confini chiari di ciò che un attaccante può fare a un sistema ed è un prerequisito per ragionare su attacchi o difese.
Ogni minaccia nel modello viene gestita mediante l'adozione di una contromisura o, in alternativa, accettando il rischio (quando il costo della contromisura supera l'impatto potenziale dell'attacco).
Il modello di minaccia è di importanza cruciale!

2.3.1 Esempio: Utente di rete

- Un utente (anonimo) che può connettersi a un servizio tramite la rete.
- Può:
  - misurare la dimensione e i tempi di richieste e risposte; ad esempio, analizzando i tempi di elaborazione, si possono individuare vulnerabilità sfruttabili.
  - eseguire sessioni parallele.
  - inviare input o messaggi malformati.
  - inviare o eliminare messaggi extra.
- Esempi di attacchi: SQL injection, XSS, CSRF, overflow del buffer/ROP payloads, ...

Nei primi attacchi, gli aggressori inserivano codice arbitrario nel buffer, riuscendo così a eseguire malware. Successivamente, per limitare l'esecuzione di codice esterno, sono stati sviluppati attacchi che sfruttano solo codice presente nello stesso programma. Questo è possibile tramite l'uso di "ROP payloads" (Return-Oriented Programming), che consentono di eseguire comandi controllati senza introdurre codice maligno nel sistema.

2.3.2 Esempio: Utente Snooping

- Utente di Internet sulla stessa rete di altri utenti di un determinato servizio.
- Ad esempio, qualcuno connesso a una rete Wi-Fi non criptata in un caffè.
- In questo caso, l'utente può anche:
  - Leggere o monitorare i messaggi degli altri utenti,
  - Intercettare, duplicare e modificare i messaggi.
- Esempi di attacchi:
  - Hijacking della sessione (e altri furti di dati),
  - Attacchi a canali laterali che violano la privacy,
  - Attacchi di Denial of Service (DoS).

2.3.3 Esempio: Utente Co-locato

- Utente di Internet sulla stessa macchina di altri utenti di un determinato servizio.
- Ad esempio, malware installato sul laptop di un utente.
- In questo caso, l'aggressore può anche:
  - Leggere/scrivere i file dell'utente (ad esempio, i cookie) e la sua memoria (leggere le password salvate in memoria non criptate),
  - Monitorare le sequenze di tasti e altri eventi,
  - Leggere/scrivere il display dell'utente (ad esempio, per fare spoofing).
- Esempi di attacchi:
  - Furto di password (e altre credenziali/segreti).

2.4 Threat-driven design

Una volta definito il tuo modello di minaccia, puoi elaborare come rispondere ad esse. Diversi modelli di minaccia richiedono risposte differenti. Ad esempio, il traffico è sicuro con attaccanti che operano solo a livello di rete: non è necessario criptarlo. Ma oggi, con le reti wireless, non è sicuro assumere che gli attaccanti non possano osservare il traffico degli altri utenti. Il tuo modello di minaccia deve riflettere la realtà! Dove devo criptare? A livello di link (WPA2), di rete (IPSec, quindi VPN) o di applicazione (SSL)? La risposta va tarata in base all'applicazione.

Assunzioni sbagliate Per ribadire: un modello mal scelto porterà a una sicurezza scadente. *Ogni assunzione che fai rappresenta una possibile vulnerabilità* che il tuo avversario potrebbe sfruttare. Un altro esempio: "il traffico criptato non rivela informazioni".

- Non è vero: la dimensione e la distribuzione dei messaggi possono rivelare lo stato dell'applicazione.

Suggerimenti per un buon modello di minaccia Tra le buone pratiche si hanno:

- Confronta il tuo modello con sistemi simili.
- Comprendi gli attacchi passati e i modelli di attacco.
- Permetti al tuo modello di evolversi mettendo in discussione le tue assunzioni ($\rightarrow$ cosa costerebbe una violazione? E posso indurire il sistema contro attacchi più forti?).

Nei sistemi complessi, gli effetti di mitigazione possono provenire da diverse fonti.

Valutazione del rischio La valutazione del rischio consiste nel tenere conto dei costi per la costruzione del sistema, dei costi di una violazione riuscita e nel ponderare la probabilità di una violazione.

2.5 Ciclo di vita del software sicuro

Lo sviluppo di software sicuro è un processo in cui la progettazione iniziale e l'implementazione rappresentano solo il punto di partenza. Fino al ritiro del software, esso viene continuamente esteso e adattato ai requisiti in evoluzione.

Di conseguenza, è necessaria una **costante attività di test**, aggiornamenti e patch, sia per correggere vulnerabilità e bug scoperti, sia per implementare mitigazioni più efficaci nel tempo. L'adozione di un ciclo di vita sicuro del software aiuta a ridurre i rischi e a garantire un livello di sicurezza adeguato durante l'intero periodo di utilizzo del software.

Hyper-Threading di Intel e vulnerabilità L'Hyper-Threading (HT) di Intel permette a un core fisico di eseguire due thread simultaneamente, migliorando le prestazioni nei carichi multi-threaded. Tuttavia, questa condivisione delle risorse ha portato a vulnerabilità critiche.
Un esempio è l'attacco **Spectre**, che sfrutta l'*esecuzione speculativa*: la CPU esegue istruzioni in anticipo per ottimizzare le prestazioni, ma un attaccante può manipolare questo comportamento per accedere a dati sensibili nella cache. Usando tecniche di *cache timing*, è possibile estrarre informazioni da altri processi.
Per mitigare questi attacchi, sono state introdotte patch software e aggiornamenti firmware, mentre alcuni sistemi hanno disabilitato HT per sicurezza.

2.5.1 Misure di sicurezza in Ubuntu

Il sistema operativo Ubuntu ha introdotto nel tempo numerose misure di sicurezza per migliorare la protezione degli utenti e dei loro dati. Di seguito, alcuni esempi dell'evoluzione delle funzionalità di sicurezza[1]:

Caratteristica	**Versione introdotta**	**Versione predefinita**
Aggiornamenti automatici di sicurezza	6.06 (opt-in)	16.04
Hash delle password	SHA-512 (8.10)	Yescrypt (22.04)
Cifratura LVM	6.06 (installer alternativo)	12.10 (installer standard)
ASLR	Stack (6.06)	PIE & VDSO (8.04)
Kernel ASLR	14.10 (opzionale)	16.10 (predefinito)
Filtraggio delle syscall	SECCOMP (8.04)	`seccomp_filter` (11.10)

Tabella 1: Evoluzione delle misure di sicurezza in Ubuntu

2.5.2 Ingegneria del Software (Software Engineering - SE)

L'ingegneria del software è una disciplina che si occupa di metodi e tecniche per lo sviluppo e la manutenzione di sistemi software che:

- si comportano in modo affidabile ed efficiente;
- sono economicamente sostenibili da sviluppare e mantenere;
- soddisfano tutti i requisiti definiti dal cliente.

Perché è importante? Il software può essere costoso da eseguire e può avere impieghi in ambiti critici per la sicurezza, rendendo essenziale un approccio strutturato al suo sviluppo.

[1] Ulteriori dettagli sulle funzionalità di sicurezza storiche di Ubuntu sono disponibili nella documentazione ufficiale: `https://wiki.ubuntu.com/Security/Features/Historical`

Sicurezza e test Lo sviluppo di software sicuro non si limita ai soli requisiti funzionali, ma include anche:

- **Requisiti di sicurezza**, come *politiche di accesso*, *privilegi* e *linee guida di sicurezza*;
- **Piani di test e aggiornamento**, per definire le azioni da intraprendere in caso di nuove vulnerabilità. I problemi si trovano o facendo testing o qualcuno che li segnala, e quindi occorre subito trovare una soluzione

2.6 Passaggi del ciclo di vita del software sicuro

Lo sviluppo di software sicuro è un processo continuo che si articola nelle seguenti fasi:

1. **Definizione dei requisiti**
2. **Progettazione (Design)**
3. **Implementazione**
4. **Testing**
5. **Aggiornamenti e patching**

Questo ciclo viene ripetuto continuamente per garantire la sicurezza del software nel tempo.

2.6.1 Definizione dei requisiti

Nell'ingegneria del software tradizionale, la specifica dei requisiti definisce la funzionalità tangibile, le singole caratteristiche, i formati dei dati e le interazioni con l'ambiente.
Dal punto di vista della sicurezza, è necessario aggiungere:

- **Specifiche di sicurezza**: definizione del modello di minaccia e valutazione del rischio.
- **Specifiche degli asset**: identificazione delle tipologie di dati e delle entità autorizzate ad accedervi.
- **Valutazione ambientale**: analisi del contesto in cui il software verrà distribuito ed eseguito.
- **Casi d'uso e d'abuso**: definizione di scenari legittimi e malevoli per testare la sicurezza.

2.6.2 Progettazione (Design)

Gli aspetti di sicurezza devono essere parte integrante della progettazione e non un'aggiunta tardiva. Il modello di minaccia viene esteso per tenere conto delle scelte architetturali e progettuali del software in sviluppo. I normali documenti di progettazione vengono arricchiti con un **design di sicurezza** dettagliato, che include:

- **attori** coinvolti (utenti benigni e attaccanti),
- **permessi** e capacità di ciascun attore,
- **mitigazioni** per le potenziali vulnerabilità.

L'intero design **viene sottoposto a revisioni iterative** per migliorare progressivamente le garanzie di sicurezza.

2.6.3 Implementazione

Questa fase segue le migliori pratiche di ingegneria del software per garantire codice robusto e sicuro. Alcune delle pratiche consigliate includono:

- utilizzo di sistemi di controllo di versione (ad esempio, `Git`), per vedere lo storico dei cambiamenti o *regression performance*[2];
- definizione di linee guida di codifica (es. pattern comuni, commenti, aggiornamenti di design)[3];
- adozione di un processo formale di revisione del codice, con attenzione alla sicurezza, alle prestazioni e all'efficienza.

2.6.4 Testing

Nel testing tradizionale, l'attenzione è rivolta alla funzionalità e alla regressione del software. Oggi, i sistemi di **Continuous Integration** permettono di testare singoli componenti (unit test) e l'intero programma a ogni commit o a intervalli regolari.
Il testing di sicurezza è diverso dal testing funzionale:

- Il **testing funzionale** verifica il rispetto dei requisiti di performance e comportamento.
- La **sicurezza**, essendo una proprietà astratta, **non è direttamente testabile** con metodi convenzionali.

2.6.5 Analisi della sicurezza

Non tutti i bug causano crash, ma anche quelli ***silenziosi*** *possono rappresentare vulnerabilità gravi.* Le tecniche di analisi della sicurezza aiutano a:

- **automatizzare i test** di sicurezza: spesso non si riescono a fare test a mano, servono strumenti adatti. `Automated` presuppone che l'umano dia un contributo decisivo, `automatic` potenzialmente non ha bisogno di alcun contributo umano. In questo caso per progetti grandi serve quindi *automatic.*
- **individuare bug** che non si manifestano con un crash, si fa *sanitization* per cercare di individuale
- **stimare la sfruttabilità** dei bug scoperti: molto di questo lavoro è ancora manuale, non c'è una metrica o un parametro per "bug critico di sicurezza", ma trovandolo bisogna ragionarci sopra, al momento non si riesce ad automatizzare questa parte.

Un ruolo fondamentale è svolto dal **red Team**, che *analizza il sistema dal punto di vista di un attaccante*, cercando falle sfruttabili nella progettazione o nell'implementazione. Il **blue Team** invece agisce dal punto di vista del difensore.

2.6.6 Aggiornamenti e patching

Le vulnerabilità di sicurezza richiedono processi dedicati per gli aggiornamenti e il rilascio di patch. Idealmente, ogni software dovrebbe disporre di un **team di risposta alla sicurezza**, che gestisce:

- la **ricezione** e l'**analisi** delle segnalazioni di vulnerabilità,
- la **definizione delle priorità** nella risoluzione dei problemi,
- il **coordinamento del rilascio** delle patch[4].

[2] A software regression is a type of software bug where a feature that has worked before stops working.
[3] Per approfondire le linee guida di programmazione sicura, si possono consultare risorse come la Google C++ Style Guide.
[4] Esempio di grave errore è *2024 CrowdStrike-related IT outages*

Gli aggiornamenti automatici sono ormai la norma, e nei sistemi complessi vengono adottate tecniche di **hot patching**, che consentono di applicare correzioni senza riavviare il sistema. Tuttavia, sviluppare un'infrastruttura sicura per la distribuzione degli aggiornamenti rappresenta una sfida complessa.

2.6.7 Conclusioni

- Il software assomiglia a un organismo biologico: vive ed evolve nel tempo.
- La sicurezza deve essere considerata una priorità assoluta durante tutto il ciclo di vita del software.
- La sicurezza del software garantisce gli usi previsti e previene quelli indesiderati.
- I principi fondamentali della sicurezza sono valutati in base a un modello di minaccia.
- Un modello di minaccia inadeguato porta a una sicurezza insufficiente.
- Un ciclo di vita sicuro del software richiede processi ben definiti e un'attenta modellazione delle minacce.

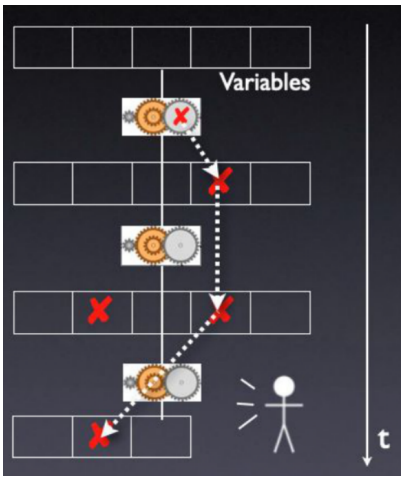
3 Programs fail

[5] Crediti: Andres Zeller, Why Programs Fail.

3.0.1 Dal difetto al guasto

1. Il programmatore introduce un **difetto** (bug), ovvero un errore nel codice.
2. Durante l'esecuzione, il difetto genera un'**infezione**, causando uno stato errato nel sistema.
3. L'infezione si propaga attraverso il sistema, compromettendo altre parti del codice.
4. Se non gestita, l'infezione porta a un **guasto** failure, ovvero un malfunzionamento osservabile.

Per garantire la sicurezza e l'affidabilità del software, è fondamentale individuare e interrompere questa catena di infezione, risalendo alla causa originaria del difetto e correggendola tempestivamente.



3.0.2 Debugging di un guasto

- Ogni **guasto** (failure) può essere ricondotto a una qualche **infezione**, e ogni infezione è causata da un **difetto**.
- Il **debugging** consiste nel *correlare un guasto specifico al difetto* che lo ha causato e nell'eliminare tale difetto.

Il processo di debugging è fondamentale per garantire che il software funzioni correttamente, identificando e rimuovendo la causa alla radice del malfunzionamento.

La ricerca deve esplorare spazio e tempo Le variabili possono sembrare coerenti, ma alcune potrebbero essere corrotte in modo silenzioso. È quindi fondamentale individuare tutte le variabili rilevanti per l'esecuzione del programma, analizzandole in dettaglio sia nello **spazio** (memoria) che nel **tempo** (evoluzione dello stato del programma).

[5] PDF: 02-sw-testing

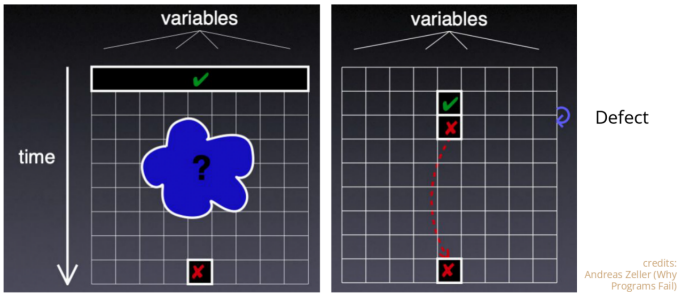


Figura 1: Esempio di ricerca per spazio e tempo

3.0.3 Problem 1: Program state

Per un debugging efficace, è necessario comprendere le complessità dello **stato del programma**, come gestione della memoria, visibilità delle variabili, processi, input esterni.
Una corretta analisi dello stato aiuta a individuare difetti difficili da rilevare, come race condition, buffer overflow e memory leak.

3.0.4 Problem 2: Defects and failures

"Testing can only show the presence of errors, not their absence" (E. Dijkstra, 1972) Not every defect causes a failure. How do we detect silent issues?

Quando si testa e non si trovano errori **non vuol dire che il programma sia esente di errori**, ma che con quel test non ce ne sono.

3.0.5 Problema 2: Difetti e guasti

> "Testing can only show the presence of errors, not their absence."
> *(E. Dijkstra, 1972)*

Non tutti i **difetti** (defects) causano un **guasto** (failure). Alcuni difetti rimangono latenti fino a quando non si verificano condizioni specifiche che ne attivano l'effetto. Questo rende particolarmente difficile individuare problemi silenziosi che potrebbero compromettere la sicurezza e l'affidabilità del software.

Rilevare problemi silenziosi Il fatto che un test non rilevi errori **non significa che il programma sia privo di difetti**, ma solo che il test in questione non ha evidenziato alcun problema.

3.0.6 Il Principio del Traffico

Proposto da Andreas Zeller nel suo libro *"Why Programs Fail?"*, il principio del traffico per il debugging si articola nei seguenti passi:

- **T**racciare il problema: Identificare e documentare il malfunzionamento. I concurrency bug sono molto rognosi e complessi, tanti bug sono deterministici ma altri no.
- **R**iprodurre: Cercare di riprodurre il problema in un ambiente di test controllato.
- **A**utomatizzare: Creare strumenti che rendano il processo di debugging ripetibile e automatizzato.
- (**F**inding) Trovare le origini: Individuare le cause profonde del problema.
- **F**ocalizzarsi: Concentrarsi sugli aspetti rilevanti per risolvere il problema.
- **I**solare: Separare il problema dalle altre parti del sistema per facilitarne la risoluzione.

- **C**orreggere: Apportare le modifiche necessarie per risolvere il difetto.

Il nostro obiettivo principale (ma non esclusivo) sarà la parte relativa all'**Automazione**, che consente di rendere il processo di debugging più efficiente e ripetibile.

3.1 Approaching software testing

3.1.1 Testing nei libri di Ingegneria del Software

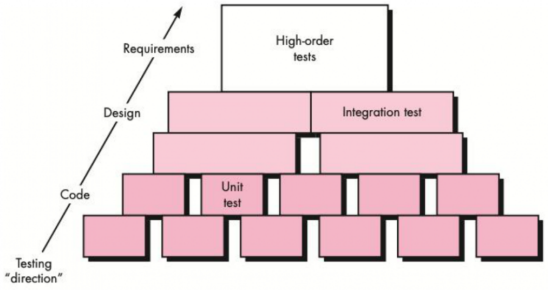
> "Il testing è il processo di esercitare un programma con l'intento specifico di trovare e, infine, correggere gli errori."
> (R. Pressman - *"Software Engineering: A Practitioner's Approach"*)

Alcuni punti chiave:

- La strategia di testing deve tenere conto delle specifiche tecniche di progettazione dei casi di test e dei metodi di testing.
- Le revisioni tecniche possono eliminare molti errori prima che il testing abbia inizio.
- Il testing inizia a livello di componente e si espande verso l'esterno (verso il sistema).
- Il testing è condotto dal sviluppatore e, per progetti più grandi, da un gruppo di test dedicato.

3.1.2 Testing works outwards

Osserviamo questa gerarchia:



3.1.3 Chi si occupa del testing?

> "Il testing è condotto dal sviluppatore e, per progetti più grandi, da un gruppo di test."

Anche quando è coinvolto un gruppo di test indipendente, lo sviluppatore continua a essere coinvolto nel processo di testing.

Bias e interessi Il ruolo di un gruppo di test indipendente è quello di eliminare i problemi intrinseci (**bias** e **conflitti di interesse**) associati al fatto che sia il costruttore stesso a testare ciò che ha costruito.

Obiettivi differenti Si potrebbe dire che gli sviluppatori tendano a testare in modo più "soft", essendo principalmente **orientati alla consegna del prodotto**, mentre i **tester indipendenti** devono imparare a conoscere il sistema, **cercare di "romperlo"** e sono motivati dalla qualità del software. Entrambi gli aspetti sono necessari per garantire un prodotto finale robusto e privo di difetti. Ciò è dovuto anche dalla pressione del delivery, di una scadenza e quindi la fase di testing è sottovalutata.

3.1.4 La cultura della ricerca nell'Ingegneria del Software

Ingegneria del Software Oltre a scrivere codice, un ingegnere del software deve **testare i programmi**, ma i test da soli (spesso superficiali) **non garantiscono l'assenza di errori**. Un software engineer vuole:

- Vuoi trovare i bug.
- Sei disposto a investire molto tempo.
- Hai il codice sorgente.
- Hai una specifica e/o dei casi di test.
- Hai un'idea di dove si trovano i bug.
- Hai un'idea di cosa sia un bug.
- Hai bisogno di indicazioni per prendere decisioni sul testing.

3.1.5 La cultura della ricerca sulla Sicurezza

Sicurezza Si concentra sul **testare programmi di terzi**, analizzando codice non scritto direttamente.

- Individuare bug senza accesso diretto al codice sorgente.
- Limitare il tempo investito nell'analisi.
- Utilizzare informazioni parziali o tracce di esecuzione.
- Non sapere dove si trovano i bug a priori.
- Provocare crash per identificare potenziali exploit.
- Automatizzare il processo il più possibile.

3.1.6 Ma io sono un ingegnere, non (ancora) un ricercatore?!?

Se sei un ingegnere, probabilmente desideri imparare come testare al meglio qualsiasi programma. Per fare ciò, è fondamentale avere **automazione completa** nel processo di testing. Qualsiasi risorsa aggiuntiva che hai è un bonus, per esempio:

- Se hai a disposizione degli input o il codice sorgente, utilizzali.
- Se hai un'idea di dove si trovano (la maggior parte) dei bug, sfruttalo.

Pertanto, il nostro obiettivo principale è cercare **automazione** nel processo di testing, mantenendo al contempo un certo **controllo** sul processo stesso.

3.2 Methodologies and techniques

3.2.1 Test statici e dinamici

I test statici verificano la presenza di difetti senza eseguire il codice. Non eseguo nulla, ma ho visibilità su tutto, posso riuscire a correlare cosa fanno le funzioni, ma il problema è complessità computazionale. I test dinamici esaminano la risposta del sistema durante l'esecuzione. Guarda solo l'esecuzione, se c'è un bug da un'altra parte e quella parte non viene esercitata, quel bug non viene visto. Il software moderno ha bisogno di **entrambi i tipi di test**.

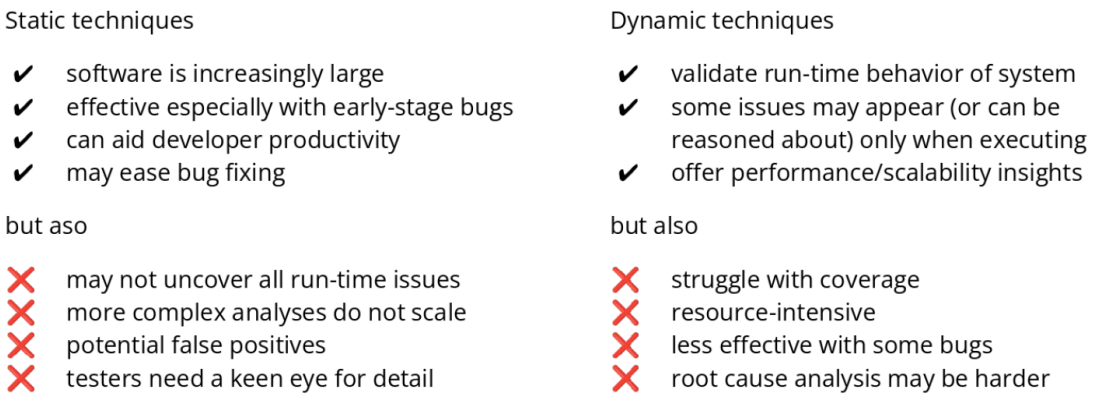


Figura 2: Pro e contro delle diverse tecniche

3.2.2 Tecniche di test statici

Le tecniche di testing statico sono principalmente orientate a **verificare l'assenza di bug** per i tipi specifici che la tecnica è progettata per rilevare.
Per ragionare sul codice, è necessario spesso manipolare sovra-approssimazioni della semantica del programma, il che può portare a **falsi positivi**.

Sovra-approssimazione Una **sovra-approssimazione** significa che, invece di trovare tutti gli stati esatti raggiungibili dal programma, si studia un superinsieme di essi, che risulta essere più semplice da analizzare. Potrebbero esserci **troppi falsi positivi**; serve un trade-off tra positivi e precisione.
Inoltre, alcuni tipi di bug possono essere troppo complessi per il ragionamento statico.

Tecniche efficaci Il testing statico si basa principalmente sull'analisi del codice sorgente. Esistono tecniche e strumenti specializzati per compiti e complessità diverse:

- **Linters** (per analisi superficiali di stile e errori comuni).
- **Analizzatori statici di codice** (come CodeQL, Coverity, SonarQube, ecc.).
- **Checker** per tipi specifici di bug.
- **Metodi generali**: abstract interpretation, symbolic execution (formule matematiche che si eseguono solo su alcune parti del programma).

3.2.3 Tecniche di test dinamico

Le tecniche di testing dinamico si concentrano principalmente sull'**individuare la presenza di bug durante una singola esecuzione del programma**.
I due principali problemi del testing dinamico sono:

- **Copertura**: può individuare bug solo nel codice che viene effettivamente eseguito.
- **Precisione**: potrebbe non rilevare un bug **(falso negativo)** a seconda di quante informazioni è in grado di acquisire sul contesto di esecuzione.

3.2.4 Distinzione tra test dinamici: Black-box e White-box

Una distinzione classica nel testing dinamico è tra **black-box testing** e **white-box testing**:

- **Black-box testing**: si concentra sulla funzionalità del sistema secondo una specifica fornita e non si occupa dei dettagli di implementazione. Opera *a scatola chiusa*, ignora i dettagli di implementazione nel codice.

- **White-box testing**: riguarda come il sistema è implementato e si concentra sulla copertura di ogni aspetto dell'implementazione (almeno una volta).

Anche in questo caso, occorre utilizzare entrambi. Allo stato dell'arte, si prende il meglio sia da una parte che dall'altra.

3.2.5 Tecniche di test dinamico: esempio di black-box

Equivalence partitioning Un esempio di testing black-box è la **partizionamento dell'equivalenza** (equivalence partitioning). Si chiama cosi perche uno vale l'altro, se il programma prende in input un numero tra 0 e 100, se gli diamo in input 1, 3, 99 è uguale. Il dominio di input del programma viene suddiviso in classi equivalenti e vengono selezionati input che soddisfano ciascuna classe (ad esempio, intervalli di valori, valori booleani, appartenenza a un insieme).
Una volta identificate le classi di equivalenza, derivare gli input è un'operazione relativamente semplice.

Boundary value analysis Un'altra tecnica nota è l'**analisi dei valori di confine**, poiché molti errori si verificano ai limiti del dominio di input anziché nel suo centro. Questa tecnica seleziona i casi di bordo per ciascuna classe di equivalenza.

3.2.6 Tecniche di test dinamico: esempio di white-box

L'obiettivo è coprire con un input tutte le feature di un programma. Le tecniche di **white-box testing** analizzano l'implementazione e generano input per coprirla secondo criteri specifici.

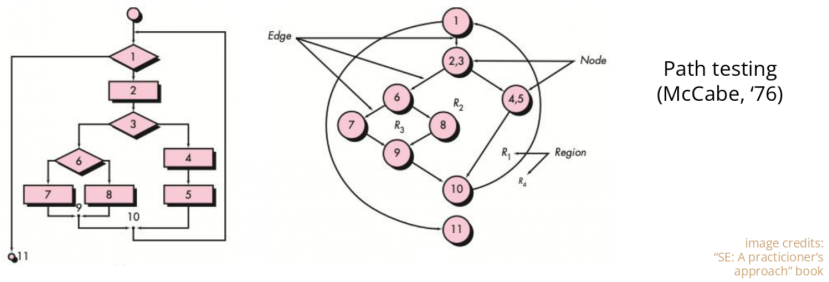


Figura 3: Path testing (McCabe, '76)

Path testing Un'idea semplice è percorrere ogni arco del grafo di flusso del programma almeno una volta (*Path testing*). Questo approccio aiuta a testare la struttura di controllo, ma non basta eseguire un percorso una sola volta.

Altri test Il *path testing* è solo una delle tecniche di testing basate sulla struttura di controllo. Altri metodi includono:

- **Condition testing**: verifica le condizioni logiche nel codice.
- **Data flow testing**: analizza il flusso delle variabili dalla loro definizione all'uso.
- **Loop testing**: si concentra sul comportamento dei cicli.

L'obiettivo è generare input per testare ogni esito indipendente, migliorando la copertura e l'affidabilità del software.

3.2.7 Tecniche di Testing Dinamico

Il testing *black-box* e il testing *white-box* si **completano** a vicenda, poiché spesso espongono diversi tipi di bug. Inoltre, tendono ad essere applicati in fasi diverse del processo di sviluppo.
Tuttavia, esistono alcune **principali limitazioni**, tra cui:

- La **necessità di conoscenze** del dominio o del codice per generare input di test adeguati.
- Ad esempio, un dato input per una determinata classe potrebbe non innescare un bug, ma uno che differisce solo leggermente potrebbe farlo: come generiamo input "buoni"?

Per questo motivo, ci si orienta verso metodi di testing automatizzati (automated) e sistematici.

3.2.8 Ingredienti per le Tecniche di Testing Moderne

I *desiderata* per le tecniche moderne di testing includono:

- Tecniche per generare input con **poca o nessuna assistenza umana**, con questi input in grado di coprire diverse parti del programma.
- Metodi per **distinguere quando un'esecuzione corrompe lo stato del programma** (poiché i crash rappresentano solo un sottoinsieme dei bug presenti nel programma).
- La capacità di **testare codice complesso** e real-world.

In questo corso, esploreremo l'uso del **fuzzing**[6] in combinazione con i **sanitizers**[7].

3.3 Bug and vulnerabilities

3.3.1 Terminologia

Un *bug* è un difetto in un software o sistema che provoca un risultato inaspettato. Si racconta che nel 1947, una falena fu trovata tra i contatti di relè del computer *Harvard Mark II* (la storia è ancora oggetto di discussione).

Nowadays I *bug* del software sono dovuti a errori umani nel codice sorgente, nel compilatore o nel sistema di esecuzione. Essi provocano uno stato non intenzionale del programma e, possibilmente, dei crash. Solo se il *bug* consente a un attaccante di compromettere la *confidentiality*, l'*integrity* o la *availability* delle risorse del sistema, si parla di *vulnerabilità*.

3.3.2 Da Bug a Exploit

Una *vulnerability* è una debolezza nel software che permette a un attaccante di sfruttare **(exploit)** un *bug*. Una *vulnerability* richiede tre componenti:

- un **sistema suscettibile** a un difetto;
- l'avversario ha **accesso al difetto**;
- l'avversario ha la **capacità di sfruttare** il difetto.

Le *vulnerabilities* si classificano in base al tipo di difetto (es. *use-after-free*) o alla primitiva computazionale che abilitano (es. *arbitrary write*). Le *attack primitive* sono i blocchi fondamentali di un exploit, che li combina per raggiungere l'obiettivo.

3.3.3 Gravità delle Vulnerabilità

Il *Common Vulnerability Scoring System* (CVSS) fornisce un metodo per classificare le vulnerabilità in base alle loro caratteristiche, assegnando un punteggio numerico che ne indica la gravità.

[6] Tecnica automatizzata di collaudo del software che prevede l'immissione di dati non validi, imprevisti o casuali come input di un programma tramite appositi programmi detti fuzzer

[7] Strumento di programmazione che rileva bug sotto forma di comportamenti indefiniti o sospetti causati da un compilatore che inserisce codice di strumentazione in fase di esecuzione.

Punteggio Il punteggio CVSS può essere convertito in una valutazione qualitativa (*basso*, *medio*, *alto*, *critico*) per aiutare le organizzazioni a gestire le vulnerabilità in base alla priorità.
Il calcolo del punteggio avviene tramite il *CVSS v4.0 Calculator*, che valuta diversi fattori, tra cui:

- **Attack vector**: contesto dell'attacco (locale, remoto, etc.).
- **Attack complexity**: difficoltà di sfruttare la vulnerabilità.
- **Attack requirements**: condizioni necessarie per l'attacco.
- **Privileges required**: privilegi richiesti per l'exploit.
- **User interaction**: necessità di interazione dell'utente.

Il risultato è un punteggio da 0 a 10, dove valori più alti indicano vulnerabilità più gravi.

3.3.4 Comportamento quando si trova una vulnerabilità

Le buone pratiche prevedono di informare le parti interessate.

- Se trovi un *bug* potenzialmente rilevante per la sicurezza nel tuo software, correggerlo non è sufficiente: dovresti informare i tuoi utenti affinché possano prendere le misure appropriate.
- Se trovi una vulnerabilità in un software sviluppato da altri, dovresti avviare un processo noto come **responsible disclosure**. I buoni fornitori hanno programmi dedicati a questo.

In entrambi i casi, un risultato eventuale potrebbe essere l'assegnazione di un identificatore per la vulnerabilità.

3.3.5 CVE (Common Vulnerabilities and Exposures)

Il programma CVE identifica, definisce e cataloga le vulnerabilità di sicurezza pubbliche. Attualmente, conta oltre 270.000 record, ricercabili per ID o parole chiave.

CVE IDs Le *CVE IDs* forniscono identificatori univoci per le vulnerabilità, rendendole lo standard di riferimento. Il programma CVE collabora con la comunità globale per ampliarne l'uso e i contenuti.
Nota: È possibile cercare vulnerabilità per ID (es. CVE-2024-1234) o per parole chiave (*SQL Injection*, *Cross-Site Scripting*, ecc.).

3.3.6 Takeaways

- La diagnosi dei difetti nel software può essere complessa.
- Non tutti i difetti portano a fallimenti (osservabili).
- Le attività di testing coinvolgono sia sviluppatori che tester dedicati.
- Sia le tecniche di testing statico che dinamico sono necessarie.
- È necessaria l'automazione e l'efficacia nella generazione degli input di test.
- I *bug* che un attaccante può sfruttare sono vulnerabilità di sicurezza.

4 Memory and type safety

4.1 Low-level bugs

4.1.1 Dai bug software alle primitive di attacco

Precedentemente abbiamo visto che:

- Le **primitive di attacco** sono i passi fondamentali per la costruzione di exploit.
- I bug software possono essere mappati su primitive di attacco.

Alcune classi di bug sono note per essere una fonte comune di tali primitive. Ci concentreremo sui linguaggi di basso livello come `C/C++`, poiché offrono varie opportunità di sfruttamento.

4.1.2 Il caso più famoso: buffer overflow

Un esempio classico di vulnerabilità è il seguente di buffer overflow:

```
void func(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    ...
}

int main() {
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```

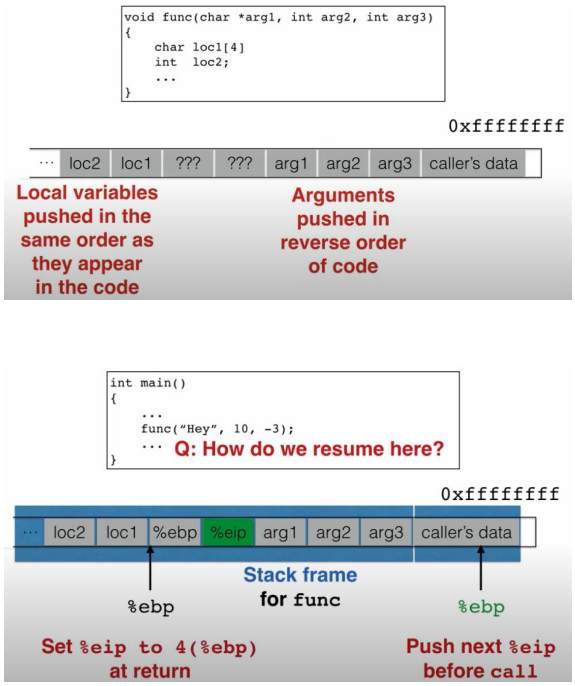
Cosa andrà storto? Il codice sopra presenta una vulnerabilità di **buffer overflow**. Analizziamo il problema:

- La funzione `strcpy` copia il contenuto di `arg1` nel buffer locale `buffer`, ma senza controllare la dimensione dell'input.
- `buffer` è dichiarato con una dimensione di 4 byte, mentre la stringa `"AuthMe!"` ha una lunghezza di 7 byte (più il terminatore nullo), causando una sovrascrittura della memoria adiacente.
- Questo può portare a corruzione dello stack, crash del programma o, in scenari più avanzati, a esecuzione di codice arbitrario.

Come mitigare il problema? Per mitigare il problema, si può:

- Usare funzioni sicure come `strncpy` invece di `strcpy`, specificando la dimensione massima del buffer.
- Implementare **canary values** per rilevare sovrascritture dello stack. Un *canary* è un valore sentinella che, se alterato da un *buffer overflow*, fa terminare il programma per prevenire exploit.
- Abilitare protezioni a livello di compilatore come `Stack Protector` (`-fstack-protector` in GCC).
- Utilizzare tecniche moderne come **Address Space Layout Randomization (ASLR)** per rendere imprevedibili gli indirizzi di memoria, ostacolando gli exploit basati su buffer overflow.

4.1.3 Richiamo: struttura base dello stack



Nel nostro caso, si ha

```c
void func(char* arg1) {
    char buffer[4]; // Dichiarazione corretta dell'array
    strcpy(buffer, arg1); // Vulnerabile a buffer overflow
    ...
}

int main() {
    char *mystr = "AuthMe!";
    func(mystr);
    ...
}
```

		M e ! \0	Indirizzo di ritorno	Primo argomento
	Auth	4d 65 21 00	%eip	&arg1

Tabella 2: Disposizione della memoria nello stack

All'atto del ritorno, a causa di `strcpy()`:

- `%EBP = 0x21654d` *(valore corrotto)*
- Probabile crash dell'applicazione per violazione di segmentazione (`SEGFAULT`)

4.1.4 Conseguenze di sicurezza

```c
void func(char* arg1) {
    int authenticated;  //NOTA
    char* buffer[4];
    authenticated = checkAuth(); //NOTA
    strcpy(buffer, arg1); // Buffer overflow possibile

    if (authenticated) { //NOTA
        ...
    }
```

```
}

int main() {
    char *mystr = "AuthMe!";
    func(mystr);
}
```

		M e ! \0			
	Auth	4d 65 21 00	%ebp	%eip	&arg1
	buffer	authenticated			

Tra i rischi osservabili:

- Rischio di bypass dell'autenticazione (**authentication bypass**): l'attaccante può manipolare la memoria e sovrascrivere variabili critiche
- **Perdita della confidenzialità:** accesso non autorizzato a dati sensibili
- Rischio di **corruzione dello stack:** possibile esecuzione di codice arbitrario

4.2 Memory and type safety

4.2.1 Memory safety

La memory safety è una delle proprietà più desiderabili per i programmi. Un puntatore dovrebbe sempre fare riferimento all'oggetto per cui è stato inizialmente definito.

Violazione Una violazione della sicurezza della memoria si verifica quando un puntatore fa riferimento a una zona di memoria diversa da quella appartenente all'oggetto per il quale il puntatore è stato inizialmente definito. Possiamo rendere operativa questa definizione attraverso due proprietà:

- sicurezza spaziale (spatial safety)
- sicurezza temporale (temporal safety)

NOTA: In Java non possiamo assegnare direttamente un puntatore (si lavora con riferimenti, *references*), il che è una limitazione, ma paghiamo questo prezzo per ottenere delle garanzie di sicurezza.

Stack e heap In stack, la memoria non è più valida dopo la return, ma nulla vieta di utilizzarla in modo improprio. Lo stesso discorso vale per l'heap: dopo la deallocazione non dovremmo più accedere a memoria deallocata. Le variabili locali vengono distrutte al termine del programma o quando esce dal loro ambito di validità.

4.2.2 Spatial safety

Una violazione della sicurezza spaziale si verifica quando un **accesso alla memoria avviene al di fuori dei limiti dell'oggetto** previsto per il puntatore. L'allocazione della memoria definisce tali limiti e ogni operazione successiva di aritmetica sui puntatori deve garantire che il puntatore rimanga all'interno dei confini dell'oggetto di interesse.

Operatività Dal punto di vista operativo, una violazione si verifica solo quando si utilizza effettivamente il dato fuori dai limiti. In alcuni casi, si possono "sfondare" i limiti per "comodità", ma è fondamentale assicurarsi che l'accesso sia sempre confinato all'interno dei boundaries definiti. Esempio:

```
char *c = (char*)malloc(24);

for (int i = 0; i < 26; ++i) {
    // 1.) Buffer overflow per i >= 24
    c[i] = 'X'+i;  // Scrittura fuori dai limiti del buffer
```

```
}

// 2.) Violazione attraverso una scrittura diretta
c[26] = 'A';  // Scrittura fuori dai limiti del buffer
c[-2] = '2';  // Scrittura prima dell'inizio del buffer

// 3.) Puntatori non validi: OK se non dereferenziati
char *d = c + 26;
d -= 3;
*d = 'C';  // Accesso a memoria non allocata
```

4.2.3 Temporal Safety

Una violazione della **sicurezza temporale** (*temporal safety*) si verifica quando un accesso alla memoria avviene tramite un puntatore che non è più valido.

Dangling pointer Un puntatore diventa **invalido** quando l'oggetto a cui faceva riferimento non è più lo stesso di quando il puntatore è stato inizialmente creato. Il caso tipico è quello di un **dangling pointer** (puntatore sospeso).

Memoria deallocata Dopo la distruzione di un oggetto, un puntatore può ancora contenere l'indirizzo di memoria deallocata. Il sistema potrebbe riassegnare quella stessa area di memoria per altri scopi. Se il programma successivamente tenta di dereferenziare il puntatore sospeso, può verificarsi un comportamento imprevedibile, causando vulnerabilità di sicurezza o crash del programma.

Differenze con garbage collection In Java, ad esempio, esiste la *garbage collection*, che gestisce automaticamente la memoria. In C, invece, la memoria deve essere esplicitamente liberata e i puntatori devono essere azzerati. Dopo una chiamata a `free`, la memoria è teoricamente disponibile, ma il puntatore che la indicava può ancora contenere un indirizzo di memoria "non valido". Per questo motivo, è buona pratica impostare il puntatore a `NULL` dopo aver liberato la memoria, al fine di evitare accessi accidentali. Un attaccante potrebbe sfruttare un puntatore non azzerato per accedere a memoria non più valida, portando a vulnerabilità.

4.2.4 Type Safety

Un oggetto in memoria può essere accessibile solo se il puntatore ha il tipo corretto.

Sicurezza dei tipi La *type safety* garantisce che il tipo di un oggetto sia coerente con il sistema di tipi del linguaggio di programmazione e che corrisponda a uno dei tipi compatibili secondo il meccanismo di ereditarietà dei tipi.
Operativamente, si ha che:

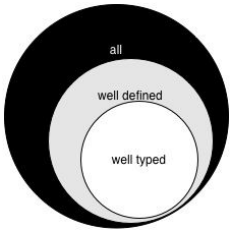
- Gli oggetti devono essere utilizzati con i loro tipi corrispondenti.
- I cast devono rispettare la gerarchia dei tipi in base al vero tipo dell'oggetto a runtime.

La **sicurezza dei tipi** (*type safety*) è strettamente legata alla **sicurezza della memoria** (*memory safety*).

Type confusion Un problema serio è la **type confusion**, che può causare l'uscita dai limiti di memoria se i tipi sono gestiti in modo errato.
Quando un programma tratta un dato con un tipo sbagliato, possono verificarsi sovrascritture o accessi a memoria non autorizzata, creando vulnerabilità. Linguaggi come Java impediscono operazioni di tipo non sicure, mentre in C, la gestione dei tipi è più flessibile e richiede attenzione per evitare problemi di sicurezza.

Well typed program Un programma ben definito e ben tipizzato è considerato sicuro dal punto di vista dei tipi.

> *"Well-typed programs cannot go wrong"* (R. Milner, 1978)



Tuttavia, ciò che viene garantito dipende dalla semantica del linguaggio.

Esempio Consideriamo il seguente codice pseudocodice che potrebbe essere rifiutato da un sistema di tipi rigoroso:

```
// Errore di tipo: x assume tipi diversi in base a p
if (p) x = 5;
else x = "hello";

if (p) return x + 5;
else return strlen(x);
```

Un sistema di tipi forte potrebbe rifiutare questo codice perché la variabile `x` viene utilizzata sia come intero che come stringa, causando comportamenti indefiniti o errori di compilazione in linguaggi fortemente tipizzati. In JavaScript, i tipi vengono assegnati a runtime, il che, se da una parte offre maggiore flessibilità, dall'altra può rappresentare un grave problema di sicurezza e stabilità.

4.2.5 Violazione della sicurezza dei tipi e della memoria

I bug che violano la **sicurezza della memoria** (*memory safety*) o la **sicurezza dei tipi** (*type safety*) possono essere sfruttati per alterare lo stato del programma a runtime.
Questo stato modificato può portare a un'esecuzione che non sarebbe possibile in un programma che segue un flusso di esecuzione benigno.

Weird machine Quando un programma non segue più i suoi normali flussi di controllo e di dati, si trasforma in una cosiddetta **weird machine**, un ambiente in cui gli attaccanti possono manipolarne il comportamento a loro vantaggio, come per esempio di prima saltare l'autenticazione.

Vulnerabilità low-level La memory and type safety garantiscono l'assenza di vulnerabilità di basso livello che violano le astrazioni ad alto livello del linguaggio di programmazione. Queste proprietà limitano gli attacchi ai soli bug logici presenti nel codice del programma.
Più precisamente, la sicurezza della memoria e dei tipi impone vincoli definiti dalla specifica del linguaggio e dall'implementazione, non dai vincoli di una macchina astratta sottostante.

4.2.6 Come ottenere sicurezza

La necessità di controlli sulla memoria e sui tipi dipende dal linguaggio di programmazione:

- **Linguaggi insicuri per la memoria:** Linguaggi come C e C++ non impongono vincoli di sicurezza sulla memoria e sui tipi, affidando al programmatore la gestione corretta della memoria.

```
char *ptr = (char*)malloc(10);
ptr[15] = 'A';  // Scrittura fuori dai limiti del buffer
free(ptr);
*ptr = 'B';  // Uso di un dangling pointer (comportamento indefinito)
```

Sempre in C/C++, si può notare che:

- L'aritmetica dei puntatori permette la modifica diretta degli indirizzi di memoria.
- Il programmatore deve garantire che i puntatori siano validi prima di accedere ai dati.
- Errori come l'accesso a memoria non valida o il riutilizzo di puntatori sospesi (*dangling pointer pointers*) possono causare vulnerabilità gravi.

- **Linguaggi con sicurezza della memoria:**
  - **Java:** garantisce la sicurezza della memoria attraverso una combinazione di caratteristiche del linguaggio di programmazione (ad esempio, i puntatori sono sostituiti da riferimenti e non sono consentiti accessi diretti alla memoria) e supporto a livello di runtime (ad esempio, controlli sui limiti e garbage collection).
    Inoltre, è necessario un **runtime support**[8] che garantisce la sicurezza in memoria, ma ad esempio non è flessibile come C nello scrivere in alcune zone precise di memoria.
  - **Rust:** utilizza un sistema di tipi rigoroso e un modello di ownership per implementare la sicurezza della memoria e dei tipi. I riferimenti sono legati a variabili e una chiara gestione della proprietà protegge contro le condizioni di data race. La memoria viene recuperata quando le variabili escono dallo scopo. Il compilatore impone molte di queste garanzie, con astrazioni a costo zero.
  - **Linguaggi funzionali** come **OCaml:** In questi linguaggi, non esistono oggetti; tutto è definito in termini di funzioni, rappresentando un'astrazione diversa rispetto ai linguaggi imperativi. Evitano i problemi di memoria eliminando il concetto di riferimenti mutabili e privilegiando il passaggio di dati attraverso strutture immutabili. Di natura, sono immuni a questi problemi, anche se non è sempre possibile fare tutto in questo modo. I linguaggi funzionali come OCaml non presentano problemi di memoria, poiché non utilizzano riferimenti, ma gestiscono dati e controllo in modi alternativi.

4.2.7 Sicurezza della memoria in C e C++

Per ottenere la sicurezza della memoria in C e C++, esistono due approcci principali:

- **Creare dialetti privi di funzionalità insicure:** Alcuni esempi sono **Cyclone** e **CCured**, linguaggi derivati da C con controlli aggiuntivi per la sicurezza della memoria. Questi dialetti limitano alcune funzionalità del linguaggio originale, ad esempio impedendo la compilazione in caso di problemi di sicurezza. Sebbene questi linguaggi siano concettualmente validi, presentano delle difficoltà pratiche, come la compatibilità con diverse architetture hardware.
- **Proteggere le funzionalità insicure con strumenti di monitoraggio:** L'uso di strumenti di analisi dinamica, come **SoftBound**, consente di rilevare violazioni della memoria in fase di esecuzione, garantendo il controllo dei limiti di accesso alla memoria. Tuttavia, queste soluzioni introducono un overhead significativo, con un impatto sulle prestazioni (ad esempio, SoftBound può comportare un aumento del 67% del tempo di esecuzione).

Questi approcci richiedono di tenere traccia di tutti gli oggetti e puntatori attivi durante l'esecuzione del programma, migliorando la sicurezza ma influendo sulle prestazioni.

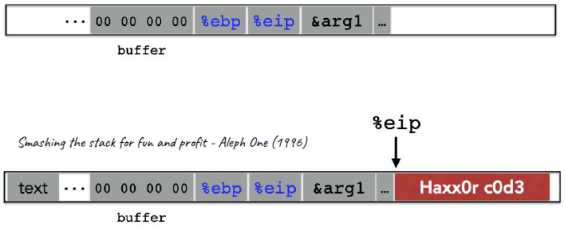
[8] Per garantire la sicurezza della memoria in Java, non basta solo la sintassi del linguaggio di programmazione, ma è necessario anche un sistema di supporto che funzioni durante l'esecuzione del programma. In altre parole, il runtime support include tutte le operazioni e i controlli che avvengono mentre il programma è in esecuzione, come ad esempio bounds checking e garbage collection

4.3 Consequences of safety violations

4.3.1 Stack Buffer Overflow

Un attacco di stack buffer overflow avviene quando i dati superano i limiti del buffer, sovrascrivendo altre variabili o indirizzi di ritorno.

```c
void func(char *arg1) {
    char buffer[4];
    snprintf(buffer, sizeof(buffer), "%s", arg1);
}
```



4.3.2 Heap Buffer Overflow

L'attaccante può sovrascrivere il puntatore alla funzione e deviare il flusso di esecuzione.

```c
typedef struct {
    char buffer[4];
    void (*fct)(int);
} mydata_t;

void func(char* arg1) {
    mydata_t* ptr = (mydata_t*)malloc(sizeof(mydata_t));
    ptr->fct = &exit; // Puntatore a funzione
    strncpy(ptr->buffer, arg1, sizeof(ptr->buffer) - 1);
    ptr->buffer[sizeof(ptr->buffer) - 1] = '\0';
    ptr->fct(0); // Se sovrascritto, il flusso di controllo e' compromesso
    free(ptr);
}
```

4.3.3 Read Overflow

Anche le letture possono essere pericolose, poiché non c'è alcuna garanzia che la lunghezza specificata (`len`) rientri nei limiti del buffer. In questo caso, la lettura potrebbe andare oltre la memoria allocata, causando un **read overflow**.

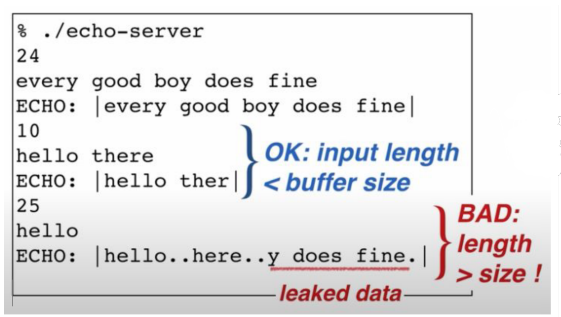
```c
int main() {
    char buf[100], *p;
    int i, len;
    while (1) {
        p = fgets(buf, sizeof(buf), stdin);      //read INT
        if (p == NULL) return 0;                 //read INT
        len = atoi(p);                           //read INT
        p = fgets(buf, sizeof(buf), stdin);      //read message
        if (p == NULL) return 0;                 //read message

        //echo back partial message
        for (i = 0; i < len && i < sizeof(buf); i++) {
            if (isprint(buf[i])) putchar(buf[i]);
            else putchar('.');
        }
        printf("\n");
    }
}
```

Problema: L'utente può specificare una lunghezza (`len`) che supera la dimensione effettiva del buffer, causando una lettura oltre i limiti del buffer, con il rischio di corrompere la memoria o causare comportamenti imprevisti.

4.3.4 Heartbleed

Il messaggio Heartbeat specifica la lunghezza del ritorno (echo-back), ma OpenSSL non la validava correttamente.



Spiegazione dell'attacco Heartbleed è una vulnerabilità critica che è stata scoperta nella libreria OpenSSL, che è una delle librerie più utilizzate per implementare la crittografia su internet. Questa vulnerabilità è stata causata dalla mancata convalida della lunghezza del messaggio nella funzionalità Heartbeat.
Quando un attaccante invia una richiesta Heartbeat, il server non controllava correttamente la lunghezza del messaggio. Questo permetteva a un attaccante di manipolare la lunghezza del messaggio e di leggere porzioni di memoria sensibile, tra cui chiavi private, credenziali, e altre informazioni riservate.
L'attacco Heartbleed è stato una delle violazioni di sicurezza più gravi nella storia recente, mettendo a rischio milioni di server online.

4.3.5 Type Confusion

Gli errori di *type confusion* si verificano quando si effettuano downcast illegali, portando a comportamenti indefiniti e vulnerabilità di sicurezza.

```
1  class P { public: int x; };
2  class C : public P {
3      public:
4          int y;
5          virtual void print();
6  };
7
8  P *Pptr = new P;
9  C *Cptr = static_cast<C*>(Pptr); // Type confusion
10 Cptr->y = 0x43; // Violazione della sicurezza della memoria
11 Cptr->print(); // Possibile hijacking del flusso di controllo
```

Nel caso delle classi C++, i puntatori sono memorizzati in una *vtable (virtual table)* che è collocata all'interno degli oggetti.

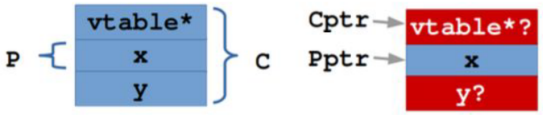


Figura 4: Rappresentazione della virtual table (vtable) e la sua relazione con gli oggetti in C++.

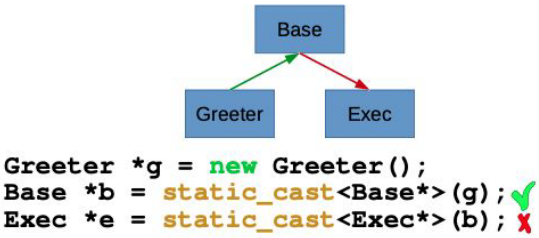


Figura 5: Esempio di *type confusion* in un contesto di sicurezza.

4.4 More subtle CdSes

4.4.1 Uso di memoria non inizializzata (Use of uninitialized memory)

Gli errori dovuti all'uso di memoria non inizializzata (UUM, Uninitialized Memory) possono avere effetti subdoli e imprevedibili sullo stato del programma, causando comportamenti anomali o vulnerabilità di sicurezza.

1. **Esempio 1 - Uso di variabile non inizializzata:**

```
int getmin(int *v, unsigned int len) {
    int min;  // sbagliato! inizializzazione corretta: int min = v[0];
    for (int i=1; i<len; ++i)
        min = (min < v[i]) ? min : v[i];
    return min;
}
```

   Nel codice, la variabile `min` viene utilizzata senza essere inizializzata, il che può portare a valori imprevedibili. È fondamentale inizializzare `min` al primo valore dell'array per evitare comportamenti imprevisti.

2. **Esempio 2 - Uso di puntatore non inizializzato:**

```
char *test_string;
if (i != err_val)
    test_string = "Hello World!"; // Inizializzazione condizionata
printf("%s", test_string);
```

   In questo caso, `test_string` è inizializzato correttamente, ma è importante assicurarsi che la variabile sia sempre inizializzata prima di essere utilizzata, per evitare che contenga valori non definiti, che potrebbero causare vulnerabilità di sicurezza.

4.4.2 Uso dopo il 'free' (Use after free)

Il "use after free" si verifica quando si tenta di accedere alla memoria che è stata già liberata, creando potenziali vulnerabilità di sicurezza.
Esempio di errore "use after free":

```
char* ptr = (char*)malloc(SIZE);
if (err) {
    abrt = 1;
    free(ptr); // Liberazione della memoria
}
if (abrt) {
    logError("operation aborted before commit", ptr); // Uso dopo free
}
```

In questo caso, si accede alla memoria dopo che è stata liberata, il che può causare comportamenti imprevedibili e potenzialmente sfruttabili per attacchi di sicurezza. È importante impostare i puntatori su `NULL` dopo averli liberati per evitare questo errore e prevenire accessi accidentali a memoria non valida.
Questo errore è particolarmente pericoloso in quanto introduce una vulnerabilità temporale, dove

la memoria viene liberata e successivamente utilizzata, violando la sicurezza della gestione della memoria. Notare quindi che questo errore è **temporal safety** perché viene liberato e DOPO viene utilizzato.

Double free Un errore correlato è il "double free"[9], che può corrompere lo stato dell'allocatore. In alcuni casi, l'attaccante può sfruttarlo per abilitare un buffer overflow.

4.4.3 Overflow intero (Integer overflow)

L'overflow intero si verifica quando un'operazione aritmetica produce un risultato che supera il valore massimo che un tipo di dato può rappresentare.
Esempio di overflow intero:

```
nresp = packet_get_int();
if (nresp > 0) {
    response = xmalloc(nresp * sizeof(char*)); // Potenziale overflow intero
    for (i = 0; i < nresp; i++)
        response[i] = packet_get_string(NULL);
}
```

In questo esempio, se `nresp` è molto grande, il calcolo di `nresp * sizeof(char*)` potrebbe causare un overflow e una successiva allocazione di memoria insufficiente, causando un buffer overflow. Si può aggiungere una verifica per evitare che `nresp` superi un limite sicuro.

4.4.4 Stringhe di formato (Format strings)

Le vulnerabilità legate alle stringhe di formato si verificano quando i dati forniti dall'utente vengono usati direttamente in una funzione come 'printf', senza una corretta validazione. Questo può permettere ad un attaccante di controllare l'esecuzione del programma.
Esempio di vulnerabilità con stringa di formato:

```
void vulnerable() {
    char buf[80];
    if(fgets(buf, sizeof(buf), stdin) == NULL)
        return;
    printf(buf); // Vulnerabilita di formato
}
```

In questo esempio, `printf` usa la stringa `buf` senza specificare un formato sicuro. Un attaccante potrebbe sfruttare questa vulnerabilità per scrivere in memoria o eseguire codice maligno. La versione sicura è la seguente:

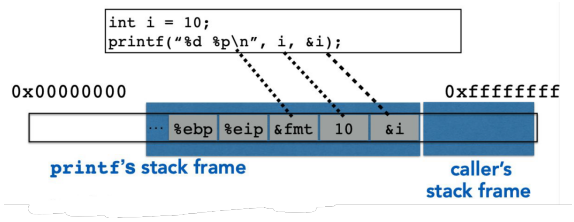
```
void secure() {
    char buf[80];
    if(fgets(buf, sizeof(buf), stdin) == NULL)
        return;
    printf("%s", buf); // Formato sicuro
}
```

Principale vulnerabilità Le stringhe di formato sono una delle principali cause di vulnerabilità di sicurezza nei programmi C. Il problema si verifica quando l'input dell'utente viene passato direttamente a funzioni come `printf` senza una corretta validazione del formato. Se non si verifica il numero e il tipo degli argomenti, l'attaccante potrebbe sfruttare queste vulnerabilità per accedere a dati sensibili o alterare il flusso del programma.
Osserviamo un esempio di comportamento normale di `printf`:

```
int i = 10;
printf("%d %p\n", i, &i); // stampa il valore di i e l'indirizzo di i
```

La funzione `printf` accetta un numero variabile di argomenti:

[9] Situazione in cui un programma tenta di liberare due volte la stessa area di memoria. L'allocatore di memoria perde il controllo su quale memoria sia effettivamente disponibile, il che può creare vulnerabilità.



- Non verifica la corretta corrispondenza tra il numero di argomenti e i segnaposti specificati nella stringa di formato.
- Non controlla i confini della stack, il che significa che l'attaccante potrebbe sovrascrivere dati sensibili o alterare il flusso del programma.
- Si presume che tu l'abbia chiamato con (almeno) tanti argomenti quanti specificati nella stringa di formato.

Codice arbitrario Se un attaccante riesce a controllare la stringa di formato passata a `printf`, può sfruttare questa vulnerabilità per accedere a dati sensibili nella stack, come variabili locali o informazioni riservate. Un attacco di questo tipo potrebbe anche alterare il flusso di esecuzione del programma, come l'esecuzione di codice arbitrario.
Ecco un esempio di codice vulnerabile:

```c
void vulnerable() {
    char buf[80];
    if (fgets(buf, sizeof(buf), stdin) == NULL) {
        buf = "%d %x"; // Esempio di stringa di formato malformata
    }
    printf(buf); // Vulnerabilita: il contenuto di buf non e' sicuro
}
```



In questo caso, un attaccante potrebbe fornire una stringa di formato malformata, come `%x` o `%n`, per manipolare la stack. Ad esempio, usando un formato come `%x`, l'attaccante potrebbe visualizzare valori presenti nella stack, mentre l'uso di `%n` potrebbe consentire di scrivere in posizioni di memoria arbitrarie, portando a un potenziale overflow del buffer o compromissione della memoria.

Tipo di buffer overflow Le stringhe di formato malformate possono essere considerate come un tipo di "buffer overflow" in cui la stack viene trattata come un buffer. Quando la stringa di formato non è corretta, il programma può sovrascrivere o leggere dati da posizioni non sicure nella memoria.

4.4.5 Altri esempi di uso errato delle stringhe di formato

Alcuni esempi di stringhe di formato che causano vulnerabilità:

- `printf("100% dave")` stampa l'entry della stack 4 byte sopra l'indirizzo salvato di %eip.
- `printf("%s")` stampa i byte a cui punta l'entry della stack sopra l'indirizzo salvato di %eip.
- `printf("%d %d %d")` stampa tre entries della stack come interi.
- `printf("100% no way!")` scrive (!), il numero 3, nell'entry della stack di ritorno.

Questi errori possono compromettere la sicurezza del programma, in quanto un attaccante può manipolare la stack per ottenere informazioni sensibili o controllare il flusso di esecuzione.

5 Sanitizers

5.1 Silent bugs

5.1.1 Testing per errori

Un errore può essere definito come una deviazione tra il comportamento osservato e quello atteso o specificato. Questo implica la violazione di una specifica sottostante, che può includere:

- Requisiti funzionali (funzionalità previste del sistema)
- Requisiti operativi (prestazioni, throughput, utilizzo della memoria, ecc., che variano a seconda dell'applicazione)

Entrambi questi tipi di requisiti possono essere verificati attraverso tecniche di *testing*. Tuttavia, i requisiti di sicurezza non sono sempre direttamente testabili.

No test sicuro Non esiste un test che possa garantire in modo assoluto che un programma sia sicuro. La sicurezza è una proprietà emergente e dipende anche dal contesto di utilizzo. È comunque possibile effettuare analisi che rilevano comportamenti sospetti o potenzialmente pericolosi, come:

- Uso non sicuro della memoria (es. *buffer overflow*, *use-after-free*, ecc.);
- Presenza di vulnerabilità note;
- Anomalie nel controllo del flusso o nella gestione degli input.

Strumenti come i *sanitizer* (es. *AddressSanitizer*, *MemorySanitizer*, *UndefinedBehaviorSanitizer*) aiutano a individuare queste problematiche durante l'esecuzione del programma, migliorando così la robustezza e la sicurezza del software.

5.1.2 Testing per problemi di sicurezza nel codice di basso livello

> "Il testing può solo evidenziare la presenza di errori, non la loro assenza"
> (E. Dijkstra, 1972)

Nel codice C/C++, possiamo testare indirettamente la sicurezza della memoria e dei tipi osservando gli effetti durante i test. L'attenzione non è sulla correttezza computazionale, piuttosto su come il programma possa violare tali proprietà durante l'esecuzione.

- **Pro**: Riproducibilità (cioè, input concreti)
- **Contro**: Copertura incompleta

5.1.3 Rilevamento degli errori (fault detection)

Il testing comune rileva i bug attraverso:

- **Assert**: Controlli espliciti che verificano condizioni attese nel codice (ad esempio con `assert` o `if`). Spesso vengono attivati solo in modalità *debug* e possono essere disabilitati facilmente tramite configurazioni del compilatore.
- **Segmentation faults**: Errori di accesso a zone di memoria non valide, tipici di accessi oltre i limiti di un array o puntatori nulli.
- **Divisione per zero**: Eccezioni generate da operazioni di divisione per zero.
- **Eccezioni non gestite**: Errori imprevisti che non vengono intercettati da costrutti di gestione delle eccezioni, portando alla terminazione del programma.

- **Meccanismi di mitigazione che forzano l'arresto**: Tecniche di sicurezza come *stack canaries*, *DEP* (Data Execution Prevention), o *ASLR* (Address Space Layout Randomization) possono provocare l'interruzione immediata del programma quando rilevano un comportamento anomalo. Questi sono simili a eccezioni non gestite, ma introdotti per motivi di sicurezza.

Questi strumenti rilevano ciò che viene definito come bug "visibili", ovvero che causano sintomi evidenti durante l'esecuzione. Tuttavia, esistono anche i **bug silenti**, che non producono crash, eccezioni o comportamenti direttamente rilevabili.

Come possiamo identificare questi bug? Ma come possiamo rilevare tali bug "silenti", cioè quelli che non causano nessuno degli errori sopra elencati? La risposta include l'utilizzo di strumenti avanzati come i *sanitizer*, l'*instrumentation* del codice, o tecniche di *fuzzing*, che permettono di scoprire comportamenti anomali non immediatamente evidenti.

5.1.4 Recall: Overflow in HeartBleed

Problema: la funzione di lettura non verificava correttamente la lunghezza del messaggio, permettendo di leggere oltre i limiti previsti. Un bug di questo tipo avrebbe potuto essere rilevato con l'uso di *sanitizers*.

5.1.5 Sanitizers

I *code sanitizers* sono strumenti che rilevano bug inserendo automaticamente codice di strumentazione durante la compilazione. Sono sostanzialmente **codice aggiuntivo** che monitora il comportamento del programma in fase di esecuzione, verificando che vengano rispettate determinate politiche di sicurezza. In caso di violazione, viene generato un errore esplicito, spesso prima che il programma si blocchi in modo anomalo (come avviene nei *segfault*).

Integrazione L'integrazione dei *sanitizers* avviene a livello di compilatore. Ad esempio, *Valgrind* può rilevare accessi fuori dai limiti di un buffer, ma opera a livello binario, con visibilità limitata sulle variabili e un elevato overhead (fino a 30x), rendendolo poco pratico per test estesi. Al contrario, i sanitizers basati su compilatore possono osservare il codice sorgente in modo più dettagliato, lavorando direttamente sulla *Intermediate Representation* (IR) generata durante la compilazione del codice C, ottenendo una visibilità più profonda sul programma e mantenendo una maggiore efficienza.

Framework di compilazione Questi strumenti possono essere visti come framework di compilazione che inseriscono controlli aggiuntivi. La loro diffusione è iniziata intorno al 2012, con l'introduzione di *AddressSanitizer* da parte di Google, pensato per rilevare bug silenti legati alla memoria. Inizialmente, i *sanitizers* come *AddressSanitizer* erano disponibili solo nel compilatore **LLVM/Clang**[10], noto per la sua architettura modulare e l'elevata capacità di analisi statica grazie alla *Intermediate Representation* (IR) dettagliata. Successivamente, anche **GCC** ha introdotto il supporto per i principali sanitizers, rendendoli accessibili a una base d'utenza più ampia [11].

Diverse fasi I sanitizers implementano politiche specifiche e migliorano significativamente l'efficacia del *testing*, rilevando bug precocemente. La maggior parte segue tre fasi principali:

- **Analisi statica**: durante la compilazione vengono estratte informazioni utili (es. grafi di tipi) o ottimizzazioni (es. accessi sicuri).
- **Strumentazione**: il programma viene arricchito con codice che raccoglie metadati e impone controlli.

[10] LLVM è progettato come una serie di moduli indipendenti. Questo lo rende ideale per strumenti di analisi statica e dinamica avanzata, e ha favorito l'introduzione precoce di *sanitizers*.

[11] Sebbene LLVM/Clang sia spesso preferito per l'integrazione con strumenti di analisi avanzata (anche per debugging), GCC offre ora funzionalità simili in termini di rilevamento di bug a runtime.

- **Analisi dinamica**: durante l'esecuzione, i metadati vengono verificati per rilevare violazioni delle politiche di sicurezza.

Questo solleva domande naturali:

- Quali politiche di sicurezza sono interessanti (e fattibili)?
- Quali metadati servono per farle rispettare?
- Dove conviene inserire i controlli?

La raccolta e la verifica dei metadati devono essere efficienti: un overhead eccessivo renderebbe impraticabile l'uso dei sanitizers nei test su larga scala.

5.2 AddressSanitizer (ASan)

ASan (AddressSanitizer) è un *sanitizer* progettato per rilevare violazioni della **memory safety**, in particolare:

- **Spatial safety**: accessi fuori dai limiti dell'oggetto (accurato, gestisce bene questa safety)
- **Temporal safety**: uso di memoria dopo la liberazione (**parzialmente** coperto, ASan non gestisce tutti i casi)

L'idea centrale di ASan è quella di "avvelenare" (poison) la memoria non valida. Se il programma tenta di accedere a una regione avvelenata, viene generato un errore controllato, spesso prima che causi un crash imprevedibile. ASan fa crashare il programma se è violata la **spatial memory safety**, ma anche in alcuni casi di **temporal memory safety**. Specificamente, ASan è in grado di rilevare:

- **Violazioni della spatial memory safety:**
  - `Buffer overflow` (heap, stack, global)
  - Accessi `out-of-bounds`
- **Violazioni della temporal memory safety:**
  - `use-after-free`
  - `use-after-return`
  - `use-after-scope`
- Memory leaks (in alcune configurazioni)

Nel primo schema, vediamo come ASan inserisca regioni **poison** a sinistra e a destra dei buffer allocati **(redzone)**, prevenendo *underflow* e *overflow*. Una memoria liberata viene anch'essa marcata come avvelenata (in grigio).

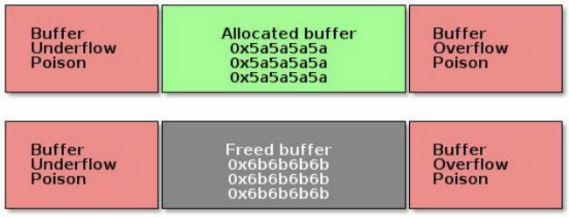


Figura 6: Memoria protetta da ASan: regioni valide (verde), avvelenate (rosso), liberate (grigio)

Nel seguente schema, si evidenzia come le strutture in memoria siano circondate da aree chiamate **redzones**. Queste zone fungono da "cuscinetti" e impediscono accessi errati tra oggetti adiacenti. Questo è un caso che strumenti come *Valgrind* non riescono a rilevare efficacemente. Il sanitizer, in questo caso, non permette di leggere B direttamente dopo, poiché tra A e B ci sono cuscinetti chiamati "poison", e se vengono toccati si genera un errore.

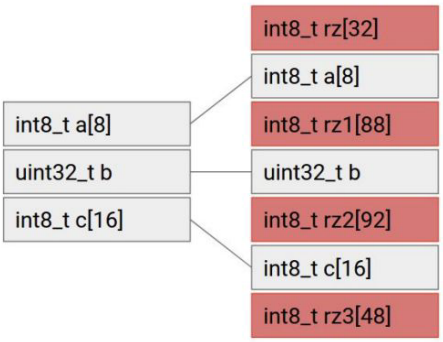


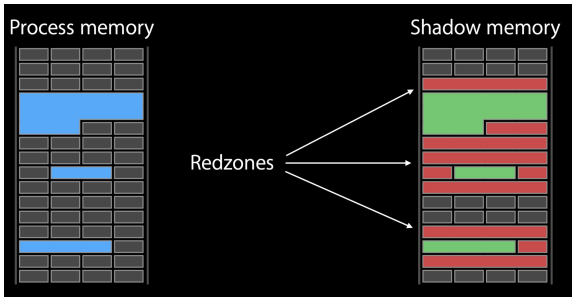
Figura 7: Cuscinetti rossi

5.2.1 Redzones

Per rilevare violazioni di memoria, *AddressSanitizer* inserisce delle **redzones**, ovvero aree di memoria avvelenate che circondano ogni oggetto allocato. Queste zone fungono da "cuscinetti di sicurezza" e, se toccate, provocano un errore controllato.

- Ogni oggetto riceve concettualmente una redzone a sinistra e una a destra.
- Le redzones hanno una lunghezza fissa e configurabile.
- Nella pratica, la **redzone destra di un oggetto può coincidere con la redzone sinistra** dell'oggetto successivo, riducendo lo spreco di memoria (però concettualmente ce ne è una a destra e una a sinistra).

Questo meccanismo introduce memoria "non valida" tra le aree utilizzabili (poiché riservata ad ASan), diminuendo la densità effettiva ma aumentando la sicurezza del programma.



Shadow memory Durante l'esecuzione, ASan mantiene una struttura parallela detta **shadow memory**, che rappresenta lo stato di validità della memoria del processo:

- I blocchi validi (in blu nella memoria del processo) sono marcati in verde nella shadow memory.
- Le redzones sono marcate in rosso.
- Ogni accesso a memoria (che è generalmente un'operazione costosa) viene strumentato con una funzione tipo `isPoisoned()`, che consulta la shadow memory per verificare la validità.

Questo approccio permette ad ASan di rilevare accessi errati prima che causino crash o corruzione, con un overhead contenuto grazie a ottimizzazioni sulla gestione della shadow memory.

5.2.2 Instrumentation for Checks

Ogni operazione di lettura o scrittura sulla memoria viene automaticamente integrata (*instrumentata*) con un controllo sulla **shadow memory**. In pratica, viene inserita una chiamata a una funzione come `isPoisoned()` che verifica se l'indirizzo di memoria è all'interno di una redzone.

- Se l'indirizzo accede a una zona avvelenata, viene immediatamente segnalato un errore.
- Questo controllo è applicato a ogni accesso, sia in **lettura** che in **scrittura**.
- Il meccanismo è ottimizzato per ridurre l'overhead a runtime, bilanciando sicurezza e performance.

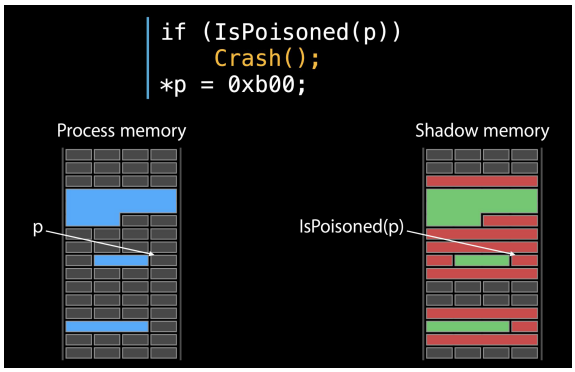


Figura 8: Strumentazione del codice per verificare la validità della memoria tramite la shadow memory

5.2.3 Memory layout is affected

L'introduzione delle *redzones* modifica la disposizione della memoria del programma: ogni oggetto è circondato da aree di memoria avvelenata, il che aumenta la quantità di memoria "non valida" per rilevare accessi errati.

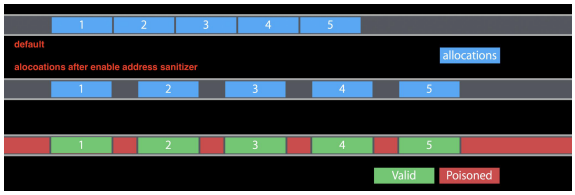


Figura 9: Impatto delle redzones sul layout della memoria

Questa modifica **riduce l'utilizzo efficiente della memoria**, ma **aumenta significativamente la possibilità di rilevare bug** legati alla sicurezza.

5.2.4 ASan è qualcosa di nuovo?

ASan può sembrare simile a strumenti esistenti come `Valgrind`, specialmente quando si usa il flag `-fsanitize=address`. Tuttavia, ci sono differenze importanti:

- **ASan è molto più veloce**: è integrato nel compilatore e sfrutta la *instrumentation* al livello di codice intermedio.
- **ASan rileva più bug**, specialmente su stack e globali, dove Valgrind può essere più debole.
- **Valgrind lavora a livello binario**: ha visibilità limitata sul codice sorgente e introduce un overhead molto elevato (fino a 30x).

Conclusione: è consigliabile usare entrambi gli strumenti in modo complementare. ASan è ideale per test frequenti e leggeri, mentre Valgrind è utile per analisi approfondite o casi complessi che richiedono visibilità sull'intero processo in esecuzione.

5.2.5 Stack variables

ASan (AddressSanitizer) può rilevare errori relativi a variabili stack e globali come soluzione a tempo di compilazione:

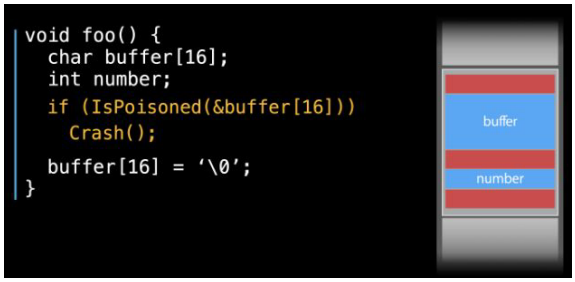


Figura 10: Esempio di buffer overflow nella funzione `foo()`

Questa opzione non è disponibile per strumenti a livello binario come `Valgrind` e `Dr. Memory`. Il layout di ciascun frame stack viene modificato per ospitare le redzone, e tutti gli operandi di offset vengono di conseguenza modificati.

5.2.6 Limitazioni di ASan

ASan può rilevare violazioni della sicurezza spaziale per overflow di buffer heap, stack e globali. Il meccanismo è lo stesso sia per le violazioni in lettura che in scrittura. **Presenta tuttavia le seguenti limitazioni di accuratezza:**

- **Natura probabilistica (per OOB):** cosa succede se l'accesso `Out-Of-Bounds` (OOB) va oltre la dimensione della redzone? Dopo una certa distanza, il **rilevamento diventa probabilistico**: se si va così tanto oltre da superare completamente la redzone, si potrebbe accedere a una zona valida ("verde"), generando un **falso negativo**. Oppure, si potrebbe saltare una zona valida e rilevare una redzone successiva, generando un errore non correttamente contestualizzato.
- **Non rileva accessi OOB intra-oggetto (all'interno di strutture):** Per quanto riguarda le struct, **non è possibile aggiungere una redzone tra i campi**, poiché questo comprometterebbe il programma: l'aritmetica dei puntatori all'interno delle struct si basa sulla sottrazione per navigare da un campo all'altro. ASan non fornisce protezione intra-oggetto, quindi non rileva overflow tra campi della stessa struttura.
- **Casi limite con accessi alla memoria non allineati:** Quando un'operazione di lettura/scrittura non rispetta l'allineamento naturale dell'architettura (es. leggere un `int` a indirizzi non multipli di 4), ASan può avere difficoltà a rilevare correttamente violazioni che attraversano parzialmente i confini delle redzones.
- **Falsi positivi con load widening (ampliamento del caricamento):** ASan può generare falsi positivi con buffer di grandi dimensioni: in questi casi la copia non avviene byte per byte ma per blocchi (chunk) gestiti dall'architettura. Con istruzioni AVX[12] si possono copiare fino a 512 byte alla volta, ma questo può causare errori se la lettura eccede i limiti del buffer. Questo fenomeno è noto come load widening e può portare a falsi positivi.

[12] Con il nome di Advanced Vector Extension (abbreviate in AVX) viene indicato un instruction set SIMD proposte e sviluppate da Intel

5.2.7 Temporal safety

ASan può rilevare casi di **use-after-free** grazie a un **meccanismo di quarantena**:

- La memoria liberata non è immediatamente disponibile per successive operazioni `malloc()` e viene inserita in quarantena;
- Non è possibile mantenere tutti i buffer in quarantena indefinitamente, poiché esaurirebbe la memoria disponibile;
- I buffer devono essere rimossi dalla quarantena dopo un certo periodo, riducendo l'efficacia nella rilevazione di use-after-free a lungo termine.

L'efficacia è parziale e dipende fortemente dal modo in cui il programma utilizza la memoria (ad esempio, un'elevata pressione sulla memoria forza i blocchi a uscire dalla quarantena prematuramente).
ASan fornisce anche strumenti per rilevare errori di tipo **use-after-return** e **use-after-scope**.

5.2.8 Diagnosing a violation



Figura 11: Output di errore di AddressSanitizer per un use-after-free

ASan mantiene traccia delle origini della memoria (es. siti di allocazione) e fornisce informazioni dettagliate sulle violazioni:

- Tipo di errore (es. `heap-use-after-free`)
- Indirizzo di memoria preciso e thread coinvolto
- Distanza dall'inizio della regione di memoria
- Stack trace completo del punto in cui avviene l'accesso
- Informazioni su quando e dove la memoria è stata allocata e liberata



Figura 12: Shadow memory map con codifica a colori delle diverse regioni di memoria

La visualizzazione della shadow memory utilizza un sistema di codifica a colori per rappresentare:

- Regioni accessibili (`00`)

- Redzones per heap, stack e variabili globali (`f1`, `f2`, `f3`, ecc.)
- Memoria liberata (`fd`)
- Memoria avvelenata dall'utente (`f7`)
- Altri stati speciali come use-after-scope (`f8`) e overflow di container (`fc`)

Queste informazioni diagnostiche sono fondamentali per identificare e risolvere rapidamente problemi di gestione della memoria.

5.2.9 Availability

ASan è ora disponibile nei principali compilatori: Clang, GCC, MSVC (Microsoft Visual C++). Per abilitarlo, si utilizza l'opzione di compilazione `-fsanitize=address`. I ricercatori continuano a lavorare per migliorarne l'accuratezza e la velocità di esecuzione.
ASan viene frequentemente utilizzato durante le campagne di test con fuzzing[13], poiché aumenta significativamente la probabilità di rilevare vulnerabilità di memoria.

5.3 Undefined Behavior Sanitizer (UBSan)

UBSan (Undefined Behavior Sanitizer) è un sanitizer specificamente progettato per **individuare comportamenti indefiniti nel codice**. Non si focalizza necessariamente su problemi che causano crash immediati, ma su **situazioni in cui il compilatore può prendere decisioni arbitrarie**, con conseguenti comportamenti imprevedibili, problemi di ottimizzazione o vulnerabilità di sicurezza.

Caratteristiche Include il rilevamento di:

- overflow di interi con segno;
- operazioni di shift fuori dai limiti (OOB) per il loro tipo di dati;
- dereferenziazione di puntatori disallineati o `null`;
- conversioni da/a/tra tipi in virgola mobile che causano overflow della destinazione;
- indici di array fuori dai limiti (quando i limiti possono essere determinati staticamente);
- e molti altri casi.

UBSan è stato originariamente sviluppato per il kernel Linux.

5.3.1 Overflow di interi con segno

L'overflow di interi con segno si verifica quando un'operazione su un intero eccede i limiti massimi o minimi che possono essere rappresentati dal tipo di dato. Nel caso di un intero a 32 bit, il valore massimo rappresentabile è `2147483647` (`0x7fffffff`). Se si tenta di sommare un valore maggiore a questo, si verifica un overflow, che può causare comportamenti imprevisti nel programma, come la scrittura di valori errati in variabili o l'esecuzione di operazioni non desiderate.

```
int main(int argc, char **argv) {
    int k = 0x7fffffff;  // valore massimo per un int a 32 bit
    k += argc;           // potenziale overflow
    return 0;
}
```

```
$ ./a.out
test.c:3:5: runtime error: signed integer overflow: 2147483647 + 1 cannot be
    represented in type 'int'
```

[13] Tecnica automatizzata di collaudo del software che prevede l'immissione di dati non validi, imprevisti o casuali come input di un programma tramite appositi programmi detti *fuzzer*.

5.3.2 Misaligned pointer

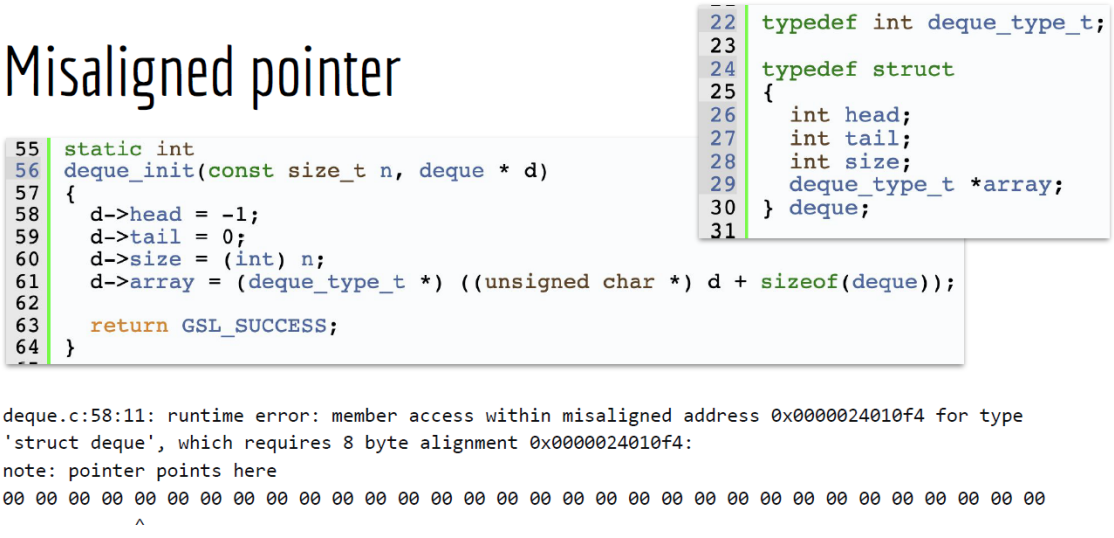


Figura 13: Esempio di errore per accesso a puntatore disallineato

In questo esempio, UBSan rileva un errore di allineamento del puntatore durante l'accesso alla struttura `deque`. Il messaggio di errore indica che l'accesso richiede un allineamento di 8 byte che non è rispettato all'indirizzo `0x000002461ef4`.

5.3.3 UBSan operation

Esistono casi in cui gli sviluppatori sono consapevoli dei potenziali problemi con alcune istruzioni e deliberatamente "giocano" con la semantica delle istruzioni. Ad esempio, nella generazione di formati di immagini, alcuni sviluppatori potrebbero sfruttare comportamenti indefiniti *by design* del compilatore per ottimizzare l'utilizzo della memoria. UBSan offre le seguenti caratteristiche per ovviare a queste situazioni:

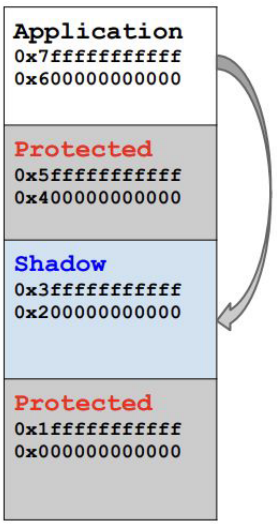
- Può essere configurato per **generare avvisi** invece di interrompere l'esecuzione;
- Fornisce **controlli sia a livello di gruppo che individuali** per le verifiche supportate;
- Può essere tipicamente **combinato con altri sanitizer** (e con il fuzzing), poiché ha un impatto limitato sulla memoria non utilizzando molti metadati;
- Disponibile in `Clang` e `GCC`.

5.4 Other sanitizers

5.4.1 MemorySanitizer

MSan (MemorySanitizer) è un sanitizer progettato per rilevare errori di uso di memoria non inizializzata (Use of Uninitialized Memory - UUM). Utilizza una memoria "ombra" (*shadow memory*) con una mappatura 1:1 per ogni byte.

Traccia tutte le istruzioni A differenza di AddressSanitizer (ASan) che traccia solo `read` e `store (load)`, **MSan deve tracciare tutte le istruzioni**, non solo letture e scritture. Questo perché, in alcuni casi, leggere più byte del necessario è accettabile per motivi di ottimizzazione, ma bisogna verificare se tali byte vengono effettivamente *usati* dal programma.



Per "uso" di un byte si intendono operazioni come:

- Chiamate di sistema (`system calls`)
- Operazioni aritmetiche o logiche su dati non inizializzati
- Uso come argomento di funzioni
- Condizioni nei rami di controllo (`if, loop, switch`)

MSan è disponibile solo per LLVM e richiede librerie compilate appositamente. A causa di limitazioni di implementazione, non può coesistere con ASan. Recentemente, QMSan ha semplificato notevolmente l'uso di MSan durante il *fuzzing*. Valgrind può essere considerato una combinazione di MSan con una versione solo heap di ASan, operando a livello binario.

5.4.2 ThreadSanitizer

ThreadSanitizer (TSan) è un sanitizer per il rilevamento delle *data races*, che si verificano quando due thread accedono simultaneamente a una stessa locazione di memoria e almeno uno dei due accessi è una scrittura. Esempio:

```
1  int Global;
2  void *Thread1(void *x) {
3      Global = 42;
4      return x;
5  }
6  int main() {
7      pthread_t t;
8      pthread_create(&t, NULL, Thread1, NULL);
9      Global = 43;
10     pthread_join(t, NULL);
11     return Global;
12 }
```

L'output è non deterministico, perché dipende dall'**ordine di esecuzione delle operazioni**. Se la lettura avviene prima della scrittura, viene letto un valore differente rispetto a una lettura dopo la scrittura. Questi sono problemi complessi di concorrenza.

Overhead importante TSan utilizza memoria "ombra" per tracciare tutti gli accessi alla memoria. È considerato uno strumento sperimentale con un **overhead significativo** in termini di tempo di esecuzione e uso di memoria, anche se è possibile configurare la *sensibilità* dell'analisi.

Il rilevamento delle *data races* è un problema complesso, e gli strumenti di rilevamento sono generalmente meno maturi ed efficienti rispetto a quelli per la sicurezza spaziale e temporale della memoria.

5.4.3 LeakSanitizer

LeakSanitizer (LSan) è un sanitizer specializzato nel rilevamento di **memory leaks**. È oggi parte integrante di ASan, che lo abilita di default, ma può anche essere utilizzato in modalità standalone.

LSan ha un overhead minimo durante l'esecuzione del programma, poiché gestisce i metadati in modo efficiente. L'analisi per rilevare i leak avviene alla terminazione del processo.

Combinando LSan con ASan, è possibile ottenere sia la rilevazione degli errori di sicurezza della memoria che dei memory leaks.

5.4.4 Altri sanitizers

Sanitizer type safety Esistono pochi sanitizers diffusi per la sicurezza dei tipi (type safety).

- LLVM offre **TypeSanitizer**, che rileva violazioni di aliasing tra tipi secondo le regole dello strict aliasing (ci sono casi più complessi).
- Alcuni progetti accademici, come `HexType` e `Type++`, affrontano problematiche più complesse legate alla sicurezza dei tipi.

Altri sanitizers Esistono anche sanitizers che non servono direttamente per la rilevazione di errori:

- **SanitizerCoverage**: utile per analizzare quanto codice è coperto rispetto a un insieme di input di test.
- **DataFlowSanitizer (DFSan)**: permette di tracciare il flusso dei dati all'interno del programma, utile ad esempio per l'analisi della propagazione di input non fidati.

5.4.5 Conclusioni

- I requisiti di sicurezza non sono sempre testabili direttamente.
- Molti bug rilevanti per la sicurezza non causano crash evidenti.
- I sanitizers aiutano nell'identificazione di errori silenziosi (silent faults).
- L'integrazione a compile-time migliora accuratezza e prestazioni.
- Sanitizers diversi rilevano classi diverse di vulnerabilità.

6 Laboratorio

6.1 Esempi di funzioni

Terminologia di ASAN Osserviamo che:

- I valori degli indici sono:

Valore esadecimale	Etichetta ASAN	Descrizione
`0xfa`	`ASAN_STACK_LEFT_REDZONE`	Redzone a sinistra di una variabile nello stack
`0xf1`	`ASAN_HEAP_LEFT_REDZONE`	Redzone a sinistra di una variabile heap
`0xf3`	`ASAN_HEAP_RIGHT_REDZONE`	Redzone a destra di una variabile heap

- `00`: byte accesibili
- `01` = solo il primo byte dei relativi 8 byte reali è accessibile; gli altri 7 sono off-limits.
- `[02]` = ASAN segnala un errore di accesso a un byte non accessibile (terzo o successivi), evidenziato tra parentesi quadre.
- Le **parentesi quadre** evidenziano il byte "incriminato", cioè quello che è stato oggetto dell'accesso illegale (lettura o scrittura). Esempio: Se è presente `[f3]`, significa che il programma ha provato ad accedere a quel byte, che si trova in una redzone (protetta), e quindi fuori dai limiti dell'allocazione valida.

6.1.1 Heap Out of Bound

Esempio dimostrativo per un accesso fuori dai limiti su un buffer allocato dinamicamente:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void out_of_bounds(char const *buf, unsigned int len) {
    for (int i = 0; i <= len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
}

int main() {
    unsigned int len = 5;
    char *buf = malloc(len);
    strncpy(buf, "Ciao", len);
    out_of_bounds(buf, len);
    return 0;
}
```

Se eseguiamo questo codice passando la stringa `"Ciao"`, verranno copiati fino a 5 byte (incluso il terminatore `"\0"` se presente). Da 0 a 5, sono 6 byte, tuttavia a seconda dell'esecuzione potrebbero venire creati altri byte finali in più. Inoltre, il ciclo accede anche all'indice `len`, andando fuori dai limiti del buffer. Compilando con `-g` possiamo includere informazioni di debug. Usando `valgrind`:

```
valgrind ./heap_oob
```

Con Valgrind l'indirizzo sarà diverso, ma è sempre un blocco da 6. Inoltre Valgrind segnala che l'allocazione (la `malloc`) avviene nella linea 12 e l'accesso fuori dai limiti (l'errore *out of bounds*) è alla linea 7.

AddressSanitizer Possiamo usare anche `AddressSanitizer`, un'alternativa moderna, compilando con:

```
gcc -m32 -g -fsanitize=address heap_oob.c -o heap_oob
./heap_oob
```

A differenza di `valgrind`, `AddressSanitizer` **si blocca al primo errore**, migliorando la localizzazione del problema. Valgrind prima stampava `buff 5` e faceva l'accesso in memoria, proprio perché questo non si ferma al primo errore, ma prosegue fino a che non c'è un crash completo[14].

Shadow memory In questo modo, con ASan, non viene eseguita proprio la `print` e ci restituisce lo stesso errore di valgrind (0 byte after a block), ma qui possiamo analizzarlo graficamente, poiché ASan fornisce una visualizzazione della `shadow memory`, utile per analizzare lo stato della memoria intorno all'indirizzo problematico.

```
Shadow bytes around the buggy address:
0x3eae00a0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00b0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00c0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00d0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00e0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
=>0x3eae00f0: fa fa [05] fa fa
0x3eae0100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
```

I byte `fa` indicano memoria inaccessibile. Il valore tra parentesi quadre segnala il byte che causa il problema.

6.1.2 Espansione del buffer

Modifichiamo la dimensione del buffer per simulare un'altra condizione di overflow:

```c
int main() {
    unsigned int len = 5 + 4;
    char *buf = malloc(len);
    strncpy(buf, "CiaoCiao", len);
}
```

Tuttavia, anche con una dimensione maggiore, possiamo comunque avere accessi fuori dai limiti se il codice non controlla correttamente la lunghezza. L'output di `AddressSanitizer` sarà simile:

```
Shadow bytes around the buggy address:
0x3eae00a0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00b0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00c0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00d0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x3eae00e0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
=>0x3eae00f0: fa fa 00 [01] fa fa 00 00 fa fa fa fa fa fa fa fa
```

Dopo aver cambiato il paradigma, passiamo ad usare `clang-15` come compilatore. Ora, eseguendo il codice, viene aggiunta un'ulteriore informazione sulla colonna, oltre a quella della riga, per facilitare l'individuazione del problema.
Invece di usare `-fsanitize=address`, utilizziamo `-fsanitize=undefined`. In questo caso:

- Con `-O0` (nessuna ottimizzazione) non otteniamo alcun risultato.
- Con `-O1` (ottimizzazione livello 1) non otteniamo risultati utili.
- Con `-O3` (ottimizzazione livello 3) il codice funziona correttamente, poiché `clang` ottimizza la gestione delle funzioni problematiche, rilevando l'overflow.

Se mettiamo la funzione `outofbounds` all'interno e commentiamo la funzione originale, con `gcc -O1` otteniamo un errore che segnala che stiamo cercando di caricare dati con uno spazio insufficiente. Ripristiniamo quindi il codice allo stato originale per testare ulteriormente.

6.1.3 Stack Buffer Overflow (Stack OOB)

Vediamo un esempio di overflow del buffer nello stack. Il codice sottostante potrebbe essere rilevato anche da UBSan (Undefined Behavior Sanitizer):

[14] Inoltre AddressSanitizer non dovrebbe essere usato in produzione, ma solo in fase di testing.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // this may be detected also by UBSan
    char buf[5];
    unsigned int len = sizeof(buf);
    strncpy(buf, "Ciao", len);
    for (int i = 0; i <= len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
    return 0;
}
```

Se compiliamo con GCC, `valgrind` non segnala errori, ma solo avvisi. Tuttavia, utilizzando `Clang` con `-fsanitize=address` (quindi con ASan), otteniamo degli errori, in particolare un `stack buffer overflow` (overflow del buffer nello stack). In questo caso, `AddressSanitizer` segnala il problema indicando sia la **riga** che la **colonna** in cui si verifica l'errore, e fornisce informazioni sulla localizzazione dell'errore nella memoria, mostrando che il problema è nell'accesso fuori dai limiti del buffer allocato nello stack.
Compilando con:

```
clang -m32 -g -fsanitize=address stack_oob_alt.c
```

Otterremo l'errore:

```
Shadow bytes around the buggy address:
0x3ec9ffb0: 00 00 00 00 00 00 00 00 00 00 00 00
0x3ec9ffc0: 00 00 00 00 00 00 00 00 00 00 00 f3
0x3ec9ffd0: 00 00 00 00 00 00 00 00 00 00 00 00
0x3ec9ffe0: 00 00 00 00 00 00 00 00 00 00 00 00
=>0x3eca0000: f1 f1 [05] f3 f3 f3 00 00 00 00 00
0x3eca0010: 00 00 00 00 00 00 00 00 00 00 00 00
```

Nel dump della `shadow memory`, possiamo vedere i byte intorno all'indirizzo problematico, evidenziando un accesso illegale a memoria che non è stata correttamente allocata o inizializzata. In questo esempio abbiamo 16 byte a sinistra e 24 a destra.
Modificando il codice per aumentare la dimensione del buffer a 9 byte (5+4), il risultato cambia come segue:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // this may be detected also by UBSan
    char buf[5+4];
    unsigned int len = sizeof(buf);
    strncpy(buf, "CiaoCiao", len);
    for (int i = 0; i <= len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
    return 0;
}
```

Compilando nuovamente con `Clang`:

```
clang -m32 -g -fsanitize=address stack_oob_alt.c
```

Il dump della memoria sarà simile a questo:

```
Shadow bytes around the buggy address:
0x3ec9ffd0: 00 00 00 00 00
0x3ec9ffe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ec9fff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3eca0010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x3eca0000: f1 f1 00 [01] f3 f3
0x3eca0020: 00 00 00 00
```

Qui vediamo che il buffer è stato sovrascritto a sinistra e a destra, con una visualizzazione della memoria in cui i byte problematici sono evidenziati come `00 [01]` (16 byte a sinistra e 16 a destra).

6.1.4 Analisi di accessi oltre la redzone

Modifichiamo il codice per testare un accesso che va oltre l'indice del buffer:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // this may be detected also by UBSan
    char buf[5+4];
    unsigned int len = sizeof(buf);
    strncpy(buf, "CiaoCiao", len);
    for (int i = 0; i <= len+2; i+=2)
        printf("buf[%d] = %c\n", i, buf[i]);
    return 0;
}
```

Se non utilizzassimo `len+2`, il ciclo si fermerebbe all'indice 10, poiché incrementiamo `i` di 2 ad ogni iterazione e quindi il programma veniva solo fermato (no falso positivo redzone successiva). Con `len+2`, il programma accede all'indice 11, che è fuori dai limiti del buffer `buf[0-8]`. Per superare la protezione della redzone, è necessario un accesso molto più distante, ad esempio `buf[35]`. Questo rende la **difesa probabilistica**: è possibile configurare redzone più grandi, ma questo degrada le performance. `AddressSanitizer` ridimensiona la redzone in base alla variabile allocata, in questo caso ritenendo che una dimensione di 16 byte sia generalmente sufficiente.

Aumento del buffer Se aumentiamo la dimensione del buffer, ad esempio a 128 byte, e tentiamo di accedere a `buf[129]`, `AddressSanitizer` creerà una redzone più grande a destra, ma non a sinistra per evitare di invadere lo spazio del chiamante. Con un buffer molto grande (ad esempio `buf[1024]`), la redzone sarà ulteriormente ampliata:

```
0x3ed40040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed40050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed40060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed40070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x3ed40080: 00 00 [f3] f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3
0x3ed40090: f3 f3 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed400a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed400b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed400c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3ed400d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Anche `UBSan` è in grado di rilevare questo tipo di errore. È possibile utilizzare entrambi i sanitizer insieme: `UBSan` identifica l'errore per primo ma non termina l'esecuzione, permettendo ad `AddressSanitizer` di trovare l'errore e poi interrompere il programma.

6.1.5 Debug con GDB e sanitizer

Tornando alla versione originale del codice, possiamo compilarlo con `AddressSanitizer` ed eseguirlo all'interno di `GDB` per un'analisi più approfondita. `AddressSanitizer` fornisce la funzione `__asan_report_error` (`_asan::ReportGenericError`) che possiamo utilizzare come punto di interruzione in `GDB`. Dopo aver avviato il debugger con `start`, impostiamo il **breakpoint**:

```
(gdb) b _asan_report_error
(gdb) continue
```

A questo punto, il programma si eseguirà fino a raggiungere il breakpoint quando viene rilevato un errore. Possiamo quindi esaminare il backtrace con il comando `bt` per identificare la sequenza di chiamate che ha portato all'errore. Avendo compilato con `gcc` e avviato con `gdb`, possiamo vedere anche gli indirizzi di memoria incriminati. Si possono anche visualizzare le funzioni della `load` rotta per verificare. Se proseguiamo con `cont` in `GDB`, il programma continuerà l'esecuzione fino a generare un errore non recuperabile da `AddressSanitizer`.

LLDB Con `LLDB` è possibile impostare il breakpoint prima dell'esecuzione e quindi eseguire il programma con `run`. Questo permette di verificare che il debugger si fermi correttamente al breakpoint. A questo punto possiamo esaminare meglio gli argomenti della funzione per comprendere in dettaglio la natura dell'errore.

6.1.6 Stack Out of Bounds (Stack OOB)

Vediamo questo esempio particolarmente **subdolo**:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void out_of_bounds(char const *buf, unsigned int len) {
    for (int i = 0; i <= len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
}

int main() {
    // will all three variables get redzones?
    int other1 = 7;
    char buf[5];
    int other2 = 12;
    unsigned int len = sizeof(buf);
    strncpy(buf, "Ciao", len);
    out_of_bounds(buf, len);
    printf("Misc: %p %d", &other1, other2);
    return 0;
}
```

Compiliamo con `clang` usando il flag `-fsanitize=address`. Otteniamo:

```
=>0x3eca0000: f1 f1 04 f2[05] f3 f3 f3 00 00 00 00 00 00 00 00
```

Togliendo la `printf`, la redzone appare diversa:

```
=>0x3eca0000: f1 f1 [05] f3 f3 f3 00 00 00 00 00 00 00 00 00 00
```

Rimettendo la `printf` e stampando due valori, la redzone rimane identica. Questo avviene perché le **redzone si applicano solo agli oggetti per cui esiste un accesso tramite indirizzo**, come array o struct. Le variabili scalari come gli `int`, se non vengono indirizzate, non hanno redzone. È raro andare out-of-bounds su un `int`, a meno che non vi sia un cast errato o un'interpretazione errata della memoria.
Nell'output seguente:

```
=>0x3eca0000: f1 f1 04 f2 [05] f3 f3 f3 00 00 00 00 00 00 00 00
```

Il valore `04` rappresenta una shadow byte che segnala la fine dell'oggetto `other1` (e quindi la redzone), e `05` quella di `buf`.

Se spostiamo la `printf` e invertiamo i parametri:

```c
int main() {
    int other1 = 7;
    char buf[5];
    int other2 = 12;
    unsigned int len = sizeof(buf);
    strncpy(buf, "Ciao", len);
    out_of_bounds(buf, len);
    printf("Misc: %p %d", other2, &other1);
    return 0;
}
```

L'output della shadow memory sarà:

```
=>0x3eca0000: f1 f1[05] f2 f2 f2 04 f3
```

Dimostrando che le redzone vengono applicate solo dove effettivamente necessarie, ovvero agli oggetti indirizzabili.

6.1.7 Global OOB

Vediamo ora come AddressSanitizer protegge le variabili globali:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char other[3];
char buf[5];

void out_of_bounds(char const *buf, unsigned int len) {
    for (int i = 0; i <= len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
}

int main() {
    unsigned int len = sizeof(buf);
    strncpy(buf, "Ciao", len);
    out_of_bounds(buf, len);
    return 0;
}
```

Compilando con `clang-15` e `-fsanitize=address`, otteniamo sempre lo stesso errore, ma la shadow bytes diventa:

```
0x2adfdee0: [05] f9 f9 f9 03 f9 f9 f9
```

Il valore `05` indica la dimensione dell'oggetto `buf`, mentre `f9` è la redzone. In questo caso, ASan fa delle assunzioni conservative, creando redzone per proteggere solo ciò che è realmente necessario (in questo caso solo a destra, poiché a sinistra è l'inizio, poteva andare solo avanti), anche per ridurre l'overhead. Con `gcc`, a volte viene aggiunta più redzone. Inoltre dice che l'errore coinvolge una certa area, mostrando che l'errore è fuori da quest'area di tot bytes.
Aggiungendo un altro `char buf` (o comunque cambiando di poco), ovviamente ASan se ne accorge anche perche il layout della memoria viene cambiato. Con `clang` vengono fatte delle assunzioni per ridurre la memoria della redzone, generalmente queste assunzioni non portano problemi ASan non traccia nella shadow memory aree di memoria che non ritiene valide o non indirizzabili.

6.1.8 Memory Leak

Vediamo cosa succede se non viene fatta la `free` e c'è un problema di leak:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void out_of_bounds(char const *buf, unsigned int len) {
    for (int i = 0; i < len; ++i)
        printf("buf[%d] = %c\n", i, buf[i]);
}

int main() {
    unsigned int len = 5;
    char *buf = malloc(len);
    strncpy(buf, "Ciao", len);
    out_of_bounds(buf, len);
    return 0;
}
```

Questo programma ha un memory leak, in quanto la `malloc` non viene seguita da una `free`. Si può notare che:

- `valgrind` lo rileva senza problemi.
- `ASan` non lo rileva automaticamente a meno che non venga aggiunto il flag `-fsanitize=leak`.

Con `-fsanitize=leak`, il leak viene correttamente segnalato. Di norma ASan doveva trovarlo di per se, senza l'aggiunta di questo flag. In alcuni casi, il leak potrebbe non essere rilevato a causa di interferenze (ad esempio, la presenza di una `printf` può impedire la rilevazione). Commentando la `printf`, il problema viene correttamente segnalato anche senza flag, suggerendo una possibile limitazione o bug dell'analisi.

6.2 Libxml2

Risorse caricate su Classroom. `libxml2` è una libreria software per il parsing di documenti XML. In questo laboratorio vediamo come testare questa libreria con gli strumenti che conosciamo.

6.2.1 Download e compilazione della versione 2.9.2 di Libxml2

Vediamo la struttura della cartella `libxml2` in cui sono presenti vari script utili alla compilazione e al testing. Il progetto consiste nel prendere una versione di Libxml2 (in questo caso la 2.9.2, attualmente siamo alla 2.14) che contiene dei bug noti, utile per sperimentare il fuzzing (che vedremo meglio successivamente) e la verifica tramite sanitizers.

Download del codice sorgente

- Scaricare il codice sorgente di Libxml2, in particolare la versione 2.9.2.
- È stato utilizzato lo script `fetch.sh` per effettuare un `git clone` e creare la cartella `source`. Questo processo consente di ottenere la versione specifica da testare.

Compilazione del codice

- La compilazione viene eseguita tramite lo script `build.sh`, che deve essere lanciato dalla cartella `source`.
- È essenziale scegliere il compilatore (generalmente `clang`, con possibilità di utilizzare `gcc` a seconda delle esigenze) e impostare adeguate opzioni di ottimizzazione.
- Per le esigenze di debug, è possibile compilare il progetto senza ottimizzazione (ottimizzazione pari a zero) per una migliore analisi, oppure abilitare ottimizzazioni realistiche per simulare un ambiente di produzione.
- Viene utilizzata l'opzione `-fno-omit-frame-pointer` che assicura il mantenimento dei frame pointer, fondamentale per ottenere stack trace più complete durante l'analisi dei crash.
- La compilazione avviene creando due binari:
  - uno **con sanitizers** (ad esempio **Asan**), al fine di rilevare errori di memoria quali buffer overflow e uso di memoria non inizializzata.
  - uno **con "noasan"** per confrontare la precisione del rilevamento dei bug: la compilazione di una versione "noasan" del binario (con lo script `build_noasan`) serve per confrontare il comportamento del parser con e senza l'ausilio di sanitizers.

6.2.2 Test harness e test suite sui file XML

Test harness specifico Una volta compilata la libreria, si può realizzare un **test harness** [15] per far eseguire alla libreria una serie di test:

- Notiamo che il parser è composto da due componenti fondamentali: il `target` e il `driver`.
- Il `target` contiene il codice per chiamare le funzioni cruciali di Libxml2, in particolare:

[15] Insieme di driver configurati per automatizzare i test di un'applicazione o componente, simulando l'ambiente di esecuzione reale.

  - Si occupa di configurare il parser, impostando ad esempio la funzione di errore tramite la chiamata a `XMLSetGenericErrorFunc`. Questa configurazione permette di ignorare o gestire errori non critici.
  - L'invocazione della funzione `xmlReadMemory` per interpretare un file XML letto in memoria. In questo modo non è necessario avere un file fisico; il parsing avviene sul buffer passato.
  - Una volta terminato il parsing, viene effettuata la deallocazione della memoria.

- Il `driver` esegue il test vero e proprio: legge gli input (file XML) direttamente dalla memoria e li passa al parser. In questo modo si stressa il parser simulando una condizione di ingresso massivo o di input "buggati".

Esecuzione della test suite

- La test suite è fornita tramite una cartella contenente numerosi file XML (alcuni validi, altri contenenti bug intenzionali) utili a mettere alla prova il comportamento del parser.
- Il comando `LIBXML2_test <path_al_file_XML>` viene utilizzato per eseguire i test. Questo permette di identificare errori e vulnerabilità nella gestione degli input.

6.2.3 Differenze harness e seed

Harness e **seed** sono due concetti fondamentali nel fuzzing, ma hanno ruoli molto diversi:

- **Fuzzing Harness**
  - È un piccolo programma o funzione che collega il fuzzer al codice da testare.
  - Si occupa di ricevere l'input generato dal fuzzer e passarlo alle funzioni target (ad esempio, una funzione di parsing di una libreria).
  - Serve a "guidare" il fuzzer su come esercitare il codice, spesso gestendo anche l'inizializzazione e la pulizia delle risorse.
  - *Esempio:* una funzione che prende un buffer e lo passa a `xmlReadMemory` di libxml2.
- **Seed**
  - È un input iniziale fornito al fuzzer, spesso un file o una stringa, che rappresenta un esempio valido (o quasi valido) di input per il programma da testare.
  - I seed vengono usati come punto di partenza per generare nuovi input tramite mutazioni.
  - La qualità e varietà dei seed influenzano la copertura e l'efficacia del fuzzing.
  - *Esempio:* un file XML valido fornito come input iniziale per il fuzzing di libxml2.

In sintesi:

- Il **harness** è il "ponte" tra il fuzzer e il codice target.
- Il **seed** è l'input di partenza da cui il fuzzer genera nuovi test case.

Entrambi sono essenziali per una campagna di fuzzing efficace, ma svolgono ruoli distinti. Il collegamento nel processo di fuzzing è proprio questo:

$$\text{seed} \rightarrow \text{fuzzer} \rightarrow \text{harness} \rightarrow \text{codice da testare}$$

- `seed`: input iniziale fornito al fuzzer (es. file, stringa, ecc.)
- `fuzzer`: prende i seed, li muta/genera nuovi input e li invia al harness
- `harness`: riceve l'input dal fuzzer e lo passa al codice target nel modo corretto codice da testare: la funzione o il programma che si vuole testare per trovare bug o vulnerabilità

6.2.4 Analisi e risoluzione dei bug

L'obiettivo dell'esercitazione è quello di analizzare la robustezza del parser di Libxml2:

- La test suite eseguita (attraverso il comando `LIBXML2 test`) processa una serie di file XML. Alcuni di questi input sono volutamente "buggati" per stimolare il parser e far emergere errori.
- Alcuni errori riscontrati includono problemi di "conditional jump or move" e, in alcuni casi, un buffer overflow durante il parsing. Questi bug sono documentati – in parte – sul tracker ufficiale del progetto (ad esempio su Bugzilla) e sono associati a CVE noti.
- Si sottolineano le differenze di comportamento tra Asan e Valgrind:
  - `Asan` risulta più preciso nel riportare l'origine dell'errore, grazie a una migliore gestione della stack trace.
  - `Valgrind` può invece mostrare un gran numero di warning, alcuni dei quali non indicativi di bug reali; per questi casi si raccomanda l'uso di flag o file di suppression per limitare i messaggi non utili (vedi prossimo paragrafo).
- L'analisi del bug viene poi approfondita studiando il codice sorgente di Libxml2: si analizzano funzioni come `xmlParseDocument`, `xmlSwitch`, e `XMLBuffCreate` per individuare il punto in cui il buffer overflow viene scatenato, correlando i dati della Stack Trace con la documentazione sul bug.

Utilizzo di Valgrind

- Valgrind viene utilizzato per verificare errori che possono non essere evidenti con ASan, specialmente errori legati a stack e variabili globali.
- Per una corretta analisi, il professore ha illustrato l'uso di flag specifici per migliorare il confronto tra i risultati ottenuti con ASan e Valgrind. Ad esempio:
  - `--undef-value-errors=no`: sopprime gli errori relativi a violazioni UUM apparenti, riducendo i falsi positivi.
  - `--exit-on-first-error=yes`: interrompe l'esecuzione al verificarsi del primo errore, permettendo una diagnosi immediata.

Debug e sicurezza informatica

- L'attivazione del frame pointer tramite opzioni come `-Fenable_omit_frame_pointer` garantisce che in caso di crash si ottenga una stack trace completa, fondamentale per individuare esattamente la riga di codice (ad es. riga 10666 in `parser.c`) in cui si è verificato l'errore.
- Importante analizzare le stack trace e verificare la corrispondenza con i bug già documentati (ad esempio, tramite Bugzilla o CVE), per comprendere se un comportamento anomalo sia effettivamente una vulnerabilità di sicurezza.

6.2.5 Considerazioni finali

Raccomandazioni

- Un'attenta compilazione con opzioni adeguate e un test approfondito siano fondamentali non solo per il debugging, ma anche per garantire la sicurezza informatica del software.
- L'uso di sanitizers e strumenti come Valgrind sono fondamentali per individuare vulnerabilità che potrebbero essere sfruttate in un contesto di attacchi informatici.
- (bonus) È possibile sfruttare le versioni precompilate se la compilazione dalla sorgente dovesse risultare troppo onerosa o problematica in termini di risorse.

Riepilogo dei comandi principali

- `fetch.sh` – Scarica la versione 2.9.2 di Libxml2 in una cartella `source`.
- `build.sh` – Compila il progetto con il compilatore scelto (es. `clang`) e con le opzioni di ottimizzazione.
- `build_noasan` – Compila una versione senza sanitizer per il confronto.
- `LIBXML2_test <path>` – Esegue la test suite sui file XML, evidenziando possibili vulnerabilità.
- Opzioni di debug: `-Fenable_omit_frame_pointer` per garantire una corretta stack trace, e flag come `--undef-value-errors=no` per adattare il comportamento di Valgrind.

6.3 Mitigations

6.4 Bugs vs mitigations

6.4.1 Il software rimarrà vulnerabile

Il software privo di bug è un obiettivo irraggiungibile. Anche se le capacità di testing migliorano costantemente, possiamo rafforzare lo *stack* software circostante per rendere l'exploitation più difficile.

Sono stati proposti centinaia di meccanismi di difesa; solo pochi sono diventati *mainstream*. Le mitigazioni si concentrano su:

- i problemi più imminenti;
- soluzioni *cost-effective* (ovvero con overhead minimo);
- mantenimento della compatibilità e della facilità di *deploy*.

6.4.2 Distruggere lo stack per divertimento e profitto

Prima dell'introduzione di difese di sistema contro gli attacchi di code injection, un attaccante poteva scrivere istruzioni della CPU direttamente nei dati, sovrascrivendo lo *return address* nello stack.

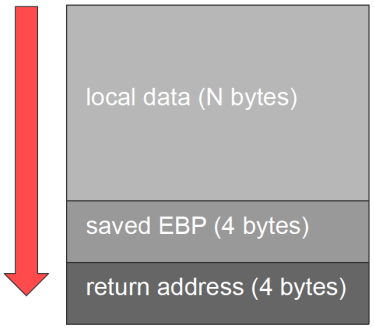


Figura 14: Struttura tipica dello stack durante una chiamata a funzione

6.4.3 Esempio di vulnerabilità: buffer overflow

Osserviamo questa funzione vulnerabile:

```
1 void foo() {
2     char buffer[10];
3     scanf("%s", buffer);
4     return;
5 }
```

Questa funzione è vulnerabile a un attacco di **buffer overflow**: la chiamata a `scanf` legge input arbitrario senza controlli di lunghezza, permettendo di sovrascrivere i dati nello stack, incluso il `return address`. Il **layout dello stack a runtime** risulta essere:

- `buffer` (10 byte)
- `saved EBP` (4 byte)
- `return address` (4 byte)

Se l'attaccante fornisce un input di lunghezza superiore a 10 byte, può corrompere la struttura dello stack. Ad esempio:

```
// Input dell'attaccante:
"aaaaaaaaaa" + "bbbb" + <indirizzo_shellcode>
     |            |
  playload       EBP
```

Dove:

- `"aaaaaaaaaa"` riempie il buffer (10 byte), può anche essere un'istruzione arbitraria;
- `"bbbb"` sovrascrive il `saved EBP`;
- `<indirizzo_shellcode>` sovrascrive il `return address`.

Quando la funzione `foo` termina, il flusso di esecuzione salta a `<indirizzo_shellcode>`, cioè alla parte del buffer contenente il codice malizioso.

Esecuzione di shellcode nello stack Quindi, un possibile caso con la funzione di prima e input:

```
// Input dell'attaccante:
"<instructions>" + "bbbb" + &buffer
     |            |
  playload       EBP
```

Un possibile **payload** contiene istruzioni per eseguire `/bin/sh` usando la syscall `execve`:

```
push 0x68732f       // "/sh"
push 0x6e69622f     // "/bin"
mov ebx, esp        // ebx punta a "/bin/sh"
mov ecx, 0x0        // ecx = NULL (argv)
mov edx, 0x0        // edx = NULL (envp)
mov eax, 0x0b       // eax = syscall execve
int 0x80            // esegue syscall
```

Spiegazione dello shellcode[16]:

- Le stringhe `"/bin/sh"` sono inserite direttamente nello stack (in formato esadecimale).
- `ebx` punta a queste stringhe.
- `ecx` e `edx` sono azzerati (parametri nulli per la syscall).
- `eax = 11` richiama la syscall `execve`.
- `int 0x80` invoca l'interrupt per eseguire la syscall.

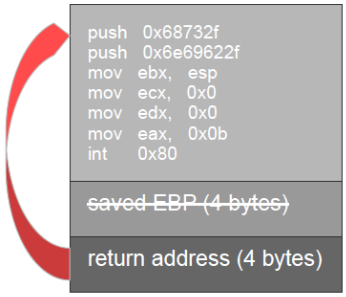
Disposizione finale nello stack:

1. Il buffer contiene: `shellcode` + padding.
2. `saved EBP` viene sovrascritto con 4 byte qualsiasi (es. `"bbbb"`).
3. `return address` punta all'inizio del buffer (dove risiede la shellcode).

Funzionamento della freccia nel ritorno La freccia curva rappresenta il salto dell'esecuzione verso la **shellcode** contenuta nel buffer. L'attaccante sovrascrive il `return address` con l'indirizzo del buffer, in modo che al termine della funzione il processore esegua direttamente il codice malevolo.

Nota sulla mitigazione tecniche moderne come `stack canaries`, `ASLR`, e `DEP/NX` rendono questi attacchi molto più complessi da eseguire, ma sono ancora rilevanti in contesti legacy o embedded.

[16] In informatica uno shellcode è un programma in linguaggio assembly che tradizionalmente esegue una shell. Uno shellcode può essere utilizzato per sfruttare un bug, o più in generale di eseguire codice arbitrario.



6.5 DEP

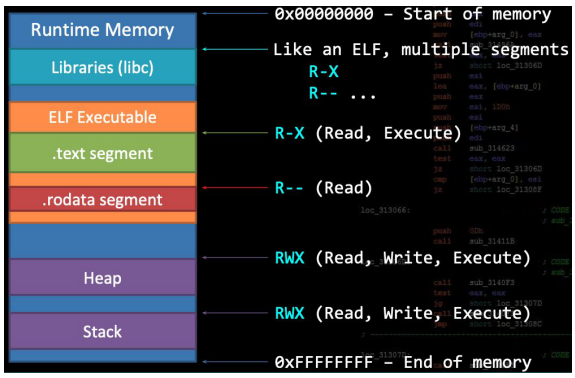
Per contrastare le vulnerabilità di code injection, i sistemi operativi hanno introdotto una mitigazione efficace: impedire l'esecuzione di codice dalle aree di memoria scrivibili. Questa politica è nota come **DEP** (Data Execution Prevention), **NX** (No-eXecute) o **WX̂** (Write xor Execute).
Il principio alla base di DEP è semplice ma potente:

- Nessun segmento di memoria dovrebbe mai essere **Writable** e **eXecutable** contemporaneamente;
- L'unica eccezione è quando viene esplicitamente richiesto dal programma stesso (es. JIT compilers).

6.5.1 Processo in esecuzione senza DEP

Senza DEP, le regioni di memoria come lo `stack` e lo `heap` possono essere sia scrivibili che eseguibili (`RWX`), rendendo possibile l'iniezione ed esecuzione di codice malevolo.

- **Segmenti come `text` e `rodata`**: normalmente `R-X` o `R--`.
- **Heap e Stack**: potenzialmente `RWX` senza DEP.



Con DEP, invece, si toglie come attributo `Execute` per heap e stack:



6.5.2 DEP è stato solo il primo passo

Con DEP attivo, un tentativo di eseguire codice iniettato nello `stack` o nello `heap` causa un errore di segmentazione (`SEGFAULT`). Questo tipo di protezione è stato introdotto:

- Linux e Windows XP SP2: Agosto 2004
- macOS 10.5: Giugno 2016

DEP rappresenta una delle principali difese da aggirare nelle moderne tecniche di exploit. Tuttavia *non è sufficiente per prevenire l'esecuzione di computazioni malevole*

6.5.3 Perché DEP non è sufficiente?

Un attacco di tipo exploitation moderno può essere scomposto in due fasi:

1. **Hijack del flusso di controllo** (*Control Flow Hijacking*)
2. **Esecuzione del payload iniettato**

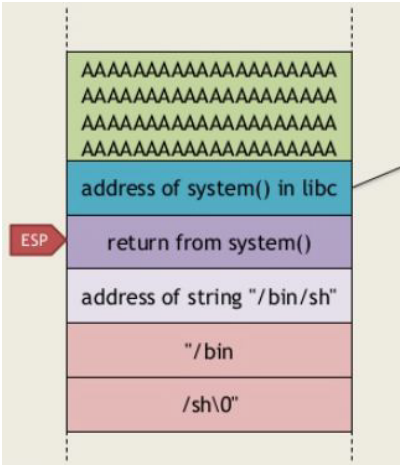
DEP è efficace solo nella seconda fase: impedisce l'esecuzione diretta del codice iniettato. Tuttavia, non impedisce di deviare il controllo verso codice già presente e legittimo (es. ROP - Return Oriented Programming).

6.6 Address randomization

6.6.1 Attacco ret2libc

Gli attaccanti hanno iniziato a scrivere exploit concatenando chiamate a codice già esistente nelle librerie, come la `libc`. Questa tecnica, detta **ret2libc**, consiste nel redirezionare il flusso di controllo verso funzioni esistenti invece di iniettare nuovo codice. Tuttavia, questa tecnica presenta limitazioni:

- Richiede che le funzioni desiderate siano già disponibili;
- Non consente facilmente decisioni di controllo condizionale.



6.6.2 Code Reuse Attacks: gadget e ROP

Per superare i limiti del `ret2libc`, sono nate le tecniche di **code reuse**, che sfruttano frammenti di codice (**Gadget**) già presenti nel programma o nelle librerie. Un **gadget** è una breve sequenza di istruzioni utili, spesso terminante con una istruzione di concatenazione:

- **Return-Oriented Programming (ROP)**: catena di gadget terminano con `ret`;
- **JOP (Jump-Oriented Programming)**: catena di gadget terminano con `jmp`.

Combinando questi gadget in catene (ROP chains), è possibile ottenere potenza espressiva Turing-completa, senza violare le protezioni DEP.

6.6.3 Esempio di catena ROP

Il **Return-Oriented Programming (ROP)** sfrutta gadget, ovvero sequenze di istruzioni terminate da `ret`, già presenti nel programma o nelle librerie. Ogni gadget svolge una piccola operazione e, grazie all'istruzione `ret`, trasferisce il controllo al gadget successivo, il cui indirizzo è specificato dallo stack.
L'attaccante costruisce una **ROP chain** scrivendo sullo stack una serie di indirizzi, ciascuno puntatore a un gadget. In questo modo è possibile concatenare operazioni arbitrarie senza iniettare codice nuovo, eludendo così la protezione **DEP**.

Esempio Costruzione di una stringa `/bin/sh` in memoria:

- Celle blu (indirizzi dei gadget) contengono le istruzioni da eseguire;
- Celle rosse contengono i dati da scrivere (le stringhe `/bin` e `/sh`);
- Obiettivo: scrivere la stringa `/bin/sh` in una zona di memoria scrivibile.

```
1 mov ebx, writeable_addr; ret    // Imposta EBX su un indirizzo scrivibile
2 pop eax; ret                    // Carica "/bin" in EAX
3 "//bin"                         // Dato: stringa "/bin" (rossa)
4 mov [ebx], eax; ret             // Scrive "/bin" in [ebx]
5 pop eax; ret                    // Carica "/sh" in EAX
6 "//sh"                          // Dato: stringa "/sh" (rossa)
7 mov [ebx+4], eax; ret           // Scrive "/sh" in [ebx+4]
```

Nota: nonostante DEP impedisca l'esecuzione di codice in memoria scrivibile, la ROP chain aggira questa protezione eseguendo solo codice già presente e legittimo.

6.6.4 Scoperta dei gadget

Le tecniche ROP richiedono la conoscenza precisa degli indirizzi dei gadget. Nei binari ELF o PE compilati con vecchi compilatori, gli indirizzi erano facilmente prevedibili, facilitando gli attacchi. Per evitare un attacco, si randomizzano le posizioni utilizzado *mitigation ASLR*.

6.6.5 Mitigazione: ASLR

Address Space Layout Randomization (ASLR) è una tecnica di mitigazione che **cambia randomicamente gli indirizzi dei segmenti di memoria ad ogni esecuzione**. Sposta gli indirizzi **a blocchi**, quindi se questi sono programmati per essere vicini devono rimanere tali. ASLR rende difficile per l'attaccante indovinare dove si trovano i gadget in memoria.
Effetti visibili: lanciando più volte `cat /proc/self/maps`, si nota che librerie, stack e heap si trovano in posizioni diverse.

6.6.6 Effetti dell'ASLR

L'**ASLR** modifica dinamicamente gli indirizzi base delle regioni di memoria (stack, heap, librerie, ecc.) ad ogni esecuzione di un programma. Tuttavia, le *posizioni relative* all'interno di ciascuna sezione restano invariate, per non compromettere la correttezza semantica del programma.

Durante il caricamento dell'eseguibile, il sistema operativo si occupa di eseguire tutte le azioni necessarie per assicurare il corretto funzionamento, nonostante la randomizzazione degli indirizzi.

L'ASLR rende inefficaci gli attacchi che si basano su gadget in posizioni di memoria note, come nel caso delle ROP chain. Per aggirare questa protezione, gli attaccanti devono disporre di una **primitive di leak**, ovvero una vulnerabilità che riveli indirizzi di memoria validi a runtime.

Timeline di adozione: 2004: OpenBSD (solo mmap) 2005: Linux 2.6.12 (stack + mmap), 2007: Windows Vista e Mac OSX 10.5 (sistemi e librerie), 2011: iOS 4.3 e OSX 10.7 (protezione completa).

6.6.7 Effetti della randomizzazione

Il livello di protezione offerto dall'ASLR dipende da tre fattori fondamentali:

1. L'**entropia** disponibile per ciascuna posizione randomizzata.
2. La **completezza della randomizzazione**, ovvero il numero e tipo di oggetti protetti.
3. L'**assenza di informazioni trapelate** (leak), che potrebbero vanificare l'efficacia della randomizzazione.

In prospettiva, l'estensione della randomizzazione comporta compromessi (trade-off) tra sovraccarico computazionale, complessità implementativa e benefici in termini di sicurezza.

6.7 PIE/PIC

6.7.1 PIE e PIC: Randomizzazione del codice

L'ASLR può randomizzare l'indirizzo di caricamento delle librerie grazie a specifiche opzioni adottate durante la loro **compilazione**. Tuttavia, randomizzare il codice degli eseguibili non è altrettanto semplice: molti programmi utilizzano riferimenti assoluti a variabili globali o salti diretti, che rendono difficile il supporto alla randomizzazione dinamica.

Nei vari SO Per le librerie Windows, durante la compilazione il compilatore può generare una cosiddetta tabella di rilocazione che tiene traccia di dove vengono utilizzati gli indirizzi globali nel codice. In fase di esecuzione, il loader corregge questi riferimenti in modo che corrispondano alla loro posizione corrente. Anche Linux supporta la rilocazione in fase di caricamento, ma per motivi di prestazioni in genere preferisce utilizzare il **codice indipendente dalla posizione (PIC)** per le librerie, che vede percorsi da posizioni relative e non assolute.

6.7.2 Position-Independent Code (PIC)

Il **Position-Independent Code** è una tecnica usata per **compilare il codice delle librerie** in modo che possa essere eseguito correttamente ovunque venga caricato in memoria.

Architetture moderne Nelle architetture moderne, la randomizzazione è facilitata dalla possibilità di esprimere gli indirizzi come offset rispetto all'instruction pointer corrente. L'uso di registri come `RIP` consente calcoli di indirizzamento relativi anziché assoluti.
PIC supera le limitazioni di indirizzamento attraverso un *livello di indirezione* per tutti i riferimenti a dati globali e funzioni mediante:

- **Global Offset Table (GOT)**: tabella contenente gli indirizzi effettivi delle variabili e funzioni,
- Il codice accede agli elementi attraverso la GOT anziché direttamente.

Architettura x86 Nell'architettura `x86`, che non offre accesso diretto al program counter, PIC utilizza un **thunk** (piccola routine) per caricare l'indirizzo corrente del codice e calcolare l'indirizzo base della GOT.

Esempio con codice Osserviamo:

```
1 int myglob = 42;
2 int ml_func(int a, int b)
3 {
4 return myglob + a + b;
5 }
```

Generazione di codice assembly per funzione PIC:

```
00000043 <ml_func>:
43c:    55                      push   ebp
43d:    89 e5                   mov    ebp, esp
43f:    e8 16 00 00 00          call   45a <__i686.get_pc_thunk.cx>
444:    81 c1 b0 1b 00 00       add    ecx, 0x1bb0
44a:    8b 81 f0 ff ff ff       mov    eax, DWORD PTR [ecx-0x10]
450:    8b 00                   mov    eax, DWORD PTR [eax]
452:    03 45 08                add    eax, DWORD PTR [ebp+0x8]
455:    03 45 0c                add    eax, DWORD PTR [ebp+0xc]
458:    5d                      pop    ebp
459:    c3                      ret
0000045a <__i686.get_pc_thunk.cx>:
45a:    8b 0c 24                mov    ecx, DWORD PTR [esp]
45d:    c3                      ret
```

Il *thunk* legge il valore EIP (Extended Instruction Pointer) dallo stack e cerca nella Global Offset Table (GOT). In questo caso, `ecx` ottiene l'indirizzo GOT e `myglob` si trova all'offset 0x10. Analizziamo il thunk:

- Il *thunk* `__i686.get_pc_thunk.cx` carica in `ecx` il valore corrente di `EIP` (simulato tramite `call`/`ret`) per calcolare l'indirizzo della GOT. Questo consente accessi RIP-relative anche su architetture prive di registri dedicati.
- **Nota (*):** la chiamata `call 45a` utilizza una vera chiamata di funzione che, come effetto collaterale, fa sì che l'indirizzo di ritorno (EIP dell'istruzione successiva) venga automaticamente inserito nello stack. Il thunk `__i686.get_pc_thunk.cx` semplicemente recupera questo valore dallo stack e lo salva in `ECX` prima di ritornare.

6.7.3 PLT

La Procedure Linkage Table (PLT) è un meccanismo che permette il linking dinamico di funzioni esterne. Con gli indirizzi di codice, c'è un ulteriore passaggio di indirezione attraverso la PLT.

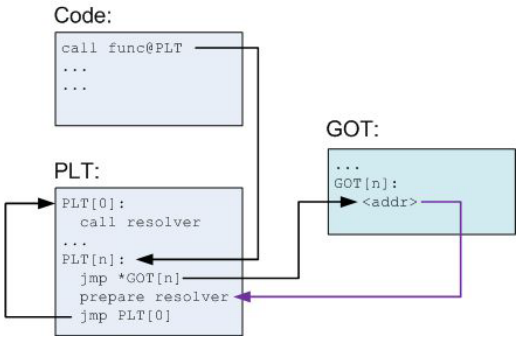


Figura 15: *
Al primo utilizzo, il target viene risolto e la GOT viene aggiornata. Le chiamate successive saranno più veloci.

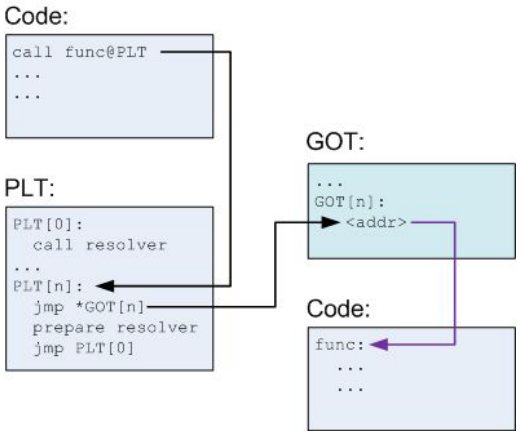


Figura 16: *
Con la GOT aggiornata, i successivi salti vanno direttamente alla funzione.

6.7.4 PIC su x64

Su architetture x64, l'accesso RIP diretto rende il meccanismo PIC più efficiente:

```
00000000000005ec <ml_func>:
5ec:    55                      push   rbp
5ed:    48 89 e5                mov    rbp, rsp
```

```
5f0:    89 7d fc                mov    DWORD PTR [rbp-0x4], edi
5f3:    89 75 f8                mov    DWORD PTR [rbp-0x8], esi
5f6:    48 8b 05 db 09 20 00    mov    rax, QWORD PTR [rip+0x2009db]
5fd:    8b 00                   mov    eax, DWORD PTR [rax]
5ff:    03 45 fc                add    eax, DWORD PTR [rbp-0x4]
602:    03 45 f8                add    eax, DWORD PTR [rbp-0x8]
605:    c9                      leave
606:    c3                      ret
```

PIC può essere controllato con il flag `-fPIC` durante la compilazione (ed è obbligatorio su x64).

6.7.5 Implicazioni per la sicurezza

PIC è fondamentale per la sicurezza moderna perché:

- Facilita l'Address Space Layout Randomization (ASLR), una protezione contro attacchi di tipo return-to-libc e ROP (Return-Oriented Programming)
- Permette di caricare librerie condivise in indirizzi arbitrari, rendendo più difficile lo sfruttamento di vulnerabilità
- Rende più complessi gli attacchi che dipendono da indirizzi assoluti conosciuti

L'uso combinato di PIE (Position Independent Executable) e PIC è parte delle moderne tecniche di hardening di sistema.

6.7.6 Position-Independent Executable (PIE)

Quando si parla di codice di programma, la randomizzazione prevede diverse disposizioni. Un eseguibile indipendente dalla posizione (PIE) è un meccanismo che favorisce l'ASLR (Address Space Layout Randomization).

Funzionamento Con PIE, ogni indirizzo è espresso come relativo a RIP (Return Instruction Pointer) durante la generazione del codice, eliminando la necessità di rilocazione (ad esempio, le chiamate non richiedono più il PLT, Procedure Linkage Table). Questo approccio migliora la sicurezza, poiché rende più difficile per un attaccante prevedere l'indirizzo di memoria delle funzioni o delle variabili.

Più snello PIE è più snello rispetto a PIC (Position-Independent Code), il che facilita l'ASLR nelle librerie, migliorando ulteriormente la protezione contro gli attacchi di tipo buffer overflow e altre vulnerabilità basate su indirizzi.
Per compilare i tuoi eseguibili con questa funzionalità, usa l'opzione `-fPIE`:

```
gcc -fPIE -pie -o my_program my_program.c
```

6.7.7 Verso più randomizzazione?

In teoria, si potrebbero ottenere ulteriori benefici di sicurezza introducendo tecniche di randomizzazione più spinte:

- Cambiare l'ordine delle funzioni nel binario
- Introdurre padding o riordinare i blocchi base
- Randomizzare la disposizione dei campi nelle strutture

Tuttavia, queste strategie richiedono modifiche profonde al compilatore e potrebbero rompere il comportamento del programma. Per questo, non sono ancora state adottate come mitigazioni standard.

6.8 Altri meccanismi di protezione del compilatore

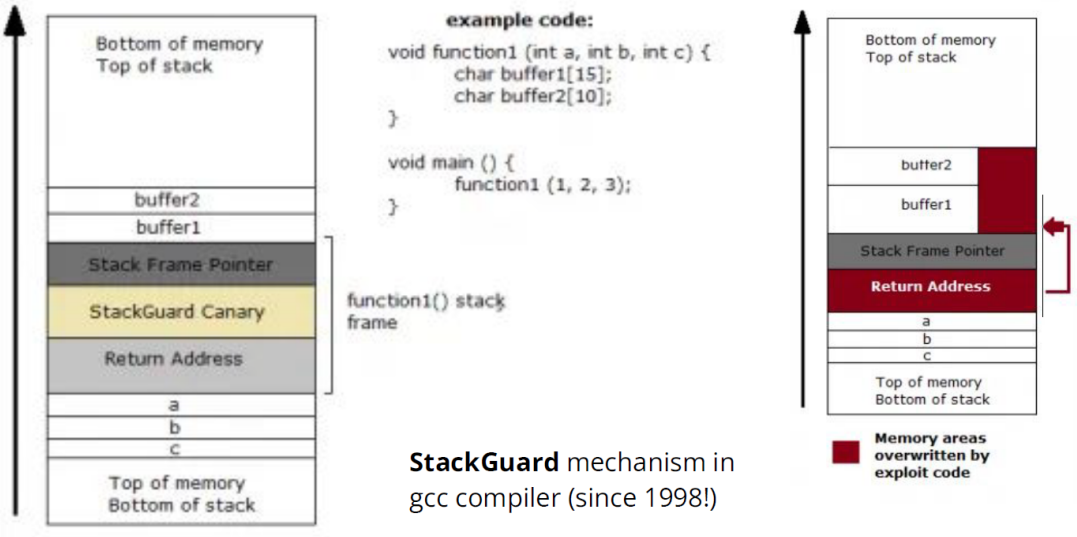
6.8.1 Stack Canaries

Gli *stack canaries* (o "biscotti della pila") aiutano a verificare l'integrità del frame dello stack prima del ritorno dalla funzione. Un canary è un intero casuale che controlla l'integrità della stack in questo modo:

- viene spinto nell'epilogo della funzione e verificato nel prologo;
- viene controllato contro un valore atteso (memorizzato in una *posizione sicura*);
- protegge contro gli overflow del buffer dello stack che corrompono l'indirizzo di ritorno;
- ha impatti sul caching e sulla disposizione dello stack.

Una limitazione importante è che **protegge solo contro sovrascritture continue**, a meno che l'attaccante non riesca a recuperare il valore del canary (ad esempio, tramite un *info leak* o una vulnerabilità nel formato della stringa).
Il meccanismo di StackGuard nel compilatore GCC, utilizzato sin dal 1998, è uno degli esempi di implementazione di questo tipo di protezione.

```
gcc -fstack-protector -o my_program my_program.c
```



6.8.2 Fortified source

FORTIFY è un insieme di estensioni di libc per catturare usi impropri di funzioni comuni, tra cui:

- overflow di buffer quando la dimensione dell'oggetto è conosciuta;
- argomenti incompleti (ad esempio, `open()` con `O_CREAT` ma senza i bit di modalità);
- argomenti errati (ad esempio, bit fuori dai limiti).

Utilizzo FORTIFY richiede l'uso dell'opzione `-D_FORTIFY_SOURCE=N`[17], dove `N==0` disabilita FORTIFY, mentre `N==1`, `N==2`, e `N==3` abilitano versioni sempre più rigorose.

Variante forzata Quando il compilatore non è in grado di determinare che gli input siano sicuri, la chiamata potrebbe essere sostituita da una variante rinforzata che effettua controlli aggiuntivi.

[17] Il compilatore fa dei check automatici e immagina se il buffet di destionazione ha abbastanza spazio

Esempio di Fortified Source Il seguente esempio mostra come l'opzione `-D_FORTIFY_SOURCE=2` possa aiutare a rilevare vulnerabilità comuni come un `buffer overflow`, ma solo se il compilatore riesce a determinare che l'input è potenzialmente pericoloso:

```c
int main(int argc, char *argv[]) {
    char buffer[10];
    char *q = argv[1];
    strcpy(buffer, q); // Potenziale overflow
    return 0;
}
```

Compilazione con controlli abilitati:

```
gcc -O3 -Wall -Wpedantic -Wextra -D_FORTIFY_SOURCE=2 test.c
```

Nota: Il controllo avviene solo se il compilatore può verificare che la dimensione di `buffer` è fissa e che `strcpy` è potenzialmente pericolosa. In caso contrario, il codice potrebbe non essere protetto. Questo mostra l'importanza del supporto statico del compilatore nel determinare la vulnerabilità.

6.8.3 Protezione dei trasferimenti di controllo

L'integrità del flusso di controllo (*Control Flow Integrity*, CFI) è un meccanismo difensivo per validare i trasferimenti di controllo, offrendo protezione contro gli attacchi di *control-flow hijacking*. L'idea è di garantire che il flusso di controllo non esca mai dal flusso predeterminato che è ritenuto valido da un'analisi a livello di sorgente o applicazione.
Per implementare CFI sono necessarie due disposizioni principali:

- costruzione del set di destinazioni (precisione vs. compatibilità);
- enforcement dinamico tramite controlli in fase di esecuzione.

CFI è un problema complesso, con molte tecniche proposte negli anni.

LLVM LLVM[18] supporta diversi schemi di *Control-Flow Integrity* (CFI) ottimizzati per l'uso in produzione. Queste protezioni permettono di rilevare trasferimenti di controllo non validi causati da cast errati o chiamate indirette malformate. È necessario compilare e linkare l'eseguibile con `-flto` (Link Time Optimization) per abilitare le protezioni CFI.
Ecco alcune delle opzioni disponibili:

```
// Abilita controlli rigorosi sui cast
-fsanitize=cfi-cast-strict

// Cast da tipo base a tipo derivato errato
-fsanitize=cfi-derived-cast

// Cast da void* o tipo non correlato a tipo errato
-fsanitize=cfi-unrelated-cast

// Chiamata non virtuale tramite oggetto con vptr errato
-fsanitize=cfi-nvcall

// Chiamata virtuale tramite oggetto con vptr errato
-fsanitize=cfi-vcall

// Chiamata indiretta a funzione con tipo dinamico errato
-fsanitize=cfi-icall

// Chiamata indiretta tramite puntatore a membro funzione errato
-fsanitize=cfi-mfcall
```

6.8.4 Protezione dai trasferimenti di controllo

CFI protegge i trasferimenti di controllo in avanti, ma come proteggere gli indirizzi di ritorno?

[18] Low Level Virtual Machine, infrastruttura di compilazione

Shadow stack Una soluzione più forte rispetto ai canaries è la *shadow stack*, che tiene traccia esplicitamente dei dati di controllo (indirizzo di ritorno, puntatore del frame, ecc.). Le difese basate sullo shadow stack mantengono una rappresentazione speculare dei valori critici dello stack del programma, aggiornandola durante i trasferimenti di controllo e validandola.

Non-mainstream Tuttavia, gli *shadow stacks* non sono ancora diventati una soluzione mainstream a causa di questioni legate all'accuratezza e alle performance. Tuttavia, l'assistenza hardware per questa tecnologia (ad esempio, Intel CET) sembra promettente.

```
gcc -fsanitize=shadow-stack -o my_program my_program.c
```

7 Fuzzing

7.1 Coverage da una test suite

Il **functional testing** verifica che ogni funzionalità del software rispetti le specifiche richieste. Tuttavia, da solo, non è sufficiente per individuare bug rilevanti per la sicurezza.

Lo **structural testing** mira a coprire il codice (es. tramite unit test), ma anch'esso è limitato.

Il testing efficace richiede tecniche dinamiche automatizzate. La copertura strutturale (es. *statement coverage*) misura se ogni linea è eseguita. Tuttavia, toccare una linea non significa necessariamente triggerare un bug.

```
int func(int elem, int *inp, int len) {
    int ret = -1;
    for (int i = 0; i <= len; ++i) {
        if (inp[i] == elem) {
            ret = i;
            break;
        }
    }
    return ret;
}
```

Con input: `elem = 2, inp = {1, 2}, len = 2`, la coverage è solo apparente: il loop non viene eseguito completamente e non viene raggiunto il potenziale buffer overflow.

7.1.1 Branch vs path coverage

- Il **branch coverage** migliora lo statement coverage ma è ancora incompleto. Per esempio, può toccare ogni ramo ma non in combinazioni significative.
- La **path coverage** mira a coprire tutte le sequenze di esecuzione. Tuttavia, non scala: il numero di path può esplodere esponenzialmente, specialmente con loop.

Una via di mezzo è contare le esecuzioni dei rami (0, 1, n volte), utile per scoprire bug nei loop. Tool come `gcov`, `lcov` o `SanitizerCoverage` possono aiutare a misurare la coverage.

7.1.2 Generazione automatica di test case

Generare test efficaci è complesso. Esistono tre approcci principali:

- **White-box testing**: analizza il codice, raccoglie condizioni e ragiona su di esse.
- **Grey-box testing**: esegue il programma più volte osservando le differenze.
- **Black-box testing**: ignora il codice, si affida a strategie intelligenti per generare input (es. basati su grammatiche per formati strutturati).

Generare una test suite con buona copertura è un compito complesso. Le decisioni di controllo e di flusso dei dati nel codice possono guidare la generazione automatica dei test. Esistono principalmente due approcci:

- **Analisi statica**: consiste nel raccogliere le condizioni (es. `if`, `while`) e ragionare esplicitamente su di esse per generare test mirati.
- **Esecuzione dinamica**: si esegue il programma più volte con input diversi, identificando e sfruttando le differenze nei comportamenti osservati.

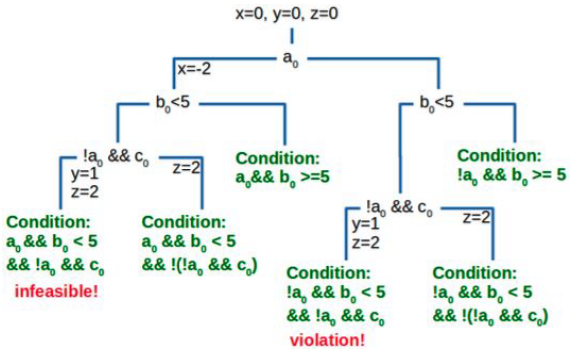
In alternativa, si può optare per il **black-box testing**, ovvero generare test senza conoscere il codice sorgente e misurare la copertura del codice solo successivamente.

7.1.3 Symbolic Execution

La *symbolic execution* trasforma i path di esecuzione in formule logiche su input simbolici. Poi usa un *solver SMT* per generare input che portano a determinati stati del programma.

```
void func(int a, int b, int c) {
    int x = 0, y = 0, z = 0;
    if (a) x = -2;
    if (b < 5) { }
    if (!a && c) y = 1;
    z = 2;
    assert(x + y + z != 3);
}
```

Si costruisce un *execution tree* dove ogni branch è una condizione. Si esplorano path fattibili, scartando quelli con condizioni insoddisfacibili (es. path infeasible).



Questo metodo è potente ma poco scalabile, poiché il numero di path cresce velocemente. Tuttavia, è efficace nel rilevare bug di sicurezza se applicato su codice critico.

Osservazioni conclusive Possiamo quindi concludere che

- Nessun metodo garantisce l'assenza di bug, solo la possibilità di trovarli.
- La coverage strutturale è necessaria ma non sufficiente.
- Approcci ibridi (grey-box + feedback) sono spesso i più pratici.
- Strumenti automatici (es. fuzzer evoluti) usano coverage feedback per migliorare i test iterativamente.

7.2 Fuzz testing

7.2.1 Fuzz Testing

Il *fuzz testing* (o semplicemente *fuzzing*) è una forma sofisticata di *random testing*, dove gli input per i casi di test vengono generati in modo semi-casuale. L'obiettivo principale del fuzzing non è verificare la correttezza funzionale, ma piuttosto assicurarsi che "non accadano cose negative" come:

- Crash del programma
- Eccezioni non gestite
- Non terminazione (hanging)

Questi comportamenti indesiderati possono rappresentare la base per vulnerabilità di sicurezza. Una caratteristica fondamentale del fuzz testing è che non ha alcuna conoscenza di come il programma dovrebbe comportarsi correttamente. Testa semplicemente il software con input inaspettati e dall'aspetto casuale ad alta frequenza per garantire che nessuno dei comportamenti problematici sopra citati si verifichi.

7.2.2 Fuzz Testing vs. Testing Standard

Standard testing	Fuzz testing
Human hand-written tests	Automatically generated inputs
Focused on expected use cases	Targets edge cases and unexpected inputs
Deterministic	Often involves randomness

Gli sviluppatori in genere scrivono test per i casi che si aspettano di incontrare. Inviando input strani o mal formati, il fuzzing trova bug gli sviluppatori non avrebbero mai considerato.

7.2.3 Origini del Fuzzing

Il termine "fuzz" fu introdotto da Barton P. Miller nel 1988. Insieme ai suoi studenti universitari, tentò di creare una suite di test per le utility a riga di comando UNIX:

- L'idea era generare input casuali e osservare i crash
- Nel 1988, riuscirono a far crashare una sorprendente percentuale (25-33%) delle utility testate
- Nel 2020, usando lo stesso metodo semplice su software moderno, hanno riscontrato che il 12% dei programmi testati su Linux andava in crash o in hang, il 19% su FreeBSD e il 15% su MacOS

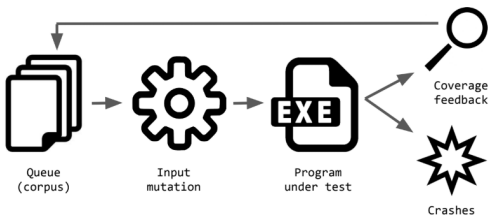
Da allora, il fuzzing si è evoluto significativamente, diventando uno strumento essenziale per il testing di sicurezza.

7.2.4 Come Funziona il Fuzzing

Un *fuzzer* è uno strumento che implementa il fuzzing con un **ciclo automatizzato** guidato dal feedback, che mantiene una **coda di casi di test**. Funziona attraverso i seguenti passaggi:

1. Inizializzazione della coda con un insieme di input iniziali (seed corpus)
2. Mutazione degli input dalla coda per crearne di nuovi
3. Esecuzione del programma con questi input
4. Osservazione dei fatti dell'esecuzione (es. crash) e raccolta di feedback
5. Mantenimento degli input interessanti per ulteriori mutazioni

Questo processo può essere rappresentato come un ciclo:



7.2.5 Considerazioni sulla Sicurezza

Il fuzz testing è estremamente utile per individuare vulnerabilità di sicurezza che potrebbero non essere rilevate tramite altre tecniche di testing. Si concentra su casi limite e input malformati che potrebbero essere ignorati nei test tradizionali. Ad esempio, il fuzz testing può scoprire *denial of service* (DoS) causato da input che portano il programma a non terminare mai o a consumare eccessive risorse di sistema.

Un caso interessante di vulnerabilità di tipo DoS è la *zipbomb*, un archivio compresso che, quando espanso, può riempire completamente il disco, generando file di dimensioni enormi da pochi byte di input. Questo tipo di vulnerabilità può essere utilizzato come attacco DoS.

7.2.6 Seed Corpus

Il *seed corpus* è un insieme di input validi o semi-validi che inizializzano la coda del fuzzer. Questi aiutano a guidare il fuzzer verso percorsi di codice realistici, superando la validazione degli input di base e arrivando alla logica interessante del programma. Anche un solo buon seed può sbloccare una copertura profonda se il mutatore è efficace. Senza seed, il fuzzer genererà molti input che portano a terminazioni precoci finché (si spera) non impara da solo come superare le fasi iniziali di validazione.

7.2.7 Mutazione degli Input

Il mutatore è un componente chiave del fuzzer: genera nuovi input partendo da uno o più input prelevati dalla coda. Applica una selezione di funzioni di mutazione, tra cui:

- Flip di bit, inserimenti/eliminazioni di byte
- Operazioni aritmetiche su interi
- Unione (splicing) di input diversi

Il mutatore può combinare strategie deterministiche e non deterministiche per selezionare e pianificare le mutazioni ad ogni tentativo di generazione di input. Questa fase viene spesso chiamata "havoc" (caos), in quanto applica trasformazioni apparentemente casuali ma controllate.

7.2.8 Esecuzione del Target

I fuzzer mirano a centinaia, se non migliaia, di tentativi di test al secondo. La performance è cruciale in due aspetti principali:

1. Ottimizzare le esecuzioni ripetute
2. Controllare come l'input raggiunge il codice da testare

Per il punto 1, ci sono diverse modalità specifiche:

- *Fork mode*: ottimizza il processo di creazione di nuovi processi
- *Persistent mode*: riutilizza lo stesso processo per più esecuzioni

Per il punto 2, il programma può ricevere input dalla linea di comando oppure lo sviluppatore scrive una funzione *harness* per leggere i dati del fuzzer e invocare il codice da testare.

7.2.9 Feedback di Coverage

Il meccanismo di feedback è fondamentale per l'efficacia del fuzzer. Per un dato caso di test, questo meccanismo assembla un profilo di esecuzione e lo confronta con la pluralità di esecuzioni fatte fino a quel momento. Se l'esecuzione mostra una novità (ma nessun bug), il caso di test viene considerato "interessante" e diventa parte della coda per un uso successivo nella generazione di casi di test. La logica è che mutare input interessanti ha più probabilità di rivelare ulteriori novità rispetto alla generazione di input a partire dai seed iniziali. Questo approccio è noto come *feedback-driven fuzzing*.

7.2.10 Ottimizzazioni del runtime

Per raggiungere alte prestazioni, i fuzzer implementano diverse ottimizzazioni:

- **Fork server**: il fuzzer controlla il processo di fork, riducendone il costo inizializzando il programma una volta sola e facendo fork dal punto di ingresso main()
- **Deferred initialization**: permette di fare fork ancora più in profondità nel programma, dopo fasi di inizializzazione indipendenti dall'input

- **Persistent mode**: riutilizza lo stesso processo per eseguire più input, evitando il costo di creare nuovi processi

Quest'ultima modalità richiede un harness ben costruito che pulisca lo stato tra le esecuzioni per evitare effetti collaterali e perdite di memoria.

7.2.11 Casi d'uso e Considerazioni

Il fuzzing è diventato uno strumento essenziale per il testing di sicurezza nelle aziende più avanzate. È particolarmente efficace per:

- Parser di formati di file (es. PNG, JPEG, XML)
- Protocolli di rete
- API e interfacce
- Librerie di terze parti

È importante notare che il fuzzing testa il software come effettivamente implementato, non come dovrebbe essere secondo una specifica. Questo è un punto di forza perché trova discrepanze tra specifiche formali e implementazioni reali, che spesso rappresentano punti deboli per la sicurezza. Una campagna di fuzzing tipicamente ha un budget di tempo (es. 24 ore) e può essere parallelizzata su più core per aumentare l'efficacia. La natura non deterministica del fuzzing significa che due campagne diverse produrranno set di test diversi, ma la copertura del codice tende ad essere simile.

7.2.12 Evoluzione del Fuzzing

Negli ultimi anni, il fuzzing ha visto significativi progressi, in particolare con l'integrazione di:

- Sanitizer (es. AddressSanitizer, UndefinedBehaviorSanitizer) per rilevare bug silenti
- Tecniche avanzate di mutazione e scheduling
- Fuzzing specifico per domini o formati particolari
- Fuzzing "smart" che comprende la struttura dei formati

Da una tecnica di nicchia, il fuzzing è diventato uno standard de facto per il testing di sicurezza, grazie alla sua efficacia nel trovare vulnerabilità che altri approcci potrebbero non rilevare.

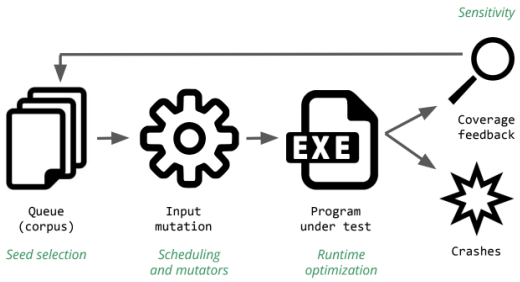
7.3 Fuzzing: Tecniche e Sfide

I *fuzzer* sono strumenti per testare software tramite l'esecuzione ripetuta con input generati automaticamente. Il processo di fuzzing presenta diversi componenti principali:

- **Queue (corpus)**: Raccolta di input di test
- **Input mutation**: Modifica degli input esistenti per generarne di nuovi
- **Program under test**: Il programma da analizzare
- **Coverage feedback**: Informazioni sull'esecuzione del programma
- **Crashes**: I fallimenti rilevati durante l'esecuzione

Le sfide principali includono:

- Selezione del seed (*seed selection*)
- Schedulazione e mutatori (*scheduling and mutators*)
- Ottimizzazione del runtime (*runtime optimization*)
- Sensibilità del feedback (*sensitivity*)



7.3.1 Runtime Operation e Ottimizzazione

I fuzzer eseguono ripetutamente il programma sotto test. Per ridurre l'overhead di esecuzione, vengono adottate diverse tecniche:

- **Fork mode**: Il programma viene inizializzato una sola volta, poi si usa `fork()` dopo l'inizializzazione o al raggiungimento della funzione `main()`
- **Deferred initialization**: La duplicazione del processo avviene in un punto più avanzato nell'esecuzione, dopo fasi di inizializzazione indipendenti dall'input
- **Persistent mode**: Riutilizzo dello stesso processo per testare più input

Il *forkserver* è un meccanismo che supporta la modalità fork, evitando di ripetere l'inizializzazione ad ogni esecuzione.

7.3.2 Harness per il Fuzzing

Un *harness* è una funzione critica per testare librerie e applicazioni complesse. Questa funzione legge l'input dal fuzzer, prepara le strutture dati necessarie e invoca il codice da testare.
Esempio di harness per `libxml2`:

```
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    xmlSetGenericErrorFunc(NULL, &ignore);

    if (auto doc = xmlReadMemory(reinterpret_cast<const char *>(data), size,
                                 "noname.xml", NULL, 0))
        xmlFreeDoc(doc);

    return 0;
}
```

In questo harness:

1. Il fuzzer invoca questa funzione ad ogni esecuzione
2. I parametri `data` e `size` contengono l'input generato dal fuzzer
3. Gli errori vengono soppressi con `xmlSetGenericErrorFunc`
4. L'input viene passato a `xmlReadMemory` per il parsing
5. La memoria viene liberata con `xmlFreeDoc`
6. Il valore di ritorno 0 indica un'esecuzione completata senza crash

Per la *persistent mode*, l'harness deve anche gestire la pulizia delle risorse dopo ogni esecuzione per evitare memory leak. In questa modalità, il fuzzer potrebbe eseguire migliaia di iterazioni prima di terminare il processo e crearne uno nuovo. Il codice scritto dallo sviluppatore li "alimenta" al programma in fase di test.

7.3.3 Monitoraggio del Programma

Per integrare l'esecuzione dell'applicazione con il fuzzer, si possono utilizzare diversi approcci:

- **Con codice sorgente disponibile**: Si utilizzano estensioni del fuzzer per compilatori come clang e gcc per generare binari specializzati per il fuzzing
- **Con binari proprietari**: Si ricorre a tecniche alternative come la strumentazione binaria dinamica (tramite QEMU-User o DynamoRIO)

Le tecniche per i binari proprietari includono:

- **Static binary rewriting**: Modifica diretta del codice assembly, inserendo "trampoline" o riscrivendo completamente parti del codice
- **Dynamic binary instrumentation**: Esecuzione del binario originale in un emulatore che traduce e strumenta[19] le istruzioni prima dell'esecuzione

Queste tecniche sono più lente rispetto alla strumentazione a livello di compilatore, ma offrono compatibilità con software proprietario.

7.3.4 Coverage Feedback

La maggior parte dei fuzzer moderni sono *coverage-guided* (o *grey-box fuzzer*): l'esplorazione del programma è guidata da meccanismi di feedback sulla copertura del codice.
Il feedback di copertura determina la sensibilità del fuzzer:

- **Sensibilità**: Capacità di riconoscere differenze significative nell'esecuzione di due casi di test
- Una sensibilità troppo bassa può far perdere percorsi interessanti
- Una sensibilità eccessiva causa "esplosione della coda" (troppe varianti simili)

I coverage-guided fuzzer sono chiamati anche **grey-box fuzzer** perché operano in una modalità intermedia tra black-box e white-box:

- Non sono completamente "black-box" (dove il fuzzer non ha alcuna visibilità sul funzionamento interno del programma)
- Non sono completamente "white-box" (dove il fuzzer avrebbe accesso completo al codice sorgente e alla struttura interna)

Invece, ottengono una "visione parziale" attraverso il feedback di coverage durante l'esecuzione, che fornisce alcune informazioni sullo stato interno senza richiedere una comprensione completa del codice. Questa visibilità limitata li posiziona in uno stato "grigio" intermedio, da cui il nome "grey-box".

7.3.5 Coverage Map

I fuzzer mantengono una *coverage map* durante l'esecuzione del programma. Quando una porzione di codice viene coperta, il fuzzer aggiorna una voce nella mappa e confronta con una mappa globale per rilevare novità nell'esecuzione.

[19] si riferisce al processo di aggiungere codice di monitoraggio o analisi al programma mentre viene eseguito, senza modificare il binario originale su disco. Quando un sistema come QEMU-User o DynamoRIO "strumenta" le istruzioni, significa che:

1. Intercetta le istruzioni originali del programma prima che vengano eseguite
2. Aggiunge istruzioni supplementari (il codice di "strumentazione") che servono a raccogliere informazioni
3. Modifica temporaneamente il flusso di esecuzione per includere queste istruzioni aggiuntive

Edge coverage L'**edge coverage** (o branch coverage) è la metrica più popolare nei fuzzer moderni. Combina:

- Tracciamento degli archi nel grafo di controllo di flusso
- Conteggio delle occorrenze ("hit counts") raggruppate in bucket di potenze di 2

Implementazione dell'Edge Coverage Per identificare gli archi di control-flow, esistono due approcci principali:

- **Approccio "classic"**: Si calcola a runtime un hash dei due blocchi base coinvolti in un salto o chiamata di funzione. Ad esempio, se abbiamo un'istruzione di salto `0x41414141: jmp 0x42424242`, il fuzzer calcola `hash_edge(0x41414141, 0x42424242)` per identificare univocamente questo edge.

  AFL In AFL (American Fuzzy Lop), l'implementazione classica funziona così:

  1. Si prende l'indirizzo sorgente e lo si shifta di 1 bit (per evitare valori speciali quando gli indirizzi sono vicini)
  2. Si combina con l'indirizzo destinazione usando un'operazione XOR
  3. Si applica un'operazione di modulo per limitare il risultato alla dimensione della coverage map

  Questo meccanismo è **probabilistico** e può generare collisioni: due branch diversi potrebbero essere mappati sulla stessa cella. È un compromesso necessario per mantenere la mappa di dimensioni gestibili.

- **Approccio "pcguard"**: Durante la compilazione, si associano identificatori univoci a ciascun edge, eliminando il rischio di collisioni. Questo approccio, più recente e sviluppato grazie ai sanitizer, funziona bene finché il numero totale di branch nel programma non è eccessivo (alcune centinaia di migliaia).

Meccanismo Il meccanismo di implementazione dell'edge coverage funziona così:

1. Quando viene eseguito un salto, si identifica l'edge usando uno dei metodi sopra descritti
2. L'identificatore dell'edge viene mappato a una specifica posizione nella coverage map (tipicamente usando una funzione hash)
3. Il contatore in quella posizione viene incrementato ogni volta che l'edge viene attraversato
4. Per ottimizzare lo spazio, anziché memorizzare il conteggio esatto, si utilizza un sistema di bucket basato su potenze di 2 (es. 1-2, 3-4, 5-8, 9-16, ecc.)

Questo sistema permette di distinguere tra:

- Edge mai percorsi (valore 0)
- Edge percorsi poche volte (bucket bassi)
- Edge percorsi molte volte, come i loop (bucket alti)

Vantaggi dell'Edge Coverage L'edge coverage è particolarmente efficace perché:

- Ha una **sensibilità bilanciata**: riesce a catturare differenze significative nell'esecuzione (⇒ maggiore efficacia nel testing)
- Comporta un **overhead contenuto**: l'impatto sul tempo di esecuzione è ridotto (⇒ throughput elevato)

Problemi da evitare

- Un feedback **troppo sensibile** causa effetti di *queue explosion*: troppe varianti simili vengono considerate interessanti, saturando la coda
- Un feedback **troppo costoso** da raccogliere riduce drasticamente il throughput di fuzzing

7.4 Fuzzer in azione

7.4.1 American Fuzzy Lop (AFL)

AFL (American Fuzzy Lop) è il più famoso *coverage-guided fuzzer*. È altamente ottimizzato, raccoglie *edge coverage* come feedback e lo utilizza anche per guidare la selezione degli input dalla coda per ulteriori mutazioni. L'ultimo aggiornamento risale al 2017, ma molti fuzzer più recenti si basano sulle idee e sul design di AFL. AFL mantiene una *coverage map* di 2^{16} elementi (64KB). Lo schema di indicizzazione degli edge è:

$$\texttt{cur_block} \wedge (\texttt{prev_block} \gg 1)$$

L'idea è di semplificare il calcolo dell'identificatore dell'edge e distribuire l'output XOR uniformemente. Tuttavia, spesso si verificano comunque collisioni. Una mappa più ampia aiuta a ridurre questo problema, ma una mappa più grande della cache L2 (256KB) compromette l'efficienza.

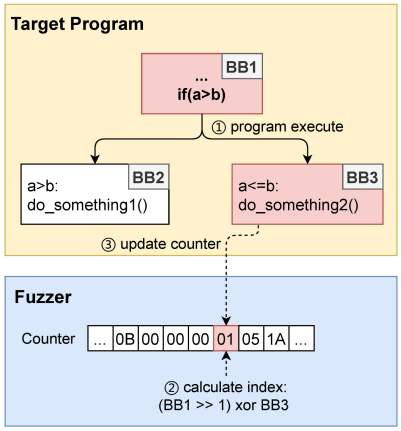


Figura 17: Coverage map in AFL

7.4.2 AFL++

AFLplusplus è la "figlia" di AFL. È stato creato inizialmente per incorporare tutte le migliori caratteristiche sviluppate negli anni per i fuzzer della famiglia AFL. Alcuni dei benefici:

- Numerosi backend per strumentazione del codice basata su sorgente e binaria
- Supporto per diverse funzioni di mutazione e scheduler
- Utility per l'elaborazione della coda

Più recentemente, `libAFL` ha introdotto ulteriore modularità per l'implementazione dei fuzzer, anche se attualmente è meno performante rispetto ad AFL++.

AFL++ in azione Utilizzeremo AFL++ nelle nostre sessioni pratiche. La documentazione originale di AFL relativa all'interfaccia di stato è disponibile su Github.

7.4.3 Metriche di performance

I bug trovati (analizzando i crash a posteriori) sono la metrica di performance definitiva per un fuzzer. Tuttavia, per comprendere i fattori che influenzano le prestazioni, esistono diverse metriche utili:

- **Throughput** (esec/sec): casi di test completati nell'unità di tempo. Un throughput elevato è importante perché più esecuzioni significano più possibilità di trovare un bug. Tuttavia, esecuzioni molto brevi possono essere fuorvianti (ad esempio, input rifiutati immediatamente).
- **Dimensione della coda** ("total paths"): casi di test distinti generati interessanti. Una coda troppo piccola suggerisce un'esplorazione superficiale, mentre una troppo grande non darà al fuzzer abbastanza tempo per mutare tutti gli input interessanti. Per questo motivo, anche il numero di cicli completati sulla coda è importante.
- **Coverage**: numero di edge coperti nel programma. Questa metrica è importante perché indica quanto del programma è stato esplorato e quali parti (ad esempio, rispetto a un altro fuzzer).

Tutte e tre queste metriche dipendono dal programma specifico. Le prime due vengono calcolate direttamente da AFL++, mentre la terza può essere derivata elaborando la coda, perché ogni input nella coda rappresenta un percorso unico che ha coperto nuovi edge nel programma; quindi, analizzando tutti gli input presenti nella coda, si può ricostruire l'insieme totale degli edge coperti dal fuzzer.

Interpretazione delle metriche Quando si analizzano le metriche di un fuzzer, è importante considerare:

- Un **throughput molto basso** (es. 5 esecuzioni/secondo) spesso indica problemi di setup, come un harness inefficiente o seed troppo grandi.
- Un **throughput molto alto** ma con **pochi input nella coda** suggerisce che il fuzzer sta respingendo quasi tutti gli input, indicando che non riesce a superare i controlli iniziali del programma.
- Un **numero eccessivo di cicli** completati in poco tempo è un segnale che qualcosa non va (in una campagna normale, un ciclo dovrebbe richiedere ore, non secondi).
- Una **coda stagnante** che non cresce nel tempo è un altro indicatore di problemi, potenzialmente con i seed iniziali.

Fuzzing roadblock Si parla di "fuzzing roadblock" quando il programma presenta sezioni difficili da superare per il fuzzer, come:

- Controlli di formato stringenti che richiedono sequenze specifiche di byte
- Confronti con stringhe o valori magici lunghi
- Checksum o hash verification

In questi casi, potrebbe essere **necessario fornire seed** che già contengono le sequenze specifiche richieste o utilizzare tecniche avanzate di mutazione.

7.4.4 Output di una campagna di fuzzing con AFL++

I fuzzer derivati da AFL distinguono tra:

- `crashes/` - crash del programma
- `hangs/` - esecuzioni che non terminano (timeout)

- `queue/` - casi di test

Per la coda, mantiene dinamicamente un insieme di casi di test non ridondanti che cumulativamente coprono tutti gli edge incontrati nella mappa fino a quel momento. Questo è importante per **mantenere efficiente il processo di fuzzing, evitando di ripetere test che coprono gli stessi percorsi** di esecuzione già esplorati.

Durante l'esecuzione, AFL++ mantiene un insieme di "input favoriti" *(favored inputs)* che rappresenta un sottoinsieme più piccolo di input che copre tutti gli archi raggiunti. Questo permette di concentrare gli sforzi di fuzzing sugli input più promettenti, migliorando l'efficienza complessiva.

Favored inputs e ottimizzazione AFL++ non seleziona i favored inputs solo per coprire tutti gli archi, ma sceglie anche:

- I test case più **piccoli e veloci** che coprono ciascun arco
- Approssimativamente risolve il problema del set cover (NP-completo) con un'euristica efficace

Durante il fuzzing, AFL++ dedica circa il 95% del tempo a mutare i favored inputs, mentre il restante 5% viene usato per esplorare altri input nella coda. Questo approccio bilancia l'esplorazione di input promettenti con la possibilità di scoprire nuovi percorsi inaspettati.

Strumenti per l'analisi della coda Per analizzare e ottimizzare la coda di AFL++, sono disponibili diversi strumenti:

- `afl-showmap`: mostra gli identificatori degli archi coperti da ciascun input
- `afl-cmin`: minimizza la coda mantenendo la stessa coverage cumulativa
- `lcov` e `llvm-cov`: strumenti per visualizzare la coverage a livello di codice sorgente quando combinati con SanitizerCoverage

7.4.5 Analisi dei crash (bug triaging)

I crash sono bug, ma due crash potrebbero derivare dallo stesso bug. Il *bug triaging* è un processo che analizza i crash cercando bug unici. Poiché i crash possono essere migliaia, le tecniche automatiche possono aiutare raggruppando crash molto simili e presentando solo un sottoinsieme di essi all'utente.

Come i fuzzer derivati da AFL vedono i crash Il fuzzer considera due input di test che causano crash come diversi se esercitano coverage differenti. Tuttavia, oltre alla posizione del crash, questo include l'intera storia di esecuzione dell'input. Questa visione è ulteriormente influenzata da:

- Non-determinismo
- Comportamenti non definiti
- Collisioni di hash

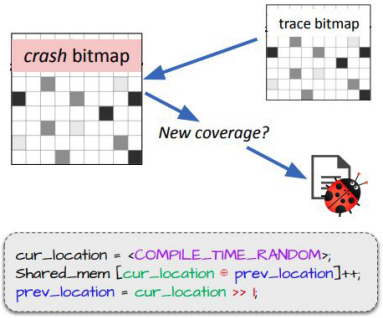


Figura 18: Come i fuzzer derivati da AFL vedono i crash

Crash deduplication Questo processo è chiamato *crash deduplication*, e un tipico criterio di clustering è l'indirizzo di crash combinato con l'identità dei primi k frame dello stack (*top-k)*.
Quando un programma va in crash, viene generato uno stack trace, cioè una lista di funzioni attive al momento del crash, partendo dalla più recente (in cima allo stack). I "primi k frame" sono le prime k funzioni in quella lista. AFL usa questi primi k frame — le prime k chiamate di funzione visibili nello stack trace — come "firma" per identificare un crash. Se due crash hanno lo stesso indirizzo di errore e gli stessi k frame iniziali, sono considerati lo stesso tipo di crash (e quindi "deduplicati").
I crash possono essere riprodotti all'interno di un debugger per facilitare l'analisi e la correzione dei bug identificati.

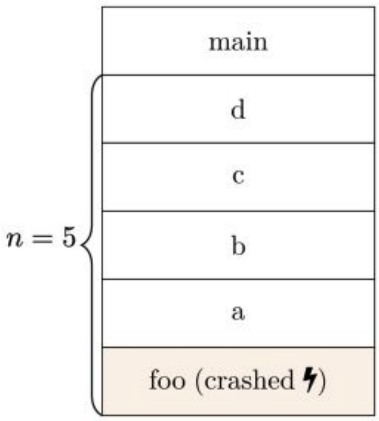


Figura 19: Crash deduplication con stack hashing

Stack hashing Per fare il triage dei crash in bug, il primo passo è solitamente raggruppare crash molto simili (logica: è probabile che siano quasi-duplicati dello stesso bug).

Per quanto riguarda il valore di k:

- Un k troppo grande porta a sovrastimare (tutti i crash vengono considerati unici)
- Un k troppo piccolo porta a sottostimare (troppo pochi crash unici)
- $k = 3$ o $k = 5$ funzionano generalmente bene nella pratica

Esistono alcuni strumenti pronti all'uso (come `AFLTriage`) per facilitare questo processo.

Dai crash raggruppati ai bug unici La deduplicazione dei crash può fornire un insieme molto più piccolo di crash unici rispetto all'output iniziale del fuzzer. Tuttavia, è qui che termina la fase di automazione: per trovare i bug, è necessario un triage manuale.

Per identificare bug unici, è necessario esaminare il codice per comprendere la causa principale di ciascun crash, che a volte può trovarsi in una regione del codice piuttosto lontana. I problemi nella **root cause analysis** includono:

- Il problema potrebbe non essere dove il programma crasha, ma in fasi precedenti dell'esecuzione
- Potrebbero esserci percorsi diversi che portano allo stesso problema
- Lo stesso bug potrebbe manifestarsi in modo diverso a seconda dell'input

Sono in corso sforzi di ricerca per automatizzare l'identificazione dei bug. Una possibilità è mutare ulteriormente i crash per identificare proprietà comuni e divergenze.

7.5 Suggerimenti pratici e metodi

7.5.1 Scelta dei seed appropriati

Le prestazioni di un fuzzer sono fortemente influenzate dalla scelta dei seed. I seed aiutano il fuzzer a superare i controlli iniziali del formato di input (e non preoccuparti: le mutazioni genereranno comunque input non validi per stressare il codice di parsing).

I **seed piccoli** vengono eseguiti velocemente e coprono il codice più superficiale, ma faticano a raggiungere codice più profondo. I **seed grandi** coprono codice più profondo, ma sono più lenti da eseguire. È possibile anche diversificare la quantità di contenuto e la correttezza del formato dei seed.

Non esiste una scelta universalmente corretta per quantità e complessità dei seed, poiché ogni programma mostrerà prestazioni diverse durante il fuzzing.

Fonti per i seed La letteratura ha esplorato diversi approcci e metodi di costruzione:

- Quando possibile, si possono ottenere file minimi per il formato di input dato: tali esempi esistono per la maggior parte dei formati comuni e gli utenti di fuzzing li condividono.
- Seed più complessi possono derivare da una campagna di fuzzing conclusa, utilizzando tecniche che minimizzano i seed sia per lunghezza che per quantità.
- Con software sconosciuto, si può tentare il reverse engineering del codice per abbozzare un input iniziale che soddisfi i primi controlli fatti dal parser.
- Se si conosce il software, è possibile estrarre i test esistenti o creare input basati sulla conoscenza del formato.
- Una buona strategia è usare più seed di piccole dimensioni con diverse caratteristiche invece di un singolo seed grande.

7.5.2 Esecuzione di più prove

Il fuzzing è un processo altamente entropico: nessuna esecuzione sarà uguale a un'altra. È fortemente consigliato eseguire più prove in parallelo. A seconda del contesto, si possono eseguire prove di 24 o 48 ore e poi imparare dai risultati, se necessario, ciò che potrebbe dover essere cambiato o ottimizzato.

Se si stanno valutando più fuzzer, è importante mantenere lo stesso setup per tutti. Spesso, si possono apprezzare più fuzzer diversi.

OSS-Fuzz Un ottimo esempio di fuzzing sistematico è il progetto OSS-Fuzz[20] di Google, che:

- Testa continuamente circa 1300 progetti open source
- Esegue quotidianamente campagne di fuzzing

[20] `https://introspector.oss-fuzz.com/`

- Utilizza diversi fuzzer e configurazioni
- Ha trovato migliaia di bug in progetti importanti
- Automatizza l'intero processo, dalla scoperta al triage fino alla segnalazione agli sviluppatori

Durata delle campagne e "plateau" Per valutare correttamente un software, è consigliabile:

- Eseguire campagne di almeno 24 ore (per progetti semplici) o più (per software complesso);
- Monitorare se la campagna raggiunge un "plateau" (un punto in cui la coverage smette di crescere);
- Valutare i risultati non solo in base ai crash trovati, ma anche alla coverage raggiunta;
- Confrontare diversi run per verificare la consistenza dei risultati.

7.5.3 Applicazioni di rete

Il fuzzing di applicazioni che accettano input di rete è **più complesso** rispetto a quelle che leggono da file (o da una funzione harness con effetto simile).

L'inizializzazione e la comunicazione tramite socket darà un throughput molto scarso. Gli utenti di fuzzing possono utilizzare librerie come `preeny/desock` e `desockmulti` che sostituiscono le chiamate alle funzioni di rete con stub emulati[21] adatti al fuzzing.

Se l'applicazione è stateful, esistono tecniche speciali per il fuzzing di rete (suggerimento: insieme alla coverage del codice, è possibile tracciare le variabili di stato e i codici di risposta).

Misurazione della code coverage Il conteggio del corpus è un indicatore poco affidabile della code coverage raggiunta[22]. Con l'edge coverage basata su hash, il valore di densità della mappa di AFL++ non misura la code coverage raggiunta in modo significativo/affidabile. Con `pcguard`, abbiamo un indicatore affidabile per gli edge distinti e `afl-showmap` può mostrarli.

Per mappare le informazioni di coverage agli elementi della linea di codice sorgente, possiamo analizzare una coda utilizzando strumenti come `lcov` o `llvm-cov` in combinazione con SanitizerCoverage.

Fuzzer diversi (e prove diverse) probabilmente toccheranno codice/edge leggermente diversi.

7.5.4 Minimizzazione della coda e dei seed

Per l'analisi dei risultati o per ulteriore fuzzing, gli utenti spesso lavorano su (una collezione di) input per scopi di minimizzazione. Sono possibili due direzioni:

- **Input individuali**: noto anche come **trimming**, consiste nel rimuovere quanti più byte possibile da un input senza influenzare la coverage risultante. AFL++ lo fa anche parzialmente durante l'esecuzione. Strumento: `afl-tmin`
- **Coda**: mantenere solo un insieme minimo di input che raggiungono una coverage cumulativa pari alla coda completa. Strumento: `afl-cmin` (costoso: è un problema di set cover!)

Trimming dettagliato Il processo di trimming funziona così:

1. Si seleziona un byte o una sezione dell'input
2. Si rimuove temporaneamente e si testa il programma
3. Se la coverage risultante non cambia, quel byte o sezione è superfluo e viene rimosso permanentemente

[21] Porzione di codice utilizzata per simulare il comportamento di funzionalità software (come una routine su un sistema remoto) o l'interfaccia COM e può fungere anche da temporaneo sostituto di codice ancora da sviluppare.

[22] Perché il numero di input nel corpus non riflette necessariamente quanti percorsi o edge distinti sono stati coperti: molti input possono coprire gli stessi percorsi, quindi una coda grande non implica automaticamente una maggiore code coverage.

4. Si ripete il processo per l'intero input

Questo approccio è particolarmente utile quando si hanno input di grandi dimensioni, come file PDF o immagini, che potrebbero contenere molti dati non necessari per eseguire il percorso di codice interessante.

Seed favoriti Durante l'operazione, AFL++ mantiene un insieme di seed favoriti come un set cover approssimativo della coda e li muta per la maggior parte del tempo, migliorando così l'efficienza del processo. *Concentrarsi* sui favored seeds permette di coprire tutti gli edge scoperti con il minor numero di input, ottimizzando così le risorse e aumentando l'efficacia delle mutazioni, invece di disperdere gli sforzi su input ridondanti.

7.5.5 Fuzzing con sanitizer

I fuzzer funzionano molto bene con i sanitizer. Una grande quantità di bug di sicurezza viene oggi trovata grazie a questa combinazione.

- **ASan** è la scelta più ricorrente. Sebbene introduca un rallentamento (2x in media), la sua capacità di rilevare un'ampia gamma di errori di memoria lo rende un compromesso utile nella maggior parte degli scenari di fuzzing.
- **UBSan** è anche una buona scelta, possibilmente con qualche regolazione fine dei controlli da abilitare o disabilitare per un dato programma (nel caso presenti anti-pattern).
- **MSan** è potente ma meno popolare per ragioni di compatibilità.

Vantaggi pratici dei sanitizer Alcuni bug possano essere individuati molto più rapidamente (o addirittura solo) grazie all'uso dei sanitizer, che permettono di rilevare **errori silenti** che altrimenti passerebbero inosservati. Un vantaggio pratico di sanitizer come ASan, UBSan e MSan è che possono aiutarci anche nel triage dei crash in bug.
Ad esempio, la natura di un bug ASan (tipo di crash e sito di allocazione dell'oggetto) può darci buoni indizi. Con UBSan, il triage è ancora più immediato, poiché viene indicata direttamente la linea di codice responsabile dell'errore.
La strumentazione ASan può essere utilizzata anche per la deduplicazione dei crash: grazie alla sua strumentazione, otteniamo uno stack trace per qualsiasi crash (senza dover ricorrere a gdb).

7.5.6 Vantaggi del fuzzing guidato dalla copertura

Il fuzzing guidato dalla copertura (*coverage-guided fuzzing*) offre diversi vantaggi rispetto al fuzzing puramente casuale:

- Esplorazione più efficiente dello spazio degli input
- Maggiore probabilità di trovare bug in percorsi di esecuzione complessi
- Generazione di input minimali che provocano comportamenti interessanti

Per questo motivo, strumenti come `AFL++` sono diventati lo standard de facto per il fuzzing di software moderno.

8 Laboratorio Fuzzing

8.1 Introduzione al Fuzzing

Il fuzzing è una tecnica di testing automatizzato che consente di trovare vulnerabilità in software inserendo input casuali o malformati. Durante questo laboratorio, utilizzeremo `AFL++` (American Fuzzy Lop plus plus), un potente strumento di fuzzing open source che è considerato uno dei più efficaci nel campo.

`AFL++` è un progetto in continua crescita, sviluppato da un team che include Andrea Fioraldi, ex studente di Sapienza. Il team ha preso il fuzzer `AFL` originale e lo ha arricchito con numerose tecniche derivate dalla letteratura scientifica più recente, aggiungendo supporto per molte piattaforme, inclusi dispositivi Android e firmware.

8.2 Installazione di AFL++

Per iniziare con il fuzzing, dobbiamo installare `AFL++`. Ci sono due modi principali:

8.2.1 Installazione della versione più recente

L'approccio consigliato è scaricare e compilare la versione più recente:

```
wget https://github.com/AFLplusplus/AFLplusplus/archive/refs/tags/v4.31c.tar.gz
tar xzvf v4.31c.tar.gz
cd AFLplusplus-4.31c
sudo apt install lld-15
make PERFORMANCE=1 LLVM_CONFIG=llvm-config-15 source-only
```

Durante questa compilazione, stiamo usando:

- `PERFORMANCE=1`: ottimizza la build per la macchina specifica in uso
- `LLVM_CONFIG=llvm-config-15`: specifica la versione di LLVM da utilizzare

Installazione globale o locale Possiamo scegliere tra:

```
# Installazione globale (richiede privilegi di root)
sudo make install

# Oppure aggiungere al PATH (per uso locale)
export PATH="$PATH:/$HOME/AFLplusplus-4.31c"
```

È consigliabile esportare anche la configurazione di `clang`:

```
export LLVM_CONFIG="llvm-config-15"
```

8.2.2 Installazione tramite repository

In alternativa, per la VM LXLE BIAR 4.6, possiamo usare il gestore pacchetti:

```
sudo apt install afl++
```

Questa opzione installerà la versione 4.00c, leggermente meno recente.

8.2.3 Configurazione del kernel per il fuzzing

Per un corretto funzionamento del fuzzer, dobbiamo modificare come il kernel Linux gestisce i crash:

```
echo core | sudo tee /proc/sys/kernel/core_pattern
```

Questa configurazione è essenziale perché quando un programma crasha durante il fuzzing, il kernel normalmente creerebbe un file *core dump*. Durante il fuzzing, però, questo comportamento rallenterebbe notevolmente il processo poiché i crash saranno frequenti.
È inoltre consigliabile disabilitare il servizio `apport`:

```
sudo systemctl disable apport.service
```

8.3 Fuzzing di xpdf

Per il nostro primo esperimento useremo `xpdf`, un toolkit per visualizzare e manipolare file PDF. In particolare, ci concentreremo sul tool `pdftotext`, che estrae testo dai PDF.

8.3.1 Download e estrazione di xpdf

Scegliamo una versione più vecchia (3.02) per aumentare la probabilità di trovare bug in un tempo ragionevole:

```
wget https://dl.xpdfreader.com/old/xpdf-3.02.tar.gz
tar xvzf xpdf-3.02.tar.gz
cd xpdf-3.02/
```

8.3.2 Compilazione con strumentazione per fuzzing

Per raccogliere informazioni di copertura *(coverage information)*, dobbiamo compilare il programma con strumentazione speciale. `AFL++` offre wrapper del compilatore che aggiungono automaticamente questa strumentazione:

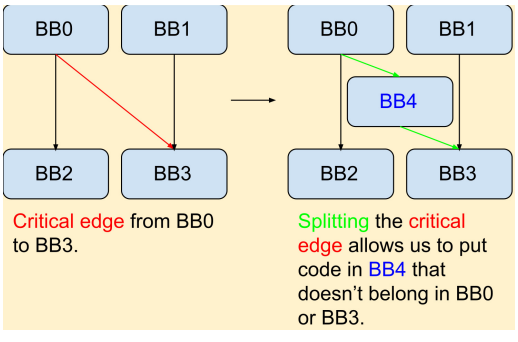
```
CC=afl-clang-fast CXX=afl-clang-fast++ ./configure \
  --prefix="$HOME/xpdf-3.02/install"
make
make install
```

Usando `afl-clang-fast` come compilatore `C` e `afl-clang-fast++` come compilatore `C++`, otteniamo la strumentazione basata su `pcguard`, che è superiore alla mappatura hash classica per il calcolo della copertura degli archi del codice.
Durante la compilazione, verranno mostrati messaggi che indicano l'aggiunta dell'istrumentazione per la copertura.

PCGuard e archi critici Il meccanismo `pcguard` utilizzato da `AFL++` è basato su un concetto chiamato "archi critici" (critical edges). A differenza delle tecniche hash-based, dove possono verificarsi collisioni tra gli archi, il meccanismo `pcguard` consente di tracciare precisamente quali percorsi di esecuzione vengono seguiti.

Un arco critico è un arco che va da un blocco che ha più di un'uscita a un blocco che ha più di un ingresso. Invece di tracciare i singoli archi (che richiederebbe memorizzare sia sorgente che destinazione), si introduce un blocco base fittizio negli archi critici. In questo modo, sapere quale blocco è stato visitato è sufficiente per dedurre quale arco è stato attraversato.

Per esempio, se abbiamo un blocco BB0 che può saltare a BB1 o BB3, e un blocco BB2 che può saltare a BB3, l'arco BB0→BB3 è critico. Inserendo un blocco BB4 su questo arco, quando si raggiunge BB3, si può dedurre il percorso: se si è passati da BB4, allora si è preso l'arco BB0→BB3, altrimenti si è preso l'arco BB2→BB3.



8.3.3 Preparazione dei seed per il fuzzing

Il fuzzer necessita di input iniziali *(seeds)* per avviare l'esplorazione. Una buona regola è scegliere file piccoli e validi:

```
mkdir seeds
cd seeds
wget https://github.com/mozilla/pdf.js-sample-files/raw/master/helloworld.pdf
wget https://www.melbpc.org.au/wp-content/uploads/2017/10/small-example-pdf-file.
    pdf
cd ..
```

8.3.4 Avvio della campagna di fuzzing

Ora possiamo avviare il fuzzing con il seguente comando:

```
afl-fuzz -i seeds/ -o queue -s 123 -- ./install/bin/pdftotext @@ tmp
```

Dove i parametri indicano:

- `-i seeds/`: directory contenente i seed
- `-o queue`: directory in cui verranno salvati i casi di test interessanti
- `-s 123`: inizializza il seed del generatore di numeri pseudo-casuali (utile per esperimenti riproducibili)
- `@@`: sintassi speciale che indica che il programma prende l'input da file
- `tmp`: file di output per `pdftotext`

Sintassi @@ La doppia chiocciola è la sintassi universale in fuzzing per indicare che l'input arriva da file e il fuzzer lo deve produrre.

8.4 Interpretazione dell'interfaccia di AFL++

Durante l'esecuzione, `AFL++` mostra un'interfaccia che fornisce informazioni in tempo reale sul processo di fuzzing. Ecco un esempio di come appare l'interfaccia dopo circa 1 ora e 27 minuti di esecuzione (ricordando che il **fuzzing non è deterministico** e quindi i dati cambiano da esecuzione a esecuzione):

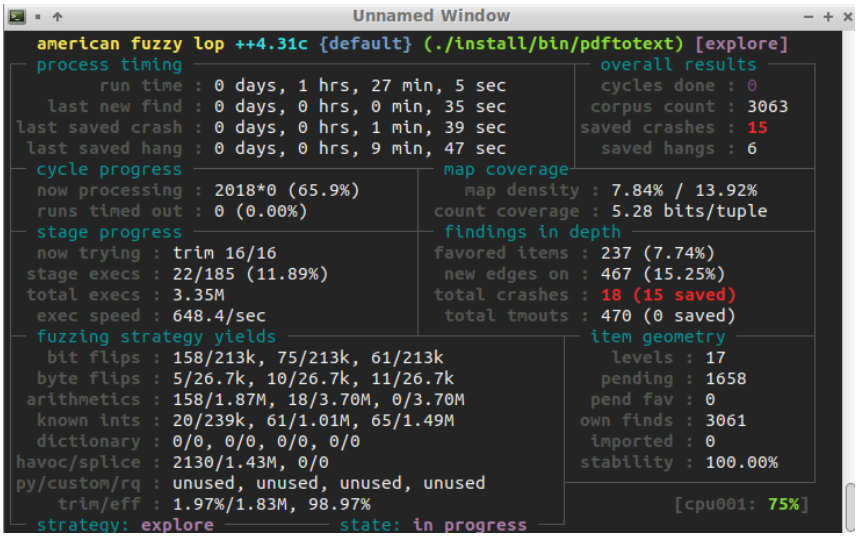


Figura 20: Interfaccia di `AFL++` durante l'esecuzione

Analisi dell'interfaccia L'interfaccia di `AFL++` è organizzata in diverse aree:

1. **Intestazione e processo:** In alto, con il testo in vari colori:
   - american fuzzy lop ++4.31c (default) (./install/bin/pdftotext) [explore]
2. **Informazioni temporali:** Nella sezione `"process timing"`:
   - run time: 0 days, 1 hrs, 27 min, 5 sec
   - last new find: 0 days, 0 hrs, 0 min, 35 sec
   - last saved crash: 0 days, 0 hrs, 1 min, 39 sec
   - last saved hang: 0 days, 0 hrs, 9 min, 47 sec
3. **Risultati complessivi:** Nella sezione `"overall results"`:
   - cycles done: 0
   - corpus count: 3063
   - saved crashes: 15
   - saved hangs: 6
4. **Progressi di ciclo e copertura:** Include nella sezione `"cycle progress"`:
   - now processing: 2018*0 (65.9%)
   - map density: 7.84% / 13.92%
   - count coverage: 5.28 bits/tuple
5. **Progressi di fase e risultati:** Include in `"stage progress"` e `"findings in depth"`:
   - now trying: trim 16/16
   - favored items: 237 (7.74%)
   - new edges on: 467 (15.25%)
   - total crashes: 18 (15 saved)
6. **Velocità di esecuzione e utilizzo CPU:** Include:
   - exec speed: 648.4/sec
   - [cpu:33%] per il processo AFL
   - [cpu:75%] per l'utilizzo complessivo
7. **Strategie di fuzzing e rendimenti:** Mostra statistiche sulle varie tecniche di mutazione:
   - bit flips, byte flips, arithmetics, known ints, havoc/splice
   - strategy: explore - indica che il fuzzer sta cercando di esplorare più codice

8.5 Struttura della coda di AFL++

Dopo l'esecuzione, `AFL++` crea una struttura di directory nella cartella di output specificata (nel nostro caso `queue`), mentre `hangs` identifica che il programma target non termina entro il timeout previsto, bloccandosi o restando in attesa indefinita durante l'esecuzione di un input generato dal fuzzer, dato da cause come loop infiniti, operazioni di I/O bloccanti, deadlock o condizioni di attesa non gestite. La directory risulta essere:

```
cmdline  fastresume.bin  fuzzer_setup  hangs      queue
crashes  fuzz_bitmap     fuzzer_stats  plot_data  target_hash
```

Il file `fastresume.bin` viene creato solo quando si interrompe un'esplorazione, per riprendere un'esplorazione interrotta.
All'interno della cartella `crashes` possiamo vedere i file che hanno causato crash:

```
id:000000,sig:11,src:000001,time:247232,execs:202709,op:havoc,rep:4
id:000001,sig:11,src:001737,time:482231,execs:342078,op:havoc,rep:4
id:000002,sig:11,src:001698,time:641335,execs:423493,op:havoc,rep:4
README.txt
```

In questi nomi di file possiamo riconoscere:

- `id:000000` - Identificatore sequenziale del crash
- `sig:11` - Segnale che ha causato il crash (`11 = SIGSEGV`, Segmentation Fault)
- `src:000001` - Test case di origine da cui è stato derivato
- `time:247232` - Timestamp relativo in secondi
- `execs:202709` - Numero di esecuzioni prima di trovare questo crash
- `op:havoc` - Operazione di mutazione che ha generato questo input
- `rep:4` - Valore di ripetizione dell'operazione

Il file `README.txt` contiene informazioni sulla command line usata per il fuzzing, utile per l'analisi dei bug.

8.6 Strategie di mutazione

`AFL++` utilizza diverse strategie per mutare gli input:

- `bit flip`: modifica singoli bit nell'input
- `byte flip`: modifica byte interi
- `arith`: operazioni aritmetiche sui byte (es. `+1`, `-1`)
- `interest`: sostituisce byte con valori di interesse (es. `0`, `-1`, `MAX_INT`)
- `havoc`: combinazione di mutazioni aggressive e casuali
- `splice`: unisce pezzi di input diversi
- `trim`: riduce la dimensione degli input mantenendo la stessa copertura

Dictionary `AFL++` può utilizzare un dizionario contenente token specifici del formato. Per esempio, per fuzzing di eseguibili, potrebbe includere le lettere `"MZ"` che sono sempre presenti nei primi due byte di un file PE[23]. Questo aiuta il fuzzer quando il formato richiede token specifici.

Fasi di esplorazione Il fuzzing alterna due fasi principali:

- `exploration`: cerca di raggiungere più codice nuovo
- `exploitation`: analizza in profondità il codice già scoperto

8.7 Analisi dei crash

Quando `AFL++` trova un crash, lo salva nella directory `crashes`.

[23] PE (Portable Executable) sono il formato standard dei file eseguibili su Windows.

8.7.1 Tempo di esposizione (TTE)

Il tempo di esposizione (Time-To-Exposure, TTE) è il tempo necessario al fuzzer per trovare un bug. Questo valore varia significativamente tra diverse esecuzioni anche con lo stesso seed, a causa della **natura non deterministica** del fuzzing.
Nel caso di `xpdf`, durante l'esecuzione di test, il primo crash è stato trovato dopo circa 4 minuti, i primi 5 crash in circa 30 minuti, e i primi 15 in meno di un'ora e mezza. Questa variabilità è normale e per questo motivo si eseguono spesso più istanze di fuzzing in parallelo.

Solo test per copertura I fuzzer possono solo distinguere i casi di test, compresi gli arresti anomali, per la copertura che inducono. Quando la coda contiene più arresti anomali, possono essere manifestazioni diverse di un numero inferiore di bug. L'ispezione manuale è l'unico modo per discriminare i bug tra gli arresti anomali.

8.7.2 Preparazione per il debug

Per analizzare i crash, è utile compilare una versione del programma più adatta al debug (senza ottimizzazioni e coverage instrumentation[24]):

```
tar xzvf ../xpdf-3.02.tar.gz
mv xpdf-3.02 debugging
cd debugging
CC=clang-15 CXX=clang++-15 CFLAGS="-g -O0" CXXFLAGS="-g -O0" ./configure
make
```

Questa versione include simboli di debug (`-g`) e disabilita le ottimizzazioni (`-O0`), rendendo più semplice l'analisi.
Per brevità, evitiamo un'installazione locale e prendiamo il binario direttamente dalla cartella xpdf:

```
biar@biar:~/xpdf-3.02/debugging$ ls xpdf/
aconf.in      aconf-win32.h  configure      fofi         Makefile.in   vms_make.com
aconf-dj.h    ANNOUNCE       configure.in   INSTALL      README        xpdf
aconf-dj.in   ChangeLog      COPYRIGHT      install-sh   splash
aconf.h       config.log     dj_make.bat    Makefile     splash-pbm.in

biar@biar:~/xpdf-3.02/debugging$ ls xpdf/pdftotext
xpdf/pdftotext
```

8.7.3 Debug dei crash con GDB

Per ispezionare un crash specifico:

```
cd xpdf/
gdb --args ./pdftotext ../../queue/default/crashes/id\:000000* tmp
```

Il carattere `*` nella command line è perché il nome varia tra campagne, ma la numerazione nel campo id dovrebbe essere preservata (es. 000000, 000001, 000002, ecc.).
Dopo aver lanciato `GDB`, eseguiamo il programma col comando `run`:

```
biar@biar:~/xpdf-3.02/debugging/xpdf$ gdb --args ./pdftotext ../../queue/default/
    crashes/id\:000000* tmp
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
...

(gdb) run
Starting program: /home/biar/xpdf-3.02/debugging/xpdf/pdftotext ../../queue/default
    /crashes/id:000000,sig:11,src:000001,time:247232,execs:202709,op:havoc,rep:4
    tmp
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
```

[24] Codice aggiuntivo nel programma per tracciare l'esecuzione e guidare il fuzzer verso percorsi nuovi o inesplorati

```
0x00007ffff7c3bc80 in __int_malloc (av=0x7ffff7c3bc80 <_main_arena>, bytes=7) at ./
    malloc/malloc.c:3982
3982    ./malloc/malloc.c: No such file or directory.
(gdb)
```

Nella *dry-run*[25] di questo laboratorio, il primo crash ha prodotto un dettagliato stack trace, di cui vediamo i primi frame:

```
#0  0x00007ffff7ae4e57 in __int_malloc (av=av@entry=0x7ffff7c5bc80 <main_arena>,
    bytes=bytes@entry=7) at ./malloc/malloc.c:3982
#1  0x00007ffff7ae6262 in __GI_libc_malloc (bytes=7) at ./malloc/malloc.c:3321
#2  0x00005555556285d8 in gmalloc (size=7) at gmen.cc:97
#3  0x000055555562881f in copyString (s=0x555555678f14 "Filter") at gmem.cc:261
#4  0x0000555555fac10 in Object::initName (this=0x5555556790b8,
    nameA=0x555555678f14 "Filter") at /home/biar/xpdf-3.02/debugging/xpdf/Object.h:
    93
#5  0x00005555555f6877 in Lexer::getObj (this=0x555555678ef0, obj=0x5555556790b8)
    at ./xpdf/Lexer.cc:344
#6  0x00005555555ffc2f in Parser::shift (this=0x555555679060) at Parser.cc:226
#7  0x000055555555ff621 in Parser::getObj (this=0x555555679060,
    objNum=0x7fffffff320, fileKey=0x0, encAlgorithm=cryptRC4, keyLength=21845,
    objNum=7, objGen=0) at Parser.cc:111
#8  0x0000555555ff41e in Parser::getObj (this=0x555555679060,
    objNum=0x7fffffff4f0, fileKey=0x0, encAlgorithm=cryptRC4, keyLength=21845,
    objNum=7, objGen=0) at Parser.cc:85
#9  0x0000555555623714 in XRef::fetch (this=0x5555556bf5b0, num=7, gen=0,
    obj=0x7fffffff4f0) at XRef.cc:823
#10 0x00005555555fa57e in Object::fetch (this=0x555555678c88,
    xref=0x5555556bf5b0, obj=0x7fffffff4f0) at Object.cc:106
#11 0x000055555555c94c in Dict::lookup (this=0x555555678c30,
    key=0x5555564ca5f "Length", obj=0x7fffffff4f0) at Dict.cc:76
#12 0x00005555555fb269 in Object::dictLookup (this=0x7fffffff770,
    key=0x5555564ca5f "Length", obj=0x7fffffff4f0)
    at /home/biar/xpdf-3.02/debugging/xpdf/Object.h:253
#13 0x00005555555ff8f0 in Parser::makeStream (this=0x555555678bb0,
    dict=0x7fffffff770, fileKey=0x0, encAlgorithm=cryptRC4, keyLength=21845,
--Type <RET> for more, q to quit, c to continue without paging--
```

8.7.4 Clustering dei crash

I fuzzer distinguono i crash solo in base alla copertura del codice che inducono. Più crash potrebbero essere manifestazioni dello stesso bug.

Criterio per clustering Un criterio comune per raggruppare i crash è considerare simili quelli che hanno lo stesso indirizzo di crash e gli stessi *top-k frame non di libreria nello stack*. Solitamente, un valore di `k=5` è un buon compromesso.

È possibile scrivere script che automatizzino questo processo di clustering, confrontando i primi 5 frame dello stack trace di ciascun crash.

Si noti che il punto in cui il programma si blocca potrebbe essere distante da dove si verifica il problema e solo l'analisi manuale del codice può discriminare accuratamente la natura di un bug (e classificare i crash in bug).

8.8 Utilizzo di sanitizer per il debug

I sanitizer come `AddressSanitizer` possono fornire informazioni aggiuntive sui bug:

```
make clean
CC=clang-15 CXX=clang++-15 CFLAGS="-g -O0 -fsanitize=address" \
CXXFLAGS="-g -O0 -fsanitize=address" ./configure
make
```

Con `AddressSanitizer` attivato, possiamo ottenere dettagli più precisi sui problemi di memoria come use-after-free, buffer overflow, accessi a puntatori nulli, ecc. Queste informazioni includono:

[25] Esecuzione simulata senza effetti permanenti, usata per eseguire il debugging o testare scenari.

- Il tipo specifico di errore di memoria
- Lo stack trace completo al momento dell'errore
- Indirizzi di memoria coinvolti

Valgrind per analisi avanzata In alternativa, è possibile usare Valgrind per identificare altri tipi di errori, come l'uso di memoria non inizializzata:

```
CC=clang-15 CXX=clang++-15 CFLAGS="-g -O0 -gdwarf-4" \
CXXFLAGS="-g -O0 -gdwarf-4" ./configure
make
valgrind ./pdftotext crashes/id\:000000* tmp
```

È necessario aggiungere `-gdwarf-4` poiché Valgrind potrebbe non supportare il formato DWARF più recente utilizzato da clang-15.

8.9 Considerazioni sulla performance

CPU cores È importante fornire al fuzzer un numero adeguato di core CPU. Si consiglia di associare ogni istanza del fuzzer a un core specifico per evitare migrazioni che possono degradare le performance.

```
taskset -c 0 afl-fuzz [opzioni] -- ./target
```

Persistent mode Per aumentare le performance, `AFL++` offre una modalità persistente che evita di ricreare processi per ogni esecuzione, aumentando significativamente la velocità di esecuzione.

Optimizations L'uso di Link Time Optimization (LTO) può migliorare ulteriormente le performance del fuzzing:

```
CC=afl-clang-lto CXX=afl-clang-lto++ ./configure
```

8.10 Fuzzing di Network Services

Il fuzzing di servizi di rete è un campo attivo di ricerca, ma presenta sfide specifiche a causa delle latenze di rete e dello stato mantenuto tra le connessioni. Alcune soluzioni includono:

- `AFL-NET`: un fuzzer specifico per protocolli di rete
- Driver di rete speciali che simulano l'instaurazione di connessioni
- Feedback aggiuntivi oltre alla copertura del codice, come i codici di risposta del server
- Tracciamento delle variabili che mantengono lo stato interno del server

Tipicamente, quando si esegue il fuzzing di applicazioni client-server, si tende a fuzzare una parte alla volta, non entrambe contemporaneamente.

Fuzzing di UI Il fuzzing con interfaccia utente è una sfida: generalmente si creano *harness*[26] appositi per automatizzare l'interazione con l'UI e testare la sua resilienza mediante input casuali o malformati.

[26] Un harness per l'UI permette di interagire automaticamente con l'interfaccia utente dell'applicazione, generando input (spesso casuali o malformati) per testare la robustezza dell'applicazione stessa.

8.11 Segnalazione etica dei bug

Quando si trovano bug, specialmente in software recenti, è importante seguire pratiche etiche:

- Verificare se il bug esiste nell'ultima versione del software
- Controllare la pagina del progetto per le linee guida sulla segnalazione dei bug
- Cercare un file `SECURITY.md` che potrebbe contenere istruzioni specifiche
- Fornire informazioni dettagliate sulla riproduzione del bug, incluso come è stato compilato il binario
- Seguire le procedure di disclosure responsabile, utilizzando canali privati quando appropriato

Anche se un bug potrebbe sembrare banale (come un crash con input malformato), potrebbe rappresentare una vulnerabilità di sicurezza significativa, specialmente se il software viene utilizzato come libreria in altre applicazioni.

9 Altro laboratorio fuzzing

Questa lezione di laboratorio si concentra su tecniche avanzate di fuzzing, in particolare:

- Fuzzing con AddressSanitizer (ASan) per migliorare la capacità di bug finding
- Clustering automatico dei crashes per deduplificazione
- Analisi della code coverage per valutare la qualità dell'esplorazione
- Monitoraggio della progressione del fuzzer nel tempo

Il target utilizzato è `xpdf-3.02`, una versione vulnerabile del visualizzatore PDF.

Prerequisiti e Setup Prima di iniziare è necessario avere:

- AFL++ installato e configurato
- LLVM 15 per gli strumenti di coverage
- Rust e Cargo per AFLTriage
- `lcov` per la visualizzazione della coverage

Script di ambiente Per facilitare il lavoro, creare uno script `fuzzenv.sh`:

```
export LLVM_CONFIG=llvm-config-15
export PATH="$PATH:/$HOME/AFLplusplus-4.31c"
```

Utilizzare con: `source fuzzenv.sh` e ogni volta che si apre una nuova shell.

9.1 Fuzzing con AddressSanitizer

Imposteremo una campagna di fuzzing con ASan abilitato

9.1.1 Motivazioni

AddressSanitizer (ASan) migliora significativamente la capacità di bug finding perché:

- **Intercetta violazioni prima del crash**: rileva errori di memoria che potrebbero non causare crash immediati
- **Cambia l'esplorazione**: l'instrumentazione modifica leggermente il comportamento del programma, portando a diverse parti di codice
- **Riduce i falsi negativi**: molti bug di memoria vengono rilevati che altrimenti passerebbero inosservati (bug silenti)

9.1.2 Compilazione con ASan

AFL++ fornisce una scorciatoia per abilitare ASan:

```
export AFL_USE_ASAN=1
```

Importante: questa variabile va impostata solo durante la compilazione, non durante il fuzzing.

9.1.3 Procedura di compilazione

1. Eseguiamo una compilazione di xpdf per il fuzzing con ASan. Per prima cosa prepariamo l'albero sorgente::

```
mkdir xpdf-asan && cd xpdf-asan
tar xzvf $HOME/xpdf-3.02.tar.gz
mv xpdf-3.02/* . && rmdir xpdf-3.02
```

2. Compilare con ASan:

```
export AFL_USE_ASAN=1
CC=afl-clang-fast CXX=afl-clang-fast++ ./configure --prefix=$PWD/install
make
make install
```

9.1.4 Verifica dell'instrumentazione

Durante la compilazione, AFL++ mostrerà messaggi come:

- `sanitizer coverage`
- `instrumented blocks/edges`
- `non-hardened asan mode`

9.1.5 Impatto sulle Performance

ASan introduce un overhead significativo:

- **Execution speed**: circa la metà rispetto al fuzzing normale
- **Causa**: ogni memory read/write richiede aggiornamento della shadow memory
- **Verifica**: controlla validità del byte, se invalido genera eccezione

9.1.6 Lancio del Fuzzing

Avremo bisogno anche di una cartella con i seed. Se non li hai a portata di mano, occorre:

```
# Preparare i seeds
mkdir seeds
cd seeds
wget https://github.com/mozilla/pdf.js-sample-files/raw/master/helloworld.pdf
wget https://www.melbpc.org.au/wp-content/uploads/2017/10/small-example-pdf-file.
    pdf
cd ..

# Avviare AFL++
afl-fuzz -i seeds/ -o queue -- ./install/bin/pdftotext @@ tmp
```

Nota: Il throughput sarà inferiore rispetto al fuzzing senza ASan, ma i crash trovati saranno più significativi.
Il front-end di AFL++ non mostrerà alcuna differenza relativa ad ASan. Tuttavia, potrai apprezzare una diversa velocità di elaborazione e i crash riscontrati saranno probabilmente diversi da prima.

Verifica della presenza di ASan nel binario Per assicurarsi che la compilazione abbia effettivamente incluso AddressSanitizer, è possibile:

- Eseguire il binario con un input che causa crash e verificare che venga mostrata una stack trace di ASan.

- Utilizzare comandi come `objdump -tC` o `nm` per cercare simboli relativi ad ASan nel binario, ad esempio:

```
objdump -tC install/bin/pdftotext | grep asan
```

- In caso di dubbi, controllare che i flag di compilazione siano corretti (`-fsanitize=address`) e che non ci siano errori di battitura.

9.2 Clustering dei Crashes

Le seguenti attività possono essere applicate a qualsiasi coda di fuzzing. Per il laboratorio di oggi, rendiamo disponibili due code di fuzzing ottenute da poche ore di fuzzing (circa 6 ore) con ASan abilitato.

9.2.1 Problema della Deduplificazione

Quando si trovano molti crashes (es. 50+), analizzarli manualmente diventa impraticabile. È necessario:

- Raggruppare crashes con cause simili
- Identificare bugs unici
- Prioritizzare l'analisi

9.2.2 Binario per il Triage

Per il clustering serve un binario con ASan ma **senza** instrumentazione di fuzzing: la strumentazione di fuzzing (come quella di AFL++) aggiunge codice per tracciare la coverage e gestire le mutazioni, che può interferire con l'analisi dei crash. Un binario con solo ASan è più "pulito" e fornisce stack trace più accurate, essenziali per il clustering dei crash. Inoltre, evita falsi positivi o comportamenti anomali causati dalla strumentazione aggiuntiva.

```
mkdir xpdf-asan-triage && cd xpdf-asan-triage
tar xzvf $HOME/xpdf-3.02.tar.gz
mv xpdf-3.02/* . && rmdir xpdf-3.02

DBGFLAGS="-g -O0 -fsanitize=address"
CC=clang-15 CXX=clang++-15 CFLAGS="$DBGFLAGS" CXXFLAGS="$DBGFLAGS" \
./configure --prefix=$PWD/install
make && make install
```

Possiamo usare il binario pdftotext da xpdf-asan-triage/install/bin per riprodurre gli input che causano crash dalla coda, ottenendo al contempo una traccia dello stack da ASan.

Consigli pratici da laboratorio

- **Organizzazione delle cartelle**: È consigliabile creare una cartella separata per ogni build (es. `xpdf-asan`, `xpdf-cov`, ecc.) invece di usare `make clean`, così da mantenere ambienti di compilazione puliti e riproducibili.
- **Gestione dei file spuri**: Nelle cartelle dei crash possono finire file non validi (ad esempio file `.txt` o file vuoti). Quando si esegue il clustering, assicurarsi di considerare solo i file di input effettivi (tipicamente quelli che iniziano con `id`).
- **Attenzione ai caratteri speciali**: Alcuni editor possono trasformare i due trattini `--` in un trattino lungo, causando errori nei comandi da terminale. Verificare sempre che i comandi copiati siano corretti.

9.2.3 AFLTriage

Alcuni strumenti possono facilitare il clustering automatico. Ad esempio, AFLTriage è un semplice strumento scritto in Rust che può automatizzare alcune attività di post-fuzzing (come il clustering). Dobbiamo prima configurare Rust:

```
# Installare Rust
sudo apt-get install cargo

# Compilare AFLTriage
cargo run
```

Possibile errore: se `cargo` si lamenta di versioni incompatibili:

```
cargo update home@0.5.11 --precise 0.5.5
```

9.2.4 Utilizzo di AFLTriage

Ora possiamo tornare alla cartella xpdf-asan-triage e deduplicare i crash:

```
afltriage -i <crashes_folder> -o ucrashes-k5 \
--bucket-strategy first_5_frames \
-- install/bin/pdftotext @@ tmp
```

Con `<crash>` come percorso alla cartella contenente i crash rilevati dal fuzzer. Nella cartella di output di ucrashes-k5, troveremo un report testuale per ogni insieme di crash raggruppati (spesso chiamati "crash unici" nella pratica) con la traccia dello stack completa e il nome di uno dei crash raggruppati, necessario per ulteriori ispezioni e follow-up.

Limitazioni Al momento, AFLTriage presenta un paio di limitazioni

- Clustering fisso a 5 frames dello stack
- Può confondere funzioni di ASan con funzioni del programma (mentre a noi interessa solo il codice del programma)

9.2.5 Versione migliorata

Esiste una versione modificata che supporta:

- `first_3_frames` per clustering più fine
- Migliore filtraggio delle funzioni ASan

Altri strumenti per il clustering dei crash Oltre ad AFLTriage, esistono altri strumenti utili per il clustering e la deduplicazione dei crash, come `casr` e `fl3g`. Questi tool possono offrire euristiche diverse (ad esempio, la severità del bug o il clustering su diversi frame della stack) e sono spesso aggiornati dalla community. In particolare, `fl3g` è uno strumento scritto in Rust che permette di deduplicare i crash in base ai primi 5 frame della stack trace, oppure secondo il nome della funzione in cui avviene il crash. In generale, è buona pratica provare più strumenti per confrontare i risultati del clustering.

9.3 Analisi della Code Coverage

9.3.1 Coverage con AFL++

AFL++ fornisce statistiche di base con `afl-showmap`. Per misurare la copertura del codice, con la strumentazione pcguard possiamo usare:

```
afl-showmap -C -i queue/default/queue -o /dev/null -- ./install/bin/pdftotext @@
    tmp
```

Questo comando mostra:

- Numero di edges/tuple coperte
- Map density (solo significativa con PCGuard instrumentation)
- Hit count massimo per edge

Esempio di output di afl-showmap: 2683 tuple trovate da 3205 input:

```
biar@biar :~ /xpdf-asan$ afl-showmap -C -i $HOME/xpdf-queues/queue/default/queue
-o /dev/null -- ./install/bin/pdftotext @@ tmp
[*] Reading from directory '/home/biar/xpdf-queues/queue/default/queue' ...
[*] Scanning '/home/biar/xpdf-queues/queue/default/queue' ..
[+] Captured 2683 tuples (map size 19990, highest value 255, total values 354522)
    in '/dev/null'.
[+] A coverage of 2683 edges were achieved out of 20032 existing (13.39%) with 3205
     input files.
```

Come riferimento, AFL++ ha riportato una densità di mappa del 13,42% prima di terminare la campagna.

9.3.2 Interpretazione dettagliata di afl-showmap

Quando eseguiamo afl-showmap, l'output fornisce informazioni preziose:

```
afl-showmap -C -i queue/default/queue -o /dev/null -- ./install/bin/pdftotext @@
    tmp
```

L'output tipico mostra:

- **Tuple trovate**: es. "2707 tuples" da 3200 input
- **Map size**: dimensione totale della mappa (es. 20.000 archi)
- **Coverage percentuale**: circa 15% nell'esempio (2707/20000)
- **Highest count**: 255 indica che almeno un edge è stato attraversato più di 255 volte
- **Anomalous entries**: contatori che hanno raggiunto il limite superiore

Limiti delle metriche di coverage È importante ricordare che la **map density** riportata da AFL++ ha significato diverso a seconda dell'instrumentazione:

- Con **Classic/Hash-based instrumentation**, la percentuale di copertura non rappresenta la reale percentuale di archi coperti, poiché la mappa ha dimensione fissa (64k) e può contenere collisioni.
- Con **PCGuard instrumentation**, la mappa è dimensionata esattamente sul numero di archi strumentati, quindi la percentuale di copertura è rappresentativa della vera edge coverage.

Per confrontare campagne diverse, è utile non solo guardare la line coverage, ma anche la branch coverage e, se possibile, confrontare direttamente gli archi coperti (ad esempio tramite `edges.txt`).

9.3.3 Salvare Identity degli Edges

Per analisi avanzate, è possibile salvare l'identità specifica degli edges:

```
afl-showmap -C -i default -o edges.txt \
-- $HOME/xpdf-asan/install/bin/pdftotext @@ outfile
```

Il file `edges.txt` conterrà tutti gli archi trovati con i relativi hit counts massimi, utile per:

- Confrontare diverse esplorazioni
- Identificare edges distintivi tra campagne
- Analizzare efficacia di strategie diverse

9.4 Coverage Dettagliata con LLVM

È possibile impostare questo strumento anche per elencare l'identità degli archi per effettuare confronti tra le code. Ma ciò che fa davvero la differenza per un utente è vedere quali elementi sorgente sono coperti.
Per raggiungere questo obiettivo, useremo llvm-cov e (un altro) binario compilato appositamente per estrarre informazioni sulla copertura da ciascun caso di test. Quindi, uniremo i risultati e calcoleremo la copertura totale del codice raggiunta dal fuzzer, mappata su specifici elementi sorgente.
Per analisi più approfondite, utilizzare gli strumenti LLVM.

9.4.1 Compilazione per Coverage

```
mkdir xpdf-cov && cd xpdf-cov
tar xzvf $HOME/xpdf-3.02.tar.gz
mv xpdf-3.02/* . && rmdir xpdf-3.02

COVERAGE="-fprofile-instr-generate -fcoverage-mapping"
CC=clang-15 CXX=clang++-15 \
CFLAGS="-g -O0 $COVERAGE" CXXFLAGS="-g -O0 $COVERAGE" \
./configure --prefix=$PWD/install
make && make install
```

9.4.2 Script per Raccolta Coverage

Useremo questa versione di xpdf per generare un profilo di esecuzione per ogni caso di test. Salva il seguente codice in uno script getCoverage.sh e rendilo eseguibile (chmod +x). Creare `getCoverage.sh`:

```
#!/bin/bash
TESTCASES="/path/to/queue/default/queue"
COVDIR="$PWD/outcov"
COVBIN="/path/to/xpdf-cov/install/bin/pdftotext"

mkdir -p $COVDIR
FILES=$(ls $TESTCASES/id*)

for FILE in $FILES[@]; do
    idx=$((idx + 1))
    name=$(basename "$FILE")
    echo "Processing testcase $idx"
    LLVM_PROFILE_FILE="$COVDIR/cvg_$name.profraw" \
    timeout 5s $COVBIN $FILE tmp
done

cd $COVDIR
ls *.profraw | xargs -n 100 -I {} \
llvm-profdata-15 merge -sparse {} -o merged_{}.profdata

cd ..
echo "Generating coverage.profdata file"
llvm-profdata-15 merge -sparse $COVDIR/merged_*.profdata \
-o $COVDIR/coverage.profdata
```

9.4.3 Generazione report

Lo strumento genererà pagine (e grafici) HTML che si potranno consultare e produrrà anche statistiche cumulative:

```
# Report testuale
llvm-cov-15 report --instr-profile \
outcov/coverage.profdata install/bin/pdftotext

# Export in formato lcov
```

```
llvm-cov-15 export --format=lcov --instr-profile \
outcov/coverage.profdata install/bin/pdftotext > lcov.dat

# Generazione HTML
sudo apt install lcov
genhtml lcov.dat -o lcovhtml/
```

```
Processing file xpdf-cov/xpdf/JPXStream.h
Processing file xpdf-cov/xpdf/JBIG2Stream.h
Processing file xpdf-cov/xpdf/Object.cc
Writing directory view page.
Overall coverage rate:
  lines ......: 19.3% (6879 of 35683 lines)
  functions ..: 28.8% (594 of 2066 functions)
```

Questo output di `genhtml` riassume la copertura ottenuta dal fuzzing:

- **Line coverage (% linee di codice eseguite)**: il 19,3% delle linee di codice è stato eseguito almeno una volta.
- **Function coverage (% funzioni eseguite)**: il 28,8% delle funzioni è stato chiamato almeno una volta.
- **Branch coverage (% percentuale di branch –condizioni– eseguite)** (non mostrata qui, ma spesso disponibile): percentuale di condizioni (if, switch, ecc.) valutate in entrambe le direzioni.

In generale, la function coverage è più alta perché basta eseguire una sola istruzione di una funzione per considerarla "coperta", mentre la line coverage richiede di toccare ogni singola riga. La branch coverage è di solito la più bassa, perché non sempre tutte le condizioni vengono esplorate in entrambi i rami. In alternativa, è possibile ottenere un'esportazione JSON sia da `llvm-cov` che da `lcov`.

Considerazioni

- È normale che la `line coverage` sia **maggiore** della `branch coverage`: si possono eseguire molte linee senza esplorare tutti i percorsi condizionali.
- Dopo 6 ore di fuzzing, questi valori indicano una buona copertura iniziale, ma c'è ancora margine per migliorare.

9.5 Progressione del fuzzer nel tempo

9.5.1 AFL-Plot

È possibile osservare più approfonditamente le prestazioni del fuzzer utilizzando afl-plot.

```
afl-plot queue/default plotdir/
```

Genera grafici che mostrano la progressione di varie metriche nel tempo. I grafici vengono salvati nella directory specificata e possono essere visualizzati aprendo `index.html`.

9.5.2 Grafico di progressione degli edges

Nella cartella di output verranno visualizzati grafici relativi a copertura del codice, throughput, ecc. nel tempo. Questo è un esempio di output da una coda generata per la precedente sessione di laboratorio: Il grafico mostra la crescita degli edges coperti nel tempo. Un plateau indica che il fuzzer ha esaurito la capacità di esplorazione con i seed e le mutazioni attuali.

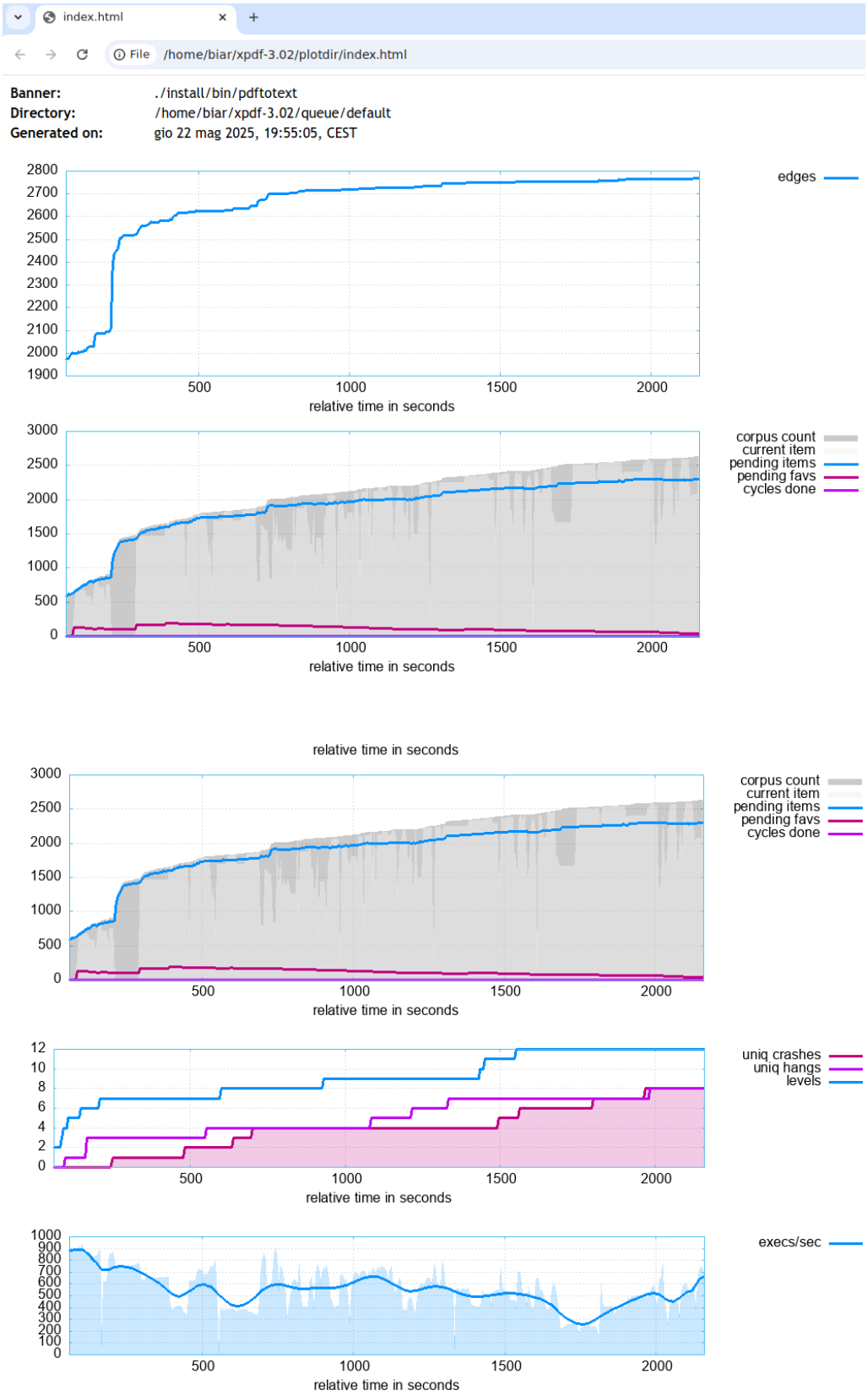


Figura 21: Progressione della scoperta di nuovi edges nel tempo durante una sessione di fuzzing di 2000 secondi

9.5.3 Interpretazione dettagliata dei grafici AFL-Plot

I grafici in figura mostrano diversi elementi informativi:

- **Header del Grafico**:
  - **Banner**: `./install/bin/pdftotext` - il comando target del fuzzing
  - **Directory**: `/home/biar/xpdf-3.02/queue/default` - percorso della coda AFL++
  - **Generated on**: timestamp di generazione del grafico (22 maggio 2025, 19:35:05 CEST)
- **Assi del Grafico**:
  - **Asse X**: tempo relativo in secondi (0-2000s, circa 33 minuti)
  - **Asse Y**: numero di edges/tuple scoperte (range 1900-2800)
  - **Linea blu**: progressione cumulativa degli edges nel tempo
- **Interpretazione della Curva**:
  - **Fase iniziale rapida** (0-500s): crescita molto veloce da 2100 a 2600 edges
  - **Rallentamento graduale** (500-1000s): la curva inizia ad appiattirsi
  - **Plateau** (1000-2000s): crescita molto lenta, indica saturazione della coverage

9.5.4 Analisi di questi grafici

Analizziamo esattamente questi grafici e vediamo che conclusioni possiamo trarne:

- **Grafico degli edges coperti nel tempo**:
  - L'asse X rappresenta il tempo (in secondi), l'asse Y il numero di edges/tuple scoperte.
  - Nella fase iniziale si osserva una crescita rapida degli edges coperti: questo indica che i seed forniti erano di buona qualità e che il fuzzer sta esplorando nuove parti del codice.
  - Dopo un certo tempo, la curva inizia ad appiattirsi (*plateau*): il fuzzer ha esplorato la maggior parte del codice raggiungibile con i seed e le mutazioni attuali. Ogni nuovo edge scoperto richiede molto più tempo.
  - **Il plateau è un segnale importante:** quando la curva si stabilizza, è il momento di valutare se introdurre nuovi seed o cambiare strategia di mutazione.
- **Grafico del corpus count e pending items**:
  - Mostra l'evoluzione della dimensione del corpus, il numero di test case ancora da processare (pending), i favorite items e i cicli completati.
  - Un corpus che cresce rapidamente indica una buona esplorazione iniziale; se il numero di pending items resta alto, il fuzzer sta ancora processando molti input nuovi.
- **Grafico dei crash e degli hang nel tempo**:
  - Traccia la scoperta di crash e hang unici. **Il numero di crash trovati non è sempre indicativo della qualità della campagna:** è fondamentale deduplicarli per identificare i veri bug unici.
- **Grafico della execution speed (exec/sec)**:
  - Mostra il throughput del fuzzer. **Fluttuazioni sono normali** e possono essere dovute alla complessità degli input o a cambiamenti nel sistema.
  - Con ASan attivo, il throughput è tipicamente dimezzato rispetto al fuzzing normale.
  - Un calo significativo e persistente può indicare problemi di sistema o input particolarmente complessi.

In cosa consiste l'interpretazione Saper leggere e interpretare questi grafici è una competenza fondamentale, anche in sede d'esame. Potrebbe essere richiesto di analizzare un grafico di progressione degli edges o del throughput e spiegare:

- Quali sono le fasi di esplorazione e di plateau.
- Cosa suggerisce la forma della curva rispetto all'efficacia dei seed e delle strategie di mutazione.
- Come interpretare la scoperta di crash e la loro deduplicazione.
- Quali azioni intraprendere quando la curva si stabilizza (ad esempio, introdurre nuovi seed, cambiare strategia, ecc.).

Inoltre, occorre non fermarsi al solo numero di crash o alla percentuale di coverage, ma di **ragionare sempre in termini di qualità dell'esplorazione** e di effettiva scoperta di nuovi percorsi nel codice.

9.5.5 Significato della progressione

1. **Efficacia dei seeds**: la rapida crescita iniziale indica che i seed forniti erano di buona qualità
2. **Exploration vs Exploitation**: il plateau suggerisce che AFL++ ha esplorato la maggior parte del codice raggiungibile
3. **Rendimenti decrescenti**: dopo 1000 secondi, ogni nuovo edge richiede molto più tempo per essere scoperto. Nei primi 1000 secondi il fuzzer trova rapidamente molti edges, poi la crescita rallenta e si stabilizza.
4. **Campagne diverse:** ad esempio, con e senza ASan possono aiutare per valutare differenze nella copertura e nella tipologia di crash.
5. **Necessità di strategie alternative**: potrebbero servire seed aggiuntivi o diverse strategie di mutazione
6. **Deduplicazione:** risulta essere fondamentale per non sovrastimare il numero di bug effettivi e identificare i veri bug distinti.

9.6 Altri grafici di AFL-Plot

AFL-Plot genera diversi grafici aggiuntivi:

9.6.1 Corpus count e pending items

Mostra l'evoluzione della dimensione del corpus, cioè l'insieme dei test case "interessanti" trovati dal fuzzer:

- **Current items**: numero totale di input unici nel corpus, cioè quelli che hanno permesso di esplorare nuovi percorsi nel programma.
- **Pending items**: input del corpus che devono ancora essere completamente processati (mutati e testati a fondo).
- **Favorite items**: sottoinsieme degli input più utili per aumentare la coverage, selezionati dal fuzzer.
- **Cycles done**: quante volte il fuzzer ha completato una scansione di tutto il corpus (un "giro" completo su tutti i test case).

9.6.2 Crashes e Hangs nel Tempo

Questo grafico mostra l'andamento temporale dei crash e degli hang scoperti durante una campagna di fuzzing:

- **Unique crashes**: numero di input distinti che causano un arresto anomalo del programma (ad esempio, un segmentation fault). Ogni crash "unico" è determinato secondo una metrica di deduplicazione (es. backtrace, offset dell'istruzione, eccezione generata) e rappresenta un potenziale bug distinto, anche se in realtà più crash potrebbero derivare dalla stessa vulnerabilità.
- **Unique hangs**: numero di input distinti che fanno entrare il programma in uno stato di stallo (ad esempio, un loop infinito o un'attesa indefinita di una risorsa), superando il timeout predefinito dal fuzzer. Anche qui, ogni hang unico indica un possibile problema specifico.
- **Mutation levels**: indica la profondità delle mutazioni necessarie per generare gli input di test. Livelli più alti corrispondono a sequenze di mutazioni più complesse partendo dagli input iniziali *(seed inputs)*.

9.6.3 Execution speed (exec/sec)

Monitora il throughput:

- **Fluttuazioni normali**: variazioni dovute a complessità diversa degli input
- **Trend decrescente**: normale con ASan per overhead crescente
- **Cali significativi**: possibili problemi di sistema o input problematici

9.6.4 Interpretazione dei grafici nel contesto ASan

Con AddressSanitizer attivo, i grafici mostrano caratteristiche specifiche:

- **Execution speed ridotta**: tipicamente 50% rispetto al fuzzing normale
- **Memory overhead**: crescita del consumo RAM nel tempo
- **Crash detection anticipata**: ASan intercetta violazioni prima del crash effettivo
- **Timing diverso**: picchi di coverage possono essere ritardati rispetto al fuzzing normale

9.7 Considerazioni Pratiche

9.7.1 Performance con ASan

- Execution speed: circa 50% rispetto al fuzzing normale
- Memory usage: significativamente maggiore
- Tempo necessario: serve più tempo per coverage equivalente

9.7.2 Confronto tra Approcci

Aspetto	Fuzzing Normale	Fuzzing + ASan
Velocità	Alta	Media (-50%)
Bug Detection	Crash evidenti	Corruption + Crash
False Negatives	Molti	Pochi
Memory Usage	Normale	Alta (+3x)

Tabella 3: Confronto tra fuzzing normale e con ASan

9.7.3 Best practices

1. **Risorse**: aumentare CPU cores disponibili (almeno 4)
2. **Clustering**: sempre dedupplicare i crashes
3. **Coverage**: analizzare per identificare aree non coperte
4. **Seeds**: utilizzare seed diversificati per migliore coverage
5. **Tempo**: pianificare campagne lunghe (24h+) con ASan

9.7.4 Troubleshooting comune

- **Compilation errors**: verificare versioni LLVM/Clang
- **Runtime errors**: controllare variabili ambiente ASan
- **Low throughput**: normale con ASan, considerare parallelizzazione
- **Memory exhaustion**: monitorare uso RAM, può crescere molto

9.8 Resume e Gestione delle Code

9.8.1 Riavvio di Campagne di Fuzzing

AFL++ permette di riprendere sessioni interrotte:

```
# Invece di -i seeds/, usare -
afl-fuzz - -o queue -- ./install/bin/pdftotext @@ tmp
```

Quando si riavvia:

- AFL++ legge il file `fuzzer_stats`
- Tenta di riprendere da `fastresume.bin` se disponibile
- Continua l'esplorazione dal punto di interruzione

Nota: il timing potrebbe non essere perfetto se la VM è stata sospesa, ma l'esplorazione continua correttamente.

9.8.2 Warning sui cicli incompleti

AFL++ può mostrare warning come:

- `Attenzione: non sono riusciti a completare il primo ciclo`
- `Potrebbe avere risultati incompleti`

Best practice: per risultati professionali, lasciare che AFL++ completi almeno un ciclo completo attraverso la coda.

9.8.3 Dettagli sui profili LLVM

I file `.profraw` generati contengono informazioni dettagliate:

- **Code coverage**: linee, funzioni, branch eseguiti
- **Frequency information**: quante volte ogni branch è stato preso
- **Hot/Cold paths**: identificazione dei percorsi più frequenti
- **Indirect calls**: tracciamento delle chiamate indirette

Utilizzi avanzati:

- **Profile-Guided Optimization (PGO)**: ricompilazione ottimizzata
- **Performance analysis**: identificazione di bottleneck
- **Fuzzing guidance**: seed selection basata su coverage

Context industriale: nei datacenter, profili simili (acquisiti via hardware sampling con tools come `perf`) vengono usati per ottimizzazioni che possono risparmiare 1-2% di energia, corrispondenti a migliaia di euro risparmiati.

9.8.4 Timeout negli script di coverage

Il `timeout 5s` nello script serve a:

- Evitare hang infiniti su input problematici
- Mantenere tempi ragionevoli per migliaia di test cases
- Gestire edge cases che potrebbero causare loop infiniti

Regolare il timeout in base a:

- Complessità del target
- Performance della macchina
- Dimensione tipica degli input

9.9 Confronto coverage tra campagne

Per confrontare l'efficacia di diverse strategie:

1. Generare edges.txt per ogni campagna
2. Utilizzare tools di diff per confrontare:

```
# Edges comuni
comm -12 edges1.txt edges2.txt

# Edges unici alla prima campagna
comm -23 edges1.txt edges2.txt

# Edges unici alla seconda campagna
comm -13 edges1.txt edges2.txt
```

3. Analizzare coverage percentuali relative

Metriche di confronto:

- **Edge coverage**: numero di archi unici
- **Line coverage**: percentuale di linee eseguite
- **Branch coverage**: copertura delle condizioni
- **Function coverage**: funzioni raggiunte

9.10 Troubleshooting AFLTriage

9.10.1 Limitazioni attuali

- **Clustering fisso**: solo `first_5_frames` nella versione standard
- **Confusione ASan**: include funzioni di sanitizer nel clustering
- **No customization**: parametri di clustering non configurabili

Soluzione: utilizzare la versione modificata che supporta `first_3_frames` e migliore filtraggio delle funzioni ASan.

9.11 Conclusioni

Il fuzzing con AddressSanitizer rappresenta un upgrade significativo rispetto al fuzzing tradizionale:

- **Migliore bug detection**: trova errori che sfuggirebbero altrimenti
- **Stack trace dettagliate**: ASan fornisce informazioni precise sui bug
- **Clustering automatico**: essenziale per gestire grandi quantità di crashes
- **Coverage analysis**: permette valutazione oggettiva della qualità del test
- **Monitoraggio progressione**: i grafici AFL-Plot guidano l'ottimizzazione delle campagne

L'overhead in termini di performance è compensato dalla qualità superiore dei risultati, rendendo questa tecnica essenziale per la ricerca di vulnerabilità in software crítico.

Raccomandazioni finali Si raccomanda quindi di:

1. Pianificare campagne lunghe (24h+) per massimizzare la coverage
2. Monitorare regolarmente i grafici per identificare quando la coverage raggiunge il plateau
3. Utilizzare sempre il clustering per analizzare efficientemente i crashes
4. Considerare l'uso di seed diversificati quando la coverage si stabilizza

10 Domande d'esame

10.1 Domanda 1: Clustering dei Crashes

Domanda: Descrivere il processo di clustering dei crashes contenuti all'interno della coda di un fuzzer al termine di una campagna. Discutere fattibilità e implicazioni di un possibile scenario dove, da 100 crash presenti nella coda, applicando stacktrace-based clustering si ottengono 20 crash unici con k=1, 30 con k=3, e 25 con k=5.

10.1.1 Risposta corretta

Il clustering dei crash si basa sull'analisi delle stack trace, dove si considerano simili i crash che condividono lo stesso indirizzo di crash e i primi k stack frame non appartenenti a librerie. Elementi chiave da includere nella risposta:

- In questa analisi è fondamentale il parametro di profondità con il quale si analizza la stack-trace
- Al crescere di k, dovrebbero aumentare (non diminuire) i crash unici identificati:
  - Con k=1 (criterio più severo) si distingue solo in base al crash site
  - Con k maggiore, si considerano più frame della stack trace, permettendo di distinguere crash che arrivano allo stesso punto ma da percorsi diversi
  - Valori troppo piccoli di k rischiano di sottostimare bug unici, valori grandi di sovrastimarli
- Lo scenario presentato è *inconsistente*: non è possibile avere meno crash unici (25) con k=5 rispetto a k=3 (30). I valori dovrebbero essere monotoni crescenti.
  - L'unica spiegazione plausibile sarebbe un non determinismo nell'esecuzione, ma questo richiederebbe ripetute deduplicazioni

10.1.2 Errori comuni da evitare

- Non confondere la severità di k: un k più grande porta a più distinzioni, non meno
- Non fare affermazioni generiche sulla "variabilità dei risultati"
- Non parlare di "efficienza del triage" senza spiegare le implicazioni specifiche

10.2 Domanda 2: Funzionamento dei sanitizers: redzone di AddressSanitizer

Domanda: Descrivere il meccanismo di redzone usato da AddressSanitizer (ASan) e quali tipi di memory safety errors possono essere intercettati da ASan grazie alle redzones. Discutere brevemente i trade-off di accuratezza ed effetti sull'esecuzione di utilizzare una redzone sinistra e destra di 16 byte oppure una di 128 byte.

10.2.1 Risposta corretta

Le redzone sono aree di memoria "avvelenate" inserite prima e dopo gli oggetti allocati. Punti chiave:

- Le redzone servono principalmente per la spatial safety: per aumentare l'efficienza, è possibile che due porzioni di codice allocate possano condividere redzones tra di loro
- ASan sceglie automaticamente la dimensione delle redzone in base al contesto, ma questa è configurabile:
  - La dimensione predefinita può variare (tipicamente 64 byte per uso generale)

  - È possibile configurare manualmente la dimensione delle redzone
  - ASan può adattare la dimensione delle redzone in base alla dimensione dell'oggetto allocato
  - Per oggetti piccoli, può usare redzone più piccole
  - Per oggetti più grandi, può utilizzare redzone più ampie

- Per spatial safety è un tipo di approccio probabilistico.
- Permettono di rilevare buffer overflow/underflow (ma ASan non riesce a catturare tutti gli errori di overflow)
- Non sono utilizzate per rilevare temporal safety errors (use-after-free, ecc.)
- Il dimensionamento delle redzone influenza:
  - Capacità di rilevare overflow più o meno ampi
  - Overhead di memoria
  - Impatto sulle prestazioni (cache, ecc.)

Trade-off dimensioni redzone:

- 16 byte:
  - Pro: minor overhead
  - Contro: può non rilevare overflow ampi (può essere scavalcata la redzone e si può finire in parte zone di memoria valide)
- 128 byte:
  - Pro: maggior protezione
  - Contro: maggior consumo di memoria, impatto su cache, prestazioni ed efficienza

10.2.2 Casi particolari e limitazioni (Bonus)

Intra-object Spatial Safety ASan presenta una limitazione importante: è "blind" rispetto agli errori di spatial safety intra-oggetto. Questo significa che:

- I singoli campi (fields) di una struct non sono trattati come entità distinte
- L'intera struct è vista come un'unica entità contigua in memoria
- Non è possibile rilevare overflow tra campi della stessa struct

Load Widening e Interceptors Un caso particolare riguarda il load widening, dove:

- Per ottimizzazione, si possono leggere più byte di quanti effettivamente necessari
- Questo può generare falsi positivi se non gestito correttamente
- ASan utilizza interceptors per gestire questi casi
- Un caso limite sono le funzioni di libreria che fanno accessi OOB (Out Of Bounds) ma controllati, purché la pagina di memoria sia allocata

Librerie ASAN utilizza una run-time library che rimpiazza la funzione di malloc per gestire le red zones, quindi per programmi particolari che utilizzano allocatori custom è necessario stare attenti a questo elemento

10.2.3 Errori comuni da evitare

- Non confondere le redzone (meccanismo per spatial safety) con i meccanismi per temporal safety:
  - Use-after-free e altri temporal errors sono gestiti dal management della shadow memory
  - La quarantena per l'allocatore heap gestisce la temporal safety
  - Le redzone non hanno ruolo nella rilevazione di temporal errors
- Non associare la dimensione della shadow memory alle redzone:
  - La shadow memory ha dimensione fissa dall'avvio del programma
  - È dimensionata in base alla memoria utilizzabile dal programma
  - Non dipende dalla configurazione delle redzone
- Distinguere correttamente load widening e violazioni delle redzone:
  - Il load widening è una ottimizzazione legittima gestita da interceptors
  - Le violazioni delle redzone sono accessi illegali alla memoria

10.3 Domanda 3: Use of Uninitialized Memory (UUM)

Domanda: Definire cosa è un errore di tipo Use of Uninitialized Memory (UUM) e spiegare brevemente perché a differenza di ASan un errore non può essere segnalato **al momento**[27] dell'accesso a memoria. Discutere inoltre se il codice nel seguito presenta criticità di tipo UUM

10.3.1 Risposta corretta

Un errore UUM si verifica quando:

- Il programma fa una computazione che dipende da memoria non inizializzata
- La sola lettura non è sufficiente per determinare un errore UUM
- Alcune letture possono essere valide se fatte per ottimizzazione
- L'errore si manifesta solo **quando il valore letto da una memoria non inizializzata viene utilizzato per una computazione significativa.** Questo è un punto centrale nella distinzione tra un semplice accesso a memoria e un errore UUM vero e proprio.

Analisi del codice fornito Il seguente codice presenta diverse criticità relative all'uso di memoria non inizializzata:

```
void weirdStringPrinter(char* string, int len) {
    char buf[256];
    if (len>255) return;
    int* src = (int*) string;     // Conversione da char* a int*
    int* dest = (int*) buf;       // Conversione da char[] a int*
    for (int i=0; i < len/4; i++)
        dest[i] = src[i];         // Copia 4 byte alla volta
    if (len != strlen(buf)) return;
    printf("%s\n", buf);
}
```

[27] Leggere memoria non inizializzata non è sempre un errore: può essere lecito per ottimizzazioni. Diventa un bug UUM solo quando il dato letto viene usato in una computazione (es. condizioni, aritmetica, accessi a memoria). Per questo MSan usa controlli differiti ("deferred check") e la shadow propagation e traccia il dato con la shadow memory dal sito di lettura fino al suo utilizzo.

Problemi di UUM identificati

- **Cast e Load widening:**
  - La conversione da `char*` a `int*` modifica la granularità degli accessi
  - Invece di copiare un byte alla volta, il codice copia 4 byte per iterazione
  - Questo è un esempio di **load widening**[28] implicito dovuto al cast dei puntatori
- **Problemi di allineamento e padding:**
  - Se `len` non è multiplo di 4, gli ultimi 1-3 byte del buffer rimangono non inizializzati
  - Esempio: con `len=5`, verranno copiati 8 byte (due int), lasciando 3 byte non inizializzati
- **Terminatore stringa:** Non viene garantita la presenza del terminatore `null`, necessario per `strlen` e `printf`
- **UUM nella validazione:** `strlen` potrebbe leggere oltre i byte inizializzati fino a trovare un byte nullo casuale

Comportamento Il codice potrebbe funzionare correttamente solo "per caso" se:

- `len` è multiplo di 4
- Il terminatore viene copiato accidentalmente durante il load widening
- I byte non inizializzati contengono casualmente valori appropriati

[28] Il load widening è una tecnica di ottimizzazione dove si leggono più byte di quanti effettivamente necessari in una singola operazione. Per esempio, quando il processore deve leggere un singolo byte, potrebbe essere più efficiente leggere 4 byte alla volta (la dimensione naturale di un int) e poi utilizzare solo il byte necessario. Questo è un comportamento legittimo e ottimizzato, ma può creare situazioni delicate quando si parla di memory safety.

11 Tabelle riepilogative

11.1 Sanitizers

Sanitizer	Meccanismo	Errori rilevati	Classe di safety	Overhead
`AddressSanitizer (ASan)`	Usa una *shadow memory* e inserisce *redzones* per rilevare accessi errati	Buffer overflow (heap/stack), use-after-free, accessi out-of-bounds	Memory safety (spatial & parziale temporal)	**Medio** (2x-3x runtime, +2x memoria)
`Undefined Behavior Sanitizer (UBSan)`	Inserisce controlli runtime per comportamenti indefiniti del C/C++	Integer overflow, shift invalidi, dereferenziazione di puntatori nulli o disallineati	Type & undefined behavior safety	**Basso** (tipicamente < 1.5x runtime)
`MemorySanitizer (MSan)`	Tracciamento dei bit non inizializzati tramite *shadow memory*	Uso di memoria non inizializzata (UUM)	Memory initialization safety	**Alto** (3x-4x runtime, +2-3x memoria)
`ThreadSanitizer (TSan)`	Analizza le relazioni *happens-before* tra thread	Race condition e data race	Concurrency safety	**Molto alto** (5x-10x runtime)
`LeakSanitizer (LSan)`	Esegue uno scan dell'heap al termine del programma	Memory leaks (perdita di memoria non liberata)	Resource / memory leak safety	**Basso** (impatto trascurabile sul runtime)
`TypeSanitizer`	Rileva violazioni del sistema di tipi tramite aliasing checks	Violazioni di *strict aliasing*, cast illegali, type confusion	Type safety	**Medio** (dipende dal programma)
`SanitizerCoverage`	Strumenti di instrumentation per il coverage (basic block, edge)	Nessun errore diretto, solo metriche di copertura	Coverage analysis / instrumentation	**Variabile** (tipicamente basso, ma dipende dalla granularità)
`DataFlowSanitizer (DFSan)`	Tracciamento dinamico di taint (taint propagation)	Violazioni di flusso d'informazione (information leak)	Data-flow / information-flow safety	**Alto** (2x-5x runtime)

Tabella 4: Riepilogo dei principali *sanitizers* con meccanismo, errori rilevati, safety e overhead

Classi di safety:

- **Memory safety**: include sia la **spatial** che la **temporal safety**; assicura che ogni operazione su memoria sia lecita.
  - **Spatial safety**: garantisce che gli accessi alla memoria avvengano entro i limiti validi degli oggetti (es. array bounds, puntatori validi).
  - **Temporal safety**: previene accessi a memoria che è stata liberata o che non è più valida nel tempo (es. use-after-free).
- **Concurrency safety**: riguarda l'accesso concorrente sicuro a dati condivisi tra thread (es. data race).
- **Type safety**: garantisce che i dati vengano utilizzati con il tipo corretto e che i cast siano sicuri.
- **Resource safety**: assicura che le risorse (come memoria heap) vengano allocate e deallocate correttamente.
- **Data-flow safety**: controlla che il flusso di dati sensibili non violi politiche di sicurezza (es. taint tracking).

11.2 Fuzzer

Fuzzer	Categoria	Meccanismo di funzionamento	Caratteristiche chiave
AFL	Grey-box, coverage-guided	Strumentazione compile-time; feedback su *edge coverage*; ciclo mutate-execute-select con *fork server*	Mutazioni deterministiche + random, bitmap 64 k edges, corpus queue, dizionario
AFL++	Evoluzione di AFL	Backend `pcguard`, modalità persistente, scheduler migliorato	Plugin modulari, multicore, integrazione ASan, strumenti `afl-cmin/tmin`

Tabella 5: Riepilogo dei *fuzzer* citati nel documento

Metriche comunemente usate nel coverage-guided fuzzing:

- **Edge coverage**: misura il numero di transizioni tra basic block (edge nel CFG) che vengono eseguite. È la metrica principale usata da AFL/AFL++; è più sensibile del branch coverage.
- **Branch coverage**: rileva se ciascun ramo condizionale (if/switch) è stato preso almeno una volta in entrambe le direzioni (vero/falso).
- **Block coverage (basic block coverage)**: misura se ogni blocco base (sequenza lineare di istruzioni senza salti) è stato eseguito.
- **Path coverage**: tiene traccia di sequenze intere di blocchi o rami eseguiti. Ha granularità più fine, ma richiede spazio esponenziale e non è comunemente usata da AFL.
- **Instruction coverage**: percentuale di istruzioni individuali del programma eseguite almeno una volta. Più fine del basic block coverage, ma più costosa da calcolare.
- **Function coverage**: indica se ogni funzione è stata invocata almeno una volta.

AFL/AFL++ usano una bitmap per tracciare l'**edge coverage**, considerando l'esecuzione di transizioni tra blocchi, senza distinguere path completi. Questo equilibrio tra precisione e performance è alla base della loro efficacia.

Glossario

AFL (American Fuzzy Lop)	Fuzzer open source che utilizza tecniche coverage-guided per trovare bug e vulnerabilità nei programmi. 76
arbitrary write	Primitiva di attacco che consente di scrivere dati in posizioni di memoria arbitrarie. 23
ASan (AddressSanitizer)	Sanitizer che rileva errori di memoria come buffer overflow, use-after-free e accessi fuori dai limiti, tramite strumentazione del codice durante la compilazione (memory-safety, principalmente spatial, parzialmente temporal). 38, 41, 114
ASLR	Address Space Layout Randomization: tecnica di sicurezza che randomizza la disposizione degli indirizzi di memoria. 13
attack primitive	Blocco fondamentale utilizzato per costruire un exploit, come arbitrary write, arbitrary read, ecc.. 23
Black-box testing	Tecnica di testing che ignora la struttura interna del programma e si basa solo sugli input e output. 68
blue Team	Gruppo che difende il sistema dagli attacchi e risponde agli incidenti. 15
branch coverage	Metrica che misura la copertura dei rami condizionali (if, else) durante l'esecuzione dei test. 68, 75
buffer overflow	Errore che si verifica quando si scrive oltre i limiti di un buffer, causando corruzione di memoria. 25
bug	Difetto o errore in un software o sistema che provoca un risultato inaspettato o indesiderato. 8
canary	Valore sentinella usato per rilevare sovrascritture dello stack. 65
CIA	Acronimo di Confidentiality, Integrity, Availability: i tre principi fondamentali della sicurezza delle informazioni. 9
compartmentalizzazione	Suddivisione di un sistema in componenti isolati che comunicano tramite interfacce ben definite. 10
CVE	Common Vulnerabilities and Exposures: sistema di identificazione univoca delle vulnerabilità. 24
CVSS	Common Vulnerability Scoring System: sistema di classificazione della gravità delle vulnerabilità. 23
dangling pointer	Puntatore che fa riferimento a una zona di memoria non più valida. 30
disponibilità (Availability)	Proprietà che garantisce che le informazioni e i servizi siano disponibili agli utenti autorizzati quando necessario. 9

edge coverage — Metrica che misura la copertura degli archi nel grafo di controllo di flusso di un programma durante il testing. 75

exploit — Tecnica o codice che sfrutta una vulnerabilità per compromettere un sistema. 23

failure — Malfunzionamento osservabile di un sistema, spesso conseguenza di un bug. 17

fuzzing — Tecnica automatizzata di testing che prevede l'invio di input casuali o malformati a un programma. 23

Gadget — Breve sequenza di istruzioni utili, spesso terminante con ret o jmp, usata negli attacchi ROP/JOP. 60

GDB — GNU Debugger, strumento per il debug di programmi, permette di eseguire il codice passo-passo, analizzare crash e variabili. 50

Grey-box testing — Tecnica di testing che combina elementi di white-box e black-box, utilizzando feedback parziale dal programma. 68

integrità (Integrity) — Proprietà che garantisce che le informazioni non siano modificate in modo non autorizzato. 9

isolamento — Separazione dei componenti di un sistema per limitare le interazioni e aumentare la sicurezza. 10

JOP (Jump-Oriented Programming) — Tecnica di attacco che sfrutta gadget terminanti con istruzioni jmp per eseguire codice arbitrario. 60

load widening — Ottimizzazione in cui vengono letti più byte del necessario in una singola operazione, potenzialmente causando accessi fuori dai limiti. 41

memory safety — Proprietà che garantisce che i puntatori accedano solo a memoria valida. 27

modello di minaccia (Threat Model) — Processo di identificazione e valutazione delle minacce a cui un sistema è esposto. 11

MSan (MemorySanitizer) — Sanitizer che rileva l'uso di memoria non inizializzata durante l'esecuzione di un programma (temporal safety). 45, 114

path coverage — Metrica che misura la copertura di tutte le possibili sequenze di esecuzione (path) in un programma. 68

principio del minimo privilegio — Principio secondo cui ogni componente deve avere solo i privilegi strettamente necessari per svolgere la propria funzione. 10

red Team — Gruppo che simula attacchi per testare la sicurezza di un sistema. 15

redzone	Regione di memoria inattiva inserita automaticamente dal compilatore attorno a variabili (soprattutto nello stack o heap) per rilevare accessi fuori dai limiti. Le redzone vengono marcate come non valide nella *shadow memory* e permettono a strumenti come AddressSanitizer di individuare buffer overflow, overread e overwrite. 38
regression	Bug che riappare dopo essere stato precedentemente corretto, spesso a causa di modifiche al codice. 15
responsible disclosure	Procedura di segnalazione responsabile di una vulnerabilità ai fornitori o sviluppatori prima della divulgazione pubblica. 24
ret2libc	Tecnica di attacco che sfrutta funzioni già presenti nelle librerie di sistema per eseguire codice arbitrario. 60
riservatezza (Confidentiality)	Proprietà che garantisce che le informazioni siano accessibili solo a chi è autorizzato. 9
sanitizer	Strumento che rileva bug e comportamenti sospetti durante l'esecuzione di un programma. 37
shadow memory	Struttura di memoria parallela usata dai sanitizer (come ASan (AddressSanitizer) e MSan (MemorySanitizer)) per tenere traccia di metadati relativi alla memoria reale del programma. Ogni indirizzo della memoria reale è mappato in un indirizzo della shadow memory, che ne indica lo stato (es. valida, non inizializzata, redzone, freed). Serve per rilevare accessi illegali o non definiti in fase di esecuzione. 39, 114
type confusion	Errore che si verifica quando un oggetto viene trattato come se fosse di un tipo diverso da quello reale, causando vulnerabilità. 28
type safety	Proprietà che garantisce che i dati siano utilizzati solo secondo il loro tipo. 29
UBSan (Undefined Behavior Sanitizer)	Sanitizer che rileva comportamenti indefiniti nel codice, come overflow aritmetici, accessi a variabili non inizializzate, ecc.. 43
use-after-free	Errore che si verifica quando si accede a memoria già liberata, causando comportamenti indefiniti o vulnerabilità. 23, 38
Valgrind	Strumento per il debug di problemi di memoria, ricerca di memory leak e profiling del software, opera a livello binario. 37
vtable (virtual table)	Struttura dati usata nei linguaggi OOP per gestire il dispatch dinamico dei metodi virtuali. 32
vulnerabilità	Debolezza di un sistema che può essere sfruttata da un attaccante. 23

White-box testing	Tecnica di testing che analizza il codice sorgente e le condizioni interne del programma. 68